Data-driven camera calibration for light field camera systems

Generated by Doxygen 1.12.0

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Namespace Documentation

4.1 CalibrationState Namespace Reference

Classes

- class CalibrationState

4.2 Controller Namespace Reference

Classes

- class Controller

4.3 depthmap Namespace Reference

Functions

- build_matchers ()
- bool imwrite_u16 (str path, np.ndarray img_u16)
- np.ndarray disparity (np.ndarray left_gray, np.ndarray right_gray)
- np.ndarray to_u16_from_disp (np.ndarray disp)
- np.ndarray fuse_disparities_median (list disp_list)
- Dict[str, np.ndarray] generate_depthmaps_for_pose (Dict[str, str] cam_to_focus_path)

Variables

- _LEFT_MATCHER
- _RIGHT_MATCHER
- _WLS

4.3.1 Function Documentation

4.3.1.1 build_matchers()

```
depthmap.build_matchers ()
```

@brief Build stereo matchers with SGBM and optional Weighted Least Squares (WLS) filtering.

@return Tuple (left_matcher, right_matcher, wls_filter)

Definition at line 11 of file depthmap.py.

```
00011 def build_matchers():
00012     """
00013     @brief Build stereo matchers with SGBM and optional Weighted Least Squares (WLS) filtering.
00014
00015     @return Tuple (left_matcher, right_matcher, wls_filter)
00016     """
00017     left_matcher = cv2.StereoSGBM_create(
00018         minDisparity=0,
00019         numDisparities=128,  # must be divisible by 16
00020         blockSize=7,
00021         P1=8 * 7 * 7,
00022         P2=32 * 7 * 7,
00023         uniquenessRatio=5,
00024         speckleWindowSize=100,
00025         speckleRange=2,
00026         disp12MaxDiff=1,
00027         preFilterCap=31,
00028         mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
00029     )
00030
00031     right_matcher = None
00032     wls = None
00033
00034     try:
00035         if hasattr(cv2, "ximgproc"):
00036             right_matcher = cv2.ximgproc.createRightMatcher(left_matcher)
00037             wls = cv2.ximgproc.createDisparityWLSFilter(left_matcher)
00038             wls.setLambda(6000)
00039             wls.setSigmaColor(1.0)
00040     except Exception:
00041         right_matcher = None
00042         wls = None
00043
00044     return left_matcher, right_matcher, wls
00045
00046
```

4.3.1.2 disparity()

```
np.ndarray depthmap.disparity (
              np.ndarray left_gray,
              np.ndarray right_gray)
```

@brief Compute stereo disparity map with optional WLS filtering.

@param left_gray Left grayscale image
@param right_gray Right grayscale image
@return Float32 disparity map with NaNs for invalid pixels (<=0)

Definition at line 64 of file depthmap.py.

```
00064 def disparity(left_gray: np.ndarray, right_gray: np.ndarray) -> np.ndarray:
00065     """
00066     @brief Compute stereo disparity map with optional WLS filtering.
00067
00068     @param left_gray Left grayscale image
00069     @param right_gray Right grayscale image
00070     @return Float32 disparity map with NaNs for invalid pixels (<=0)
00071     """
```

```
00072     dl = _LEFT_MATCHER.compute(left_gray, right_gray)  # int16 scaled by 16
00073
00074     if _RIGHT_MATCHER is not None and _WLS is not None:
00075         dr = _RIGHT_MATCHER.compute(right_gray, left_gray)
00076         d = _WLS.filter(dl, left_gray, None, dr)
00077     else:
00078         d = dl
00079
00080     d = d.astype(np.float32) / 16.0
00081     d[d <= 0] = np.nan
00082     return d
00083
00084
```

4.3.1.3 fuse_disparities_median()

```
np.ndarray depthmap.fuse_disparities_median (
             list disp_list)
```

```
@brief Fuse multiple disparity maps robustly using median.

@param disp_list List of float32 disparity maps to fuse
@return Float32 fused disparity map with 0 where no valid data exists
```

Definition at line 103 of file depthmap.py.

```
00103 def fuse_disparities_median(disp_list: list) -> np.ndarray:
00104     """
00105     @brief Fuse multiple disparity maps robustly using median.
00106
00107     @param disp_list List of float32 disparity maps to fuse
00108     @return Float32 fused disparity map with 0 where no valid data exists
00109     """
00110     stack = np.stack(disp_list, axis=0)  # (K,H,W)
00111
00112     # Locations where at least one value is valid
00113     valid_any = np.any(~np.isnan(stack), axis=0)
00114
00115     # Initialize result with zeros
00116     disp_final = np.zeros(stack.shape[1:], dtype=np.float32)
00117
00118     # Compute median only where we have valid data
00119     if np.any(valid_any):
00120         disp_final[valid_any] = np.nanmedian(stack[:, valid_any], axis=0)
00121
00122     # Remove non-positive disparities
00123     disp_final[disp_final <= 0] = 0.0
00124     return disp_final
00125
00126
```

4.3.1.4 generate_depthmaps_for_pose()

```
Dict[str, np.ndarray] depthmap.generate_depthmaps_for_pose (
             Dict[str, str] cam_to_focus_path)
```

```
@brief Generate depth maps for all cameras in a multi-camera pose.

@param cam_to_focus_path Dictionary mapping camera IDs to image file paths
@return Dictionary mapping camera IDs to uint16 depth maps
```

Definition at line 127 of file depthmap.py.

```
00127 def generate_depthmaps_for_pose(cam_to_focus_path: Dict[str, str]) -> Dict[str, np.ndarray]:
00128     """
00129     @brief Generate depth maps for all cameras in a multi-camera pose.
00130
00131     @param cam_to_focus_path Dictionary mapping camera IDs to image file paths
00132     @return Dictionary mapping camera IDs to uint16 depth maps
00133     """
```

```
00134
00135     # 1) Load images
00136     imgs = {}
00137     for cam, path in cam_to_focus_path.items():
00138         im = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
00139         if im is None:
00140             continue
00141         imgs[cam] = im
00142
00143     if len(imgs) < 2:
00144         return {}
00145
00146     # helper
00147     def opposite_name(name: str) -> Optional[str]:
00148         if name.startswith("Left"):
00149             return name.replace("Left", "Right", 1)
00150         if name.startswith("Right"):
00151             return name.replace("Right", "Left", 1)
00152         if name.startswith("Up"):
00153             return name.replace("Up", "Down", 1)
00154         if name.startswith("Down"):
00155             return name.replace("Down", "Up", 1)
00156         return None
00157
00158     depthmaps = {}
00159
00160     for cam, img in imgs.items():
00161         disp_list = []
00162
00163         if cam == "Center":
00164             # Center uses all other views
00165             for other_cam, other_img in imgs.items():
00166                 if other_cam == "Center":
00167                     continue
00168                 d = disparity(img, other_img)
00169                 # Keep only if there is any valid disparity
00170                 if np.any(~np.isnan(d)):
00171                     disp_list.append(d)
00172
00173         else:
00174             # Opposite direction if available
00175             opp = opposite_name(cam)
00176             if opp is not None and opp in imgs:
00177                 if cam.startswith("Right") or cam.startswith("Down"):
00178                     d = disparity(imgs[opp], img)
00179                 else:
00180                     d = disparity(img, imgs[opp])
00181
00182                 if np.any(~np.isnan(d)):
00183                     disp_list.append(d)
00184
00185             # Add Center view
00186             if "Center" in imgs:
00187                 d = disparity(img, imgs["Center"])
00188                 if np.any(~np.isnan(d)):
00189                     disp_list.append(d)
00190
00191         if not disp_list:
00192             # No valid disparity found; skip
00193             continue
00194
00195         # Robust fusion
00196         disp_final = fuse_disparities_median(disp_list)
00197
00198         # Convert to uint16 depth map
00199         depth16 = to_u16_from_disp(disp_final)
00200         depthmaps[cam] = depth16
00201
00202     return depthmaps
```

4.3.1.5 imwrite_u16()

```
bool depthmap.imwrite_u16 (
            str path,
            np.ndarray img_u16)
```

```
@brief Write uint16 png, create dirs automatically.

@param path Output file path
@param img_u16 Image array in uint16 format
@return True if write succeeded, False otherwise
```

Definition at line 50 of file depthmap.py.

```
00050 def imwrite_u16(path: str, img_u16: np.ndarray) -> bool:
00051     """
00052     @brief Write uint16 png, create dirs automatically.
00053
00054     @param path Output file path
00055     @param img_u16 Image array in uint16 format
00056     @return True if write succeeded, False otherwise
00057     """
00058     os.makedirs(os.path.dirname(path), exist_ok=True)
00059     if img_u16.dtype != np.uint16:
00060         img_u16 = img_u16.astype(np.uint16)
00061     return bool(cv2.imwrite(path, img_u16))
00062
00063
```

4.3.1.6 to_u16_from_disp()

```
np.ndarray depthmap.to_u16_from_disp (
            np.ndarray disp)
```

```
@brief Convert disparity to uint16 depth map to be compatible with LiFCal.

@param disp Float32 disparity map
@return Uint16 depth map
```

Definition at line 85 of file depthmap.py.

```
00085 def to_u16_from_disp(disp: np.ndarray) -> np.ndarray:
00086     """
00087     @brief Convert disparity to uint16 depth map to be compatible with LiFCal.
00088
00089     @param disp Float32 disparity map
00090     @return Uint16 depth map
00091     """
00092     disp = np.nan_to_num(disp, nan=0.0, posinf=0.0, neginf=0.0)
00093
00094     mx = float(np.percentile(disp, 99))
00095     if not np.isfinite(mx) or mx <= 1e-6:
00096         return np.zeros(disp.shape, dtype=np.uint16)
00097
00098     disp = np.clip(disp, 0.0, mx)
00099     depth16 = (disp / mx * 65535.0).astype(np.uint16)
00100     return depth16
00101
00102
```

4.3.2 Variable Documentation

4.3.2.1 _LEFT_MATCHER

```
depthmap._LEFT_MATCHER  [protected]
```

Definition at line 47 of file depthmap.py.

4.3.2.2 _RIGHT_MATCHER

```
depthmap._RIGHT_MATCHER  [protected]
```

Definition at line 47 of file depthmap.py.

4.3.2.3 _WLS

depthmap._WLS [protected]

Definition at line 47 of file depthmap.py.

4.4 evaluate_calibration Namespace Reference

Functions

- rot_angle_deg (R)
- as_vec3 (v)
- compute_metrics_for_file (Path calib_path, Path gt_path)
- print_per_file_table (results_by_name)
- print_scenario_table (results_by_name)
- main ()

Variables

- THIS_DIR = Path(__file__).resolve().parent
- CALIB_LFC_DIR = THIS_DIR.parent
- PROJECT_ROOT = CALIB_LFC_DIR.parent
- str RESULTS_DIR = CALIB_LFC_DIR / "results"
- str GT_DIR = PROJECT_ROOT / "TestEnvironment" / "params"
- list CAM_IDS
- dict CALIB_FILES
- dict GT_FILES
- dict SCENARIOS

4.4.1 Function Documentation

4.4.1.1 as_vec3()

evaluate_calibration.as_vec3 (
v)

Convert any translation vector to robust 3-element vector.

Definition at line 62 of file evaluate_calibration.py.

```
00062 def as_vec3(v):
00063     """Convert any translation vector to robust 3-element vector."""
00064     arr = np.array(v, dtype=float).reshape(-1)
00065     if arr.size != 3:
00066         raise ValueError(f"Expected 3 entries for translation, got {arr.size}")
00067     return arr
00068
00069
```

4.4.1.2 compute_metrics_for_file()

```
evaluate_calibration.compute_metrics_for_file (
            Path calib_path,
            Path gt_path)
```

```
Loads a calibration JSON and groundtruth JSON and computes:
  - numPoses
  - mean T (m)
  - mean R (°)
  - mean reprojection error (px, from intrinsics)

Definitions as in your extrinsic plot:
  R_cam = angle(R_est * R_gt^T)
  T_cam = || T_est - T_gt ||_2
```

Definition at line 70 of file evaluate_calibration.py.

```
00070 def compute_metrics_for_file(calib_path: Path, gt_path: Path):
00071     """
00072     Loads a calibration JSON and groundtruth JSON and computes:
00073       - numPoses
00074       - mean T (m)
00075       - mean R (°)
00076       - mean reprojection error (px, from intrinsics)
00077
00078     Definitions as in your extrinsic plot:
00079       R_cam = angle(R_est * R_gt^T)
00080       T_cam = || T_est - T_gt ||_2
00081     """
00082     with open(gt_path, "r", encoding="utf-8") as f:
00083         gt = json.load(f)
00084
00085     with open(calib_path, "r", encoding="utf-8") as f:
00086         calib = json.load(f)
00087
00088     meta = calib.get("meta", {})
00089     extr_est = calib["state"]["extrinsics"]
00090     intr_est = calib["state"]["intrinsics"]
00091
00092     bundle_adjust = bool(meta.get("bundleAdjust", False))
00093     num_poses = int(meta.get("numPoses", -1))
00094
00095     rot_err = []
00096     trans_err = []
00097     reproj_err = []
00098
00099     for cam in CAM_IDS:
00100         if cam not in extr_est or cam not in gt:
00101             print(f"[WARN] Camera '{cam}' not in both files for {calib_path.name}, skipping this
      cam.")
00102             continue
00103
00104         # Extrinsics
00105         R_est = np.array(extr_est[cam]["rotationMatrix"], dtype=float)
00106         R_gt  = np.array(gt[cam]["rotationMatrix"], dtype=float)
00107
00108         T_est = as_vec3(extr_est[cam]["translationVector"])
00109         T_gt  = as_vec3(gt[cam]["translationVector"])
00110
00111         # Rotation error as in plot
00112         R_err = R_est @ R_gt.T
00113         ang = rot_angle_deg(R_err)
00114
00115         # Translation error (absolute, m)
00116         d_abs = float(np.linalg.norm(T_est - T_gt))
00117
00118         rot_err.append(ang)
00119         trans_err.append(d_abs)
00120
00121         # Intrinsic reprojection error
00122         if cam in intr_est:
00123             reproj_err.append(float(intr_est[cam]["reprojectionError"]))
00124
00125     # Mean values
00126     mean_dT = float(np.mean(trans_err))     if trans_err else float("nan")
00127     mean_dR = float(np.mean(rot_err))       if rot_err else float("nan")
00128     mean_reproj = float(np.mean(reproj_err)) if reproj_err else float("nan")
00129
00130     return {
```

```
00131         "bundleAdjust": bundle_adjust,
00132         "numPoses": num_poses,
00133         "mean_dT": mean_dT,
00134         "mean_dR": mean_dR,
00135         "mean_reproj": mean_reproj,
00136     }
00137
00138 # Table output (console)
```

4.4.1.3 main()

```
evaluate_calibration.main ()
```

Definition at line 217 of file evaluate_calibration.py.

```
00217 def main():
00218     results_by_name = {}
00219
00220     for name, calib_path in CALIB_FILES.items():
00221         if not calib_path.exists():
00222             print(f"[WARN] File not found: {calib_path}")
00223             continue
00224
00225         gt_path = GT_FILES.get(name)
00226         if gt_path is None:
00227             print(f"[WARN] No GT file mapped for {name}, skipping.")
00228             continue
00229         if not gt_path.exists():
00230             print(f"[WARN] GT file not found for {name}: {gt_path}")
00231             continue
00232
00233         metrics = compute_metrics_for_file(calib_path, gt_path)
00234         results_by_name[name] = metrics
00235
00236     if not results_by_name:
00237         print("[ERROR] No valid results computed. Check file paths.")
00238         return
00239
00240     print("Detailed results per calibration:\n")
00241     print_per_file_table(results_by_name)
00242
00243     print_scenario_table(results_by_name)
00244
00245
```

4.4.1.4 print_per_file_table()

```
evaluate_calibration.print_per_file_table (
              results_by_name)
```

Prints one line per calibration JSON.

Definition at line 139 of file evaluate_calibration.py.

```
00139 def print_per_file_table(results_by_name):
00140     """
00141     Prints one line per calibration JSON.
00142     """
00143     header = (
00144         f"{'Name':<12} "
00145         f"{'#Poses':>6} "
00146         f"{'BA':<4} "
00147         f"{'mean dT [m]':>12} "
00148         f"{'mean dR [deg]':>13} "
00149         f"{'mean reproj [px]':>16}"
00150     )
00151     print(header)
00152     print("-" * len(header))
00153
00154     for name, res in sorted(results_by_name.items()):
00155         ba_flag = "yes" if res["bundleAdjust"] else "no"
00156         print(
00157             f"{name:<12} "
00158             f"{res['numPoses']:6d} "
00159             f"{ba_flag:<4} "
00160             f"{res['mean_dT']:12.5f} "
00161             f"{res['mean_dR']:12.5f} "
00162             f"{res['mean_reproj']:16.5f}"
00163         )
00164
```

4.4.1.5 print_scenario_table()

```
evaluate_calibration.print_scenario_table (
              results_by_name)
```

Aggregates over scenarios (Robustness / Generalization)
and prints a compact table.

Definition at line 165 of file evaluate_calibration.py.

```
00165 def print_scenario_table(results_by_name):
00166     """
00167     Aggregates over scenarios (Robustness / Generalization)
00168     and prints a compact table.
00169     """
00170     print("\n\nScenario Summary:\n")
00171
00172     header = (
00173         f"{'Szenario':<22} "
00174         f"{'#Runs':>6} "
00175         f"{'Avg#Poses':>9} "
00176         f"{'mean dT [m]':>12} "
00177         f"{'mean dR [deg]':>13} "
00178         f"{'mean reproj [px]':>16}"
00179     )
00180     print(header)
00181     print("-" * len(header))
00182
00183     for scen_name, calib_names in SCENARIOS.items():
00184         dT_vals = []
00185         dR_vals = []
00186         reproj_vals = []
00187         pose_counts = []
00188
00189         for cname in calib_names:
00190             res = results_by_name.get(cname)
00191             if res is None:
00192                 continue
00193
00194             dT_vals.append(res["mean_dT"])
00195             dR_vals.append(res["mean_dR"])
00196             reproj_vals.append(res["mean_reproj"])
00197             pose_counts.append(res["numPoses"])
00198
00199         if not dT_vals:
00200             continue
00201
00202         mean_dT = float(np.mean(dT_vals))
00203         mean_dR = float(np.mean(dR_vals))
00204         mean_reproj = float(np.mean(reproj_vals))
00205         mean_poses = float(np.mean(pose_counts))
00206         num_runs = len(dT_vals)
00207
00208         print(
00209             f"{scen_name:<22} "
00210             f"{num_runs:6d} "
00211             f"{mean_poses:9.1f} "
00212             f"{mean_dT:12.5f} "
00213             f"{mean_dR:13.5f} "
00214             f"{mean_reproj:16.5f}"
00215         )
00216
```

4.4.1.6 rot_angle_deg()

```
evaluate_calibration.rot_angle_deg (
              R)
```

Extract rotation angle (degrees) from 3x3 rotation matrix.

Definition at line 55 of file evaluate_calibration.py.

```
00055 def rot_angle_deg(R):
00056     """Extract rotation angle (degrees) from 3x3 rotation matrix."""
00057     tr = np.trace(R)
00058     c = (tr - 1.0) / 2.0
00059     c = np.clip(c, -1.0, 1.0)
00060     return float(np.degrees(np.arccos(c)))
00061
```

4.4.2 Variable Documentation

4.4.2.1 CALIB_FILES

```
dict evaluate_calibration.CALIB_FILES
```

Initial value:

```
00001 =  {
00002     "imageset1": RESULTS_DIR / "calibration_initial_imageset1_20251215_173701.json",
00003     "imageset2": RESULTS_DIR / "calibration_initial_imageset2_20251215_181903.json",
00004     "imageset3": RESULTS_DIR / "calibration_initial_imageset3_20251215_185041.json",
00005     "imageset4": RESULTS_DIR / "calibration_initial_imageset4_20251215_165644.json",
00006     "imageset5": RESULTS_DIR / "calibration_initial_imageset5_20251217_011656.json",
00007     "imageset6": RESULTS_DIR / "calibration_initial_imageset6_20251215_201527.json",
00008     "imageset7": RESULTS_DIR / "calibration_initial_imageset7_20251215_203849.json",
00009 }
```

Definition at line 24 of file evaluate_calibration.py.

4.4.2.2 CALIB_LFC_DIR

```
evaluate_calibration.CALIB_LFC_DIR = THIS_DIR.parent
```

Definition at line 7 of file evaluate_calibration.py.

4.4.2.3 CAM_IDS

```
list evaluate_calibration.CAM_IDS
```

Initial value:

```
00001 =  [
00002     "Center",
00003     "Up1", "Up2", "Up3",
00004     "Down1", "Down2", "Down3",
00005     "Left1", "Left2", "Left3",
00006     "Right1", "Right2", "Right3",
00007 ]
```

Definition at line 15 of file evaluate_calibration.py.

4.4.2.4 GT_DIR

```
str evaluate_calibration.GT_DIR = PROJECT_ROOT / "TestEnvironment" / "params"
```

Definition at line 12 of file evaluate_calibration.py.

4.4.2.5 GT_FILES

```
dict evaluate_calibration.GT_FILES
```

Initial value:

```
00001 =  {
00002     # Rig0 for imageset1-4
00003     "imageset1": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00004     "imageset2": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00005     "imageset3": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00006     "imageset4": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00007
00008     # Different rigs/sets:
00009     "imageset5": GT_DIR / "groundtruth_extrinsics_Rig1_set5.json",
00010     "imageset6": GT_DIR / "groundtruth_extrinsics_Rig2_set6.json",
00011     "imageset7": GT_DIR / "groundtruth_extrinsics_Rig3_set7.json",
00012 }
```

Definition at line 35 of file evaluate_calibration.py.

4.4.2.6 PROJECT_ROOT

```
evaluate_calibration.PROJECT_ROOT = CALIB_LFC_DIR.parent
```

Definition at line 8 of file evaluate_calibration.py.

4.4.2.7 RESULTS_DIR

```
str evaluate_calibration.RESULTS_DIR = CALIB_LFC_DIR / "results"
```

Definition at line 9 of file evaluate_calibration.py.

4.4.2.8 SCENARIOS

```
dict evaluate_calibration.SCENARIOS
```

Initial value:

```
00001 =  {
00002     "A_Robustheit": ["imageset1", "imageset2", "imageset3", "imageset4"],
00003     "B_Generalisation":    ["imageset5", "imageset6", "imageset7"],
00004 }
```

Definition at line 49 of file evaluate_calibration.py.

4.4.2.9 THIS_DIR

```
evaluate_calibration.THIS_DIR = Path(__file__).resolve().parent
```

Definition at line 6 of file evaluate_calibration.py.

4.5 HealthMonitor Namespace Reference

Functions

- float clamp (float x, float lo=0.0, float hi=1.0)
- safe_float (v, default=None)
- float score_from_errors (float E_a, float E_b, float w_a=0.6, float w_b=0.4)
- Dict[str, Any] compute_initial_health_from_state (Dict[str, Any] state_dict, float r_best=R_BEST, float r_worst=R_WORST, float s_best=S_BEST, float s_worst=S_WORST)
- Dict[str, Optional[float]] parse_lifcal_protocol (str protocol_path)
- Dict[str, Any] compute_lifcal_health_from_xy (float x_std, float y_std, float x_mae, float y_mae, float r_best=R_BEST, float r_worst=R_WORST, float s_best=S_BEST, float s_worst=S_WORST)
- Dict[str, Any] compute_lifcal_health_from_combined_dict (Dict[str, Any] combined)
- Dict[str, Any] compute_lifcal_health_from_combined_path (str combined_json_path)

Variables

- float R_BEST = 0.03
- float R_WORST = 1.0
- float S_BEST = 0.0
- float S_WORST = 50.0

4.5.1 Detailed Description

```
HealthScore Formulas (0..100)

(A) INITIAL Calibration (OpenCV/Controller JSON)
    HealthScore_initial = 100 * [ 1 - (0.6 * E_reproj^2 + 0.4 * E_rms^2) ]

    E_reproj = clamp( (reproj_mean - R_BEST) / (R_WORST - R_BEST), 0, 1 )
    E_rms    = clamp( (rms_mean    - S_BEST) / (S_WORST - S_BEST), 0, 1 )

    Uses:
      - intrinsics[*].reprojectionError  (per camera)
      - extrinsics[*].stereoRms          (per camera)

(B) LiFCal Calibration (protocol-based)
    HealthScore_lifcal = 100 * [ 1 - (0.6 * E_std^2 + 0.4 * E_mae^2) ]

    E_std = clamp( ( sqrt(x_std^2 + y_std^2) - S_BEST ) / (S_WORST - S_BEST), 0, 1 )
    E_mae = clamp( ( sqrt(x_mae^2 + y_mae^2) - R_BEST ) / (R_WORST - R_BEST), 0, 1 )

    Uses:
      - std.Dev. x/y  and mae x/y from LiFCal calibrationProtocol.txt

Shared thresholds (for BOTH methods):
  R_BEST = 0.03, R_WORST = 1.0, S_BEST = 0.0, S_WORST = 50.0
```

4.5.2 Function Documentation

4.5.2.1 clamp()

```
float HealthMonitor.clamp (
            float x,
            float  lo = 0.0,
            float  hi = 1.0)
```

```
@brief Clamp value between lower and upper bounds.

@param x Value to clamp
@param lo Lower bound (default: 0.0)
@param hi Upper bound (default: 1.0)
@return Clamped value
```

Definition at line 44 of file HealthMonitor.py.

```
00044 def clamp(x: float, lo: float = 0.0, hi: float = 1.0) -> float:
00045     """
00046     @brief Clamp value between lower and upper bounds.
00047
00048     @param x Value to clamp
00049     @param lo Lower bound (default: 0.0)
00050     @param hi Upper bound (default: 1.0)
00051     @return Clamped value
00052     """
00053     if x < lo:
00054         return lo
00055     if x > hi:
00056         return hi
00057     return x
00058
00059
```

4.5.2.2 compute_initial_health_from_state()

```
Dict[str, Any] HealthMonitor.compute_initial_health_from_state (
            Dict[str, Any] state_dict,
            float  r_best = R_BEST,
            float  r_worst = R_WORST,
            float  s_best = S_BEST,
            float  s_worst = S_WORST)
```

```
@brief Compute health score from initial calibration state.

@param state_dict Dictionary containing calibration state with intrinsics and extrinsics
@param r_best Best reprojection error threshold (default: R_BEST)
@param r_worst Worst reprojection error threshold (default: R_WORST)
@param s_best Best stereo RMS threshold (default: S_BEST)
@param s_worst Worst stereo RMS threshold (default: S_WORST)
@return Dictionary with health score, method name, and detailed metrics
```

Definition at line 90 of file HealthMonitor.py.

```
00096 ) -> Dict[str, Any]:
00097     """
00098     @brief Compute health score from initial calibration state.
00099
00100     @param state_dict Dictionary containing calibration state with intrinsics and extrinsics
00101     @param r_best Best reprojection error threshold (default: R_BEST)
00102     @param r_worst Worst reprojection error threshold (default: R_WORST)
00103     @param s_best Best stereo RMS threshold (default: S_BEST)
00104     @param s_worst Worst stereo RMS threshold (default: S_WORST)
00105     @return Dictionary with health score, method name, and detailed metrics
00106     """
00107     st = state_dict.get("state", state_dict)
00108     intr = st.get("intrinsics", {}) or {}
00109     extr = st.get("extrinsics", {}) or {}
00110
00111     reproj_vals = []
00112     rms_vals = []
00113
00114     for cam, intr_data in intr.items():
00115         reproj = safe_float(intr_data.get("reprojectionError"), None)
00116         if reproj is not None:
00117             reproj_vals.append(reproj)
00118
00119         ex = extr.get(cam, {})
00120         rms = safe_float(ex.get("stereoRms"), None)
00121         if rms is not None:
00122             rms_vals.append(rms)
00123
00124     if not reproj_vals and not rms_vals:
00125         return {
00126             "health": 0.0,
00127             "method": "initial",
00128             "details": {"reason": "no reprojectionError and no stereoRms found"},
00129         }
00130
00131     reproj_mean = (sum(reproj_vals) / len(reproj_vals)) if reproj_vals else r_worst
00132     rms_mean = (sum(rms_vals) / len(rms_vals)) if rms_vals else s_worst
00133
00134     E_reproj = clamp((reproj_mean - r_best) / (r_worst - r_best), 0.0, 1.0)
00135     E_rms = clamp((rms_mean - s_best) / (s_worst - s_best), 0.0, 1.0)
00136
00137     score = score_from_errors(E_reproj, E_rms)
00138
00139     return {
00140         "health": float(score),
00141         "method": "initial",
00142         "details": {
00143             "reproj_mean": float(reproj_mean),
00144             "rms_mean": float(rms_mean),
00145             "E_reproj": float(E_reproj),
00146             "E_rms": float(E_rms),
00147             "thresholds": {
00148                 "R_BEST": float(r_best),
00149                 "R_WORST": float(r_worst),
00150                 "S_BEST": float(s_best),
00151                 "S_WORST": float(s_worst),
00152             },
00153         },
00154     }
00155
00156
```

4.5.2.3 compute_lifcal_health_from_combined_dict()

```
Dict[str, Any] HealthMonitor.compute_lifcal_health_from_combined_dict (
            Dict[str, Any] combined)
```

@brief Compute LiFCal health from combined.json structure.

@param combined Dictionary from combined.json with parameters for all cameras
@return Dictionary with global health score, method name, and per-camera metrics

Definition at line 228 of file HealthMonitor.py.

```
00228 def compute_lifcal_health_from_combined_dict(combined: Dict[str, Any]) -> Dict[str, Any]:
00229     """
00230     @brief Compute LiFCal health from combined.json structure.
00231
00232     @param combined Dictionary from combined.json with parameters for all cameras
00233     @return Dictionary with global health score, method name, and per-camera metrics
00234     """
00235     params_all = combined.get("state", {}).get("parameters", {})
00236     if not isinstance(params_all, dict) or len(params_all) == 0:
00237         return {"health": 0.0, "method": "lifcal", "perCam": {}}
00238
00239     per_cam = {}
00240     scores = []
00241
00242     for cam, p in params_all.items():
00243         rep = (p or {}).get("reprojection", {}) or {}
00244
00245         x_std = rep.get("x_std", None)
00246         y_std = rep.get("y_std", None)
00247         x_mae = rep.get("x_mae", None)
00248         y_mae = rep.get("y_mae", None)
00249
00250         # Skip camera if any field is missing
00251         if any(v is None for v in [x_std, y_std, x_mae, y_mae]):
00252             per_cam[cam] = {"health": 0.0, "reason": "missing reprojection fields"}
00253             continue
00254
00255         std_mag = math.sqrt(float(x_std) ** 2 + float(y_std) ** 2)
00256         mae_mag = math.sqrt(float(x_mae) ** 2 + float(y_mae) ** 2)
00257
00258         E_std = clamp((std_mag - S_BEST) / (S_WORST - S_BEST), 0.0, 1.0)
00259         E_mae = clamp((mae_mag - R_BEST) / (R_WORST - R_BEST), 0.0, 1.0)
00260
00261         health = 100.0 * (1.0 - (0.6 * (E_std ** 2) + 0.4 * (E_mae ** 2)))
00262         health = clamp(health, 0.0, 100.0)
00263
00264         per_cam[cam] = {
00265             "health": float(health),
00266             "std_mag": float(std_mag),
00267             "mae_mag": float(mae_mag),
00268             "E_std": float(E_std),
00269             "E_mae": float(E_mae),
00270         }
00271         scores.append(float(health))
00272
00273     if len(scores) == 0:
00274         return {"health": 0.0, "method": "lifcal", "perCam": per_cam}
00275
00276     # Global health: average across all cameras
00277     global_health = sum(scores) / len(scores)
00278
00279     return {"health": float(global_health), "method": "lifcal", "perCam": per_cam}
00280
00281
```

4.5.2.4 compute_lifcal_health_from_combined_path()

```
Dict[str, Any] HealthMonitor.compute_lifcal_health_from_combined_path (
            str combined_json_path)
```

@brief Load combined.json from file and compute LiFCal health.

@param combined_json_path Path to the combined.json file
@return Dictionary with global health score, method name, and per-camera metrics

Definition at line 282 of file HealthMonitor.py.

```
00282 def compute_lifcal_health_from_combined_path(combined_json_path: str) -> Dict[str, Any]:
00283     """
00284     @brief Load combined.json from file and compute LiFCal health.
00285
00286     @param combined_json_path Path to the combined.json file
00287     @return Dictionary with global health score, method name, and per-camera metrics
00288     """
00289     with open(combined_json_path, "r", encoding="utf-8") as f:
00290         combined = json.load(f)
00291     return compute_lifcal_health_from_combined_dict(combined)
```

4.5.2.5 compute_lifcal_health_from_xy()

```
Dict[str, Any] HealthMonitor.compute_lifcal_health_from_xy (
            float x_std,
            float y_std,
            float x_mae,
            float y_mae,
            float  r_best = R_BEST,
            float  r_worst = R_WORST,
            float  s_best = S_BEST,
            float  s_worst = S_WORST)
```

```
@brief Compute LiFCal health score from x/y standard deviation and MAE values.

@param x_std Standard deviation in x direction
@param y_std Standard deviation in y direction
@param x_mae Mean absolute error in x direction
@param y_mae Mean absolute error in y direction
@param r_best Best reprojection error threshold (default: R_BEST)
@param r_worst Worst reprojection error threshold (default: R_WORST)
@param s_best Best stereo RMS threshold (default: S_BEST)
@param s_worst Worst stereo RMS threshold (default: S_WORST)
@return Dictionary with health score, method name, and detailed metrics
```

Definition at line 180 of file HealthMonitor.py.

```
00189 ) -> Dict[str, Any]:
00190     """
00191     @brief Compute LiFCal health score from x/y standard deviation and MAE values.
00192
00193     @param x_std Standard deviation in x direction
00194     @param y_std Standard deviation in y direction
00195     @param x_mae Mean absolute error in x direction
00196     @param y_mae Mean absolute error in y direction
00197     @param r_best Best reprojection error threshold (default: R_BEST)
00198     @param r_worst Worst reprojection error threshold (default: R_WORST)
00199     @param s_best Best stereo RMS threshold (default: S_BEST)
00200     @param s_worst Worst stereo RMS threshold (default: S_WORST)
00201     @return Dictionary with health score, method name, and detailed metrics
00202     """
00203     std_norm = (x_std * x_std + y_std * y_std) ** 0.5
00204     mae_norm = (x_mae * x_mae + y_mae * y_mae) ** 0.5
00205
00206     E_std = clamp((std_norm - s_best) / (s_worst - s_best), 0.0, 1.0)
00207     E_mae = clamp((mae_norm - r_best) / (r_worst - r_best), 0.0, 1.0)
00208
00209     score = score_from_errors(E_std, E_mae)
00210
00211     return {
00212         "health": float(score),
00213         "method": "lifcal",
00214         "details": {
00215             "std_norm": float(std_norm),
00216             "mae_norm": float(mae_norm),
00217             "E_std": float(E_std),
00218             "E_mae": float(E_mae),
00219             "thresholds": {
00220                 "R_BEST": float(r_best),
00221                 "R_WORST": float(r_worst),
00222                 "S_BEST": float(s_best),
00223                 "S_WORST": float(s_worst),
00224             },
00225         },
00226     }
00227
```

4.5.2.6 parse_lifcal_protocol()

```
Dict[str, Optional[float]] HealthMonitor.parse_lifcal_protocol (
            str protocol_path)
```

```
@brief Parse LiFCal calibration protocol text file for error metrics.

@param protocol_path Path to the calibration protocol text file
@return Dictionary with x_std, y_std, x_mae, y_mae values or None if not found
```

Definition at line 157 of file HealthMonitor.py.

```
00157 def parse_lifcal_protocol(protocol_path: str) -> Dict[str, Optional[float]]:
00158     """
00159     @brief Parse LiFCal calibration protocol text file for error metrics.
00160
00161     @param protocol_path Path to the calibration protocol text file
00162     @return Dictionary with x_std, y_std, x_mae, y_mae values or None if not found
00163     """
00164     txt = open(protocol_path, "r", encoding="utf-8", errors="ignore").read()
00165
00166     def find_float(pattern: str) -> Optional[float]:
00167         m = re.search(pattern, txt, re.MULTILINE)
00168         if not m:
00169             return None
00170         return safe_float(m.group(1), None)
00171
00172     x_std = find_float(r"std\.\s*Dev\.\s*x:\s*([0-9.+-eE]+)")
00173     y_std = find_float(r"std\.\s*Dev\.\s*y:\s*([0-9.+-eE]+)")
00174     x_mae = find_float(r"mae\s*x:\s*([0-9.+-eE]+)")
00175     y_mae = find_float(r"mae\s*y:\s*([0-9.+-eE]+)")
00176
00177     return {"x_std": x_std, "y_std": y_std, "x_mae": x_mae, "y_mae": y_mae}
00178
00179
```

4.5.2.7 safe_float()

```
HealthMonitor.safe_float (
            v,
            default = None)
```

```
@brief Convert value to float safely, return default on error.

@param v Value to convert
@param default Default value to return on conversion error
@return Converted float value or default
```

Definition at line 60 of file HealthMonitor.py.

```
00060 def safe_float(v, default=None):
00061     """
00062     @brief Convert value to float safely, return default on error.
00063
00064     @param v Value to convert
00065     @param default Default value to return on conversion error
00066     @return Converted float value or default
00067     """
00068     try:
00069         if v is None:
00070             return default
00071         return float(v)
00072     except Exception:
00073         return default
00074
00075
```

4.5.2.8 score_from_errors()

```
float HealthMonitor.score_from_errors (
            float E_a,
            float E_b,
            float  w_a = 0.6,
            float  w_b = 0.4)
```

```
@brief Compute health score from two normalized error metrics.

@param E_a First normalized error metric (0..1)
@param E_b Second normalized error metric (0..1)
@param w_a Weight for first error metric (default: 0.6)
@param w_b Weight for second error metric (default: 0.4)
@return Health score between 0 and 100
```

Definition at line 76 of file HealthMonitor.py.

```
00076 def score_from_errors(E_a: float, E_b: float, w_a: float = 0.6, w_b: float = 0.4) -> float:
00077     """
00078     @brief Compute health score from two normalized error metrics.
00079
00080     @param E_a First normalized error metric (0..1)
00081     @param E_b Second normalized error metric (0..1)
00082     @param w_a Weight for first error metric (default: 0.6)
00083     @param w_b Weight for second error metric (default: 0.4)
00084     @return Health score between 0 and 100
00085     """
00086     val = 100.0 * (1.0 - (w_a * (E_a ** 2) + w_b * (E_b ** 2)))
00087     return clamp(val / 100.0, 0.0, 1.0) * 100.0
00088
00089
```

4.5.3 Variable Documentation

4.5.3.1 R_BEST

```
float HealthMonitor.R_BEST = 0.03
```

Definition at line 38 of file HealthMonitor.py.

4.5.3.2 R_WORST

```
float HealthMonitor.R_WORST = 1.0
```

Definition at line 39 of file HealthMonitor.py.

4.5.3.3 S_BEST

```
float HealthMonitor.S_BEST = 0.0
```

Definition at line 40 of file HealthMonitor.py.

4.5.3.4 S_WORST

```
float HealthMonitor.S_WORST = 50.0
```

Definition at line 41 of file HealthMonitor.py.

4.6 ImageSet Namespace Reference

Classes

- class ImageSet

4.7 InitialCalibration Namespace Reference

Classes

- class InitialCalibration

4.8 lifcal_to_json Namespace Reference

Functions

- dict parse_protocol (str protocol_path)
- Dict[str, Any] parse_extrinsics_xml (str xml_path)
- str export_lifcal_parameters_json_in_place (str result_dir, str cam_id, str image_dir, str run_type="recalib", float pixel_size_mm_fallback=0.0055, str protocol_name="calibrationProtocol.txt", str extr_name="extrinsicOrientations.xml", str out_name="parameters.json", Optional[float] pixel_size_mm=None, Optional[float] fallback_cx=None, Optional[float] fallback_cy=None)
- str build_combined_parameters_json (str run_root, list camera_ids, str out_name="combined.json")

4.8.1 Function Documentation

4.8.1.1 build_combined_parameters_json()

```
str lifcal_to_json.build_combined_parameters_json (
             str run_root,
             list camera_ids,
             str  out_name = "combined.json")
```

```
@brief Combine per-camera parameters.json files into a single combined.json.

@param run_root Root directory containing per-camera subdirectories
@param camera_ids List of camera identifiers
@param out_name Output filename (default: "combined.json")
@return Path to written combined.json file
```

Definition at line 206 of file lifcal_to_json.py.

```
00206 def build_combined_parameters_json(run_root: str, camera_ids: list, out_name: str = "combined.json")
      -> str:
00207     """
00208     @brief Combine per-camera parameters.json files into a single combined.json.
00209
00210     @param run_root Root directory containing per-camera subdirectories
00211     @param camera_ids List of camera identifiers
00212     @param out_name Output filename (default: "combined.json")
00213     @return Path to written combined.json file
00214     """
00215     combined = {
00216         "meta": {
00217             "runType": "combined",
```

```
00218             "cameraIds": camera_ids,
00219         },
00220         "state": {
00221             "parameters": {},
00222             "extrinsicsByPose": {},
00223             "timeStamp": None
00224         }
00225     }
00226 
00227     found_any = False
00228 
00229     for cam in camera_ids:
00230         cam_dir = os.path.join(run_root, cam)
00231         jpath = os.path.join(cam_dir, "parameters.json")
00232 
00233         if not os.path.isfile(jpath):
00234             continue
00235 
00236         with open(jpath, "r", encoding="utf-8") as f:
00237             data = json.load(f)
00238 
00239         st = data.get("state", {})
00240         params = st.get("parameters", {})
00241         extr_pose = st.get("extrinsicsByPose", {})
00242 
00243         if cam in params:
00244             combined["state"]["parameters"][cam] = params[cam]
00245             found_any = True
00246 
00247         if cam in extr_pose:
00248             combined["state"]["extrinsicsByPose"][cam] = extr_pose[cam]
00249             found_any = True
00250 
00251     if not found_any:
00252         raise RuntimeError("No parameters.json found to combine.")
00253 
00254     out_path = os.path.join(run_root, out_name)
00255     with open(out_path, "w", encoding="utf-8") as f:
00256         json.dump(combined, f, indent=2)
00257 
00258     return out_path
```

4.8.1.2 export_lifcal_parameters_json_in_place()

```
str lifcal_to_json.export_lifcal_parameters_json_in_place (
              str result_dir,
              str cam_id,
              str image_dir,
              str  run_type = "recalib",
              float  pixel_size_mm_fallback = 0.0055,
              str  protocol_name = "calibrationProtocol.txt",
              str  extr_name = "extrinsicOrientations.xml",
              str  out_name = "parameters.json",
              Optional[float]  pixel_size_mm = None,
              Optional[float]  fallback_cx = None,
              Optional[float]  fallback_cy = None)


@brief Write LiFCal parameters.json for a single camera.

@param result_dir Directory containing LiFCal output files
@param cam_id Camera identifier
@param image_dir Directory containing input images
@param run_type Type of calibration run (default: "recalib")
@param pixel_size_mm_fallback Fallback pixel size in mm (default: 0.0055)
@param protocol_name Protocol filename (default: "calibrationProtocol.txt")
@param extr_name Extrinsics filename (default: "extrinsicOrientations.xml")
@param out_name Output JSON filename (default: "parameters.json")
@param pixel_size_mm Optional pixel size override
@param fallback_cx Optional fallback cx (compatibility only, not used)
@param fallback_cy Optional fallback cy (compatibility only, not used)
@return Path to written parameters.json file
```

Definition at line 112 of file lifcal_to_json.py.

```
00125 ) -> str:
00126     """
00127     @brief Write LiFCal parameters.json for a single camera.
00128
00129     @param result_dir Directory containing LiFCal output files
00130     @param cam_id Camera identifier
00131     @param image_dir Directory containing input images
00132     @param run_type Type of calibration run (default: "recalib")
00133     @param pixel_size_mm_fallback Fallback pixel size in mm (default: 0.0055)
00134     @param protocol_name Protocol filename (default: "calibrationProtocol.txt")
00135     @param extr_name Extrinsics filename (default: "extrinsicOrientations.xml")
00136     @param out_name Output JSON filename (default: "parameters.json")
00137     @param pixel_size_mm Optional pixel size override
00138     @param fallback_cx Optional fallback cx (compatibility only, not used)
00139     @param fallback_cy Optional fallback cy (compatibility only, not used)
00140     @return Path to written parameters.json file
00141     """
00142     if pixel_size_mm is not None:
00143         pixel_size_mm_fallback = float(pixel_size_mm)
00144
00145     protocol_path = os.path.join(result_dir, protocol_name)
00146     extr_path = os.path.join(result_dir, extr_name)
00147
00148     if not os.path.isfile(protocol_path):
00149         raise FileNotFoundError(protocol_path)
00150     if not os.path.isfile(extr_path):
00151         raise FileNotFoundError(extr_path)
00152
00153     protocol = parse_protocol(protocol_path)
00154     extr_by_pose = parse_extrinsics_xml(extr_path)
00155
00156     # Ensure pixel size is available
00157     px = protocol.get("pixelSizeMm", None)
00158     if px is None or (not np.isfinite(px)) or px <= 0:
00159         protocol["pixelSizeMm"] = float(pixel_size_mm_fallback)
00160
00161     # Compact reprojection error (average MAE)
00162     rep = protocol.get("reprojection", {})
00163     x_mae = rep.get("x_mae", None)
00164     y_mae = rep.get("y_mae", None)
00165     avg_mae = None
00166     if x_mae is not None and y_mae is not None and np.isfinite(x_mae) and np.isfinite(y_mae):
00167         avg_mae = float((x_mae + y_mae) / 2.0)
00168
00169     data = {
00170         "meta": {
00171             "runType": run_type,
00172             "numPoses": int(len(extr_by_pose)),
00173             "imageDir": image_dir,
00174             "cameraIds": [cam_id],
00175             "bundleAdjust": True,
00176         },
00177         "state": {
00178             "parameters": {
00179                 cam_id: {
00180                     "pixelSizeMm": protocol.get("pixelSizeMm"),
00181                     "fL_mm": protocol.get("fL_mm"),
00182                     "bL0_mm": protocol.get("bL0_mm"),
00183                     "B_mm": protocol.get("B_mm"),
00184                     "cx_px": protocol.get("cx_px"),
00185                     "cy_px": protocol.get("cy_px"),
00186                     "a0": protocol.get("a0"),
00187                     "a1": protocol.get("a1"),
00188                     "b0": protocol.get("b0"),
00189                     "b1": protocol.get("b1"),
00190                     "reprojection": rep,
00191                     "reprojectionAvgMae": avg_mae,
00192                 }
00193             },
00194             "extrinsicsByPose": {cam_id: extr_by_pose},
00195             "timeStamp": None
00196         }
00197     }
00198
00199     out_path = os.path.join(result_dir, out_name)
00200     with open(out_path, "w", encoding="utf-8") as f:
00201         json.dump(data, f, indent=2)
00202
00203     return out_path
00204
00205
```

4.8.1.3 parse_extrinsics_xml()

```
Dict[str, Any] lifcal_to_json.parse_extrinsics_xml (
            str xml_path)
```

```
@brief Parse LiFCal extrinsic orientations XML file.

@param xml_path Path to extrinsicOrientations.xml file
@return Dictionary mapping pose keys to rotation and translation data
```

Definition at line 70 of file lifcal_to_json.py.

```
00070 def parse_extrinsics_xml(xml_path: str) -> Dict[str, Any]:
00071     """
00072     @brief Parse LiFCal extrinsic orientations XML file.
00073
00074     @param xml_path Path to extrinsicOrientations.xml file
00075     @return Dictionary mapping pose keys to rotation and translation data
00076     """
00077     root = ET.parse(xml_path).getroot()
00078     out: Dict[str, Any] = {}
00079
00080     for frame in root.findall("Frame"):
00081         fid = int(frame.attrib.get("id"))
00082         pose_key = "pose%03d" % fid
00083
00084         rot = frame.find("Rotation")
00085         tra = frame.find("Translation")
00086
00087         rvec = [0.0, 0.0, 0.0]
00088         tvec = [0.0, 0.0, 0.0]
00089
00090         if rot is not None:
00091             for c in rot.findall("Coeff"):
00092                 i = int(c.attrib["i"])
00093                 rvec[i] = float(c.text.strip())
00094
00095         if tra is not None:
00096             for c in tra.findall("Coeff"):
00097                 i = int(c.attrib["i"])
00098                 tvec[i] = float(c.text.strip())
00099
00100         rvec_np = np.array(rvec, dtype=np.float64).reshape(3, 1)
00101         R, _ = cv2.Rodrigues(rvec_np)
00102
00103         out[pose_key] = {
00104             "rotationMatrix": R.astype(float).tolist(),
00105             "rotationVector": [[float(rvec[0])], [float(rvec[1])], [float(rvec[2])]],
00106             "translationVector": [[float(tvec[0])], [float(tvec[1])], [float(tvec[2])]],
00107         }
00108
00109     return out
00110
00111
```

4.8.1.4 parse_protocol()

```
dict lifcal_to_json.parse_protocol (
            str protocol_path)
```

```
@brief Parse LiFCal calibration protocol text file.

@return Dictionary containing parsed calibration parameters and reprojection errors
```

Definition at line 15 of file lifcal_to_json.py.

```
00015 def parse_protocol(protocol_path: str) -> dict:
00016     """
00017     @brief Parse LiFCal calibration protocol text file.
00018
00019     @return Dictionary containing parsed calibration parameters and reprojection errors
00020     """
00021     txt = open(protocol_path, "r", encoding="utf-8", errors="ignore").read()
```

```
00022
00023     def find_float(pattern: str, default=None) -> Optional[float]:
00024         m = re.search(pattern, txt, re.MULTILINE)
00025         if not m:
00026             return default
00027         try:
00028             return float(m.group(1))
00029         except Exception:
00030             return default
00031
00032     pixel_size = find_float(r"Pixel Size:\s*([0-9.+-eE]+)\s*mm", None)
00033
00034     fL  = find_float(r"^\s*fL\s*:\s*([0-9.+-eE]+)\s*$", None)
00035     bL0 = find_float(r"^\s*bL0\s*:\s*([0-9.+-eE]+)\s*$", None)
00036     B   = find_float(r"^\s*B\s*:\s*([0-9.+-eE]+)\s*$", None)
00037     cx  = find_float(r"^\s*cx\s*:\s*([0-9.+-eE]+)\s*$", None)
00038     cy  = find_float(r"^\s*cy\s*:\s*([0-9.+-eE]+)\s*$", None)
00039
00040     a0 = find_float(r"^\s*a0\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00041     a1 = find_float(r"^\s*a1\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00042     b0 = find_float(r"^\s*b0\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00043     b1 = find_float(r"^\s*b1\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00044
00045     x_std = find_float(r"std\.\s*Dev\.\s*x:\s*([0-9.+-eE]+)", None)
00046     y_std = find_float(r"std\.\s*Dev\.\s*y:\s*([0-9.+-eE]+)", None)
00047     x_mae = find_float(r"mae\s*x:\s*([0-9.+-eE]+)", None)
00048     y_mae = find_float(r"mae\s*y:\s*([0-9.+-eE]+)", None)
00049
00050     return {
00051         "pixelSizeMm": pixel_size,
00052         "fL_mm": fL,
00053         "bL0_mm": bL0,
00054         "B_mm": B,
00055         "cx_px": cx,
00056         "cy_px": cy,
00057         "a0": a0,
00058         "a1": a1,
00059         "b0": b0,
00060         "b1": b1,
00061         "reprojection": {
00062             "x_std": x_std,
00063             "y_std": y_std,
00064             "x_mae": x_mae,
00065             "y_mae": y_mae
00066         }
00067     }
00068
00069
```

4.9 ResultLogger Namespace Reference

Classes

- class ResultLogger

4.10 run_imageset_creation Namespace Reference

Functions

- ensure_camera_subfolders (str base_dir)
- group_focus_images_by_pose ()
- copy_focus_into_camera_folders (str pose, dict cam_to_path)
- cleanup_focus_root ()
- main ()

Variables

- str ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
- FOCUS_DIR = os.path.join(ROOT, "focus")
- DEPTH_DIR = os.path.join(ROOT, "depth")
- bool CLEAN_FOCUS_ROOT_AFTER = True
- list CAMERA_IDS
- PATTERN

4.10.1 Function Documentation

4.10.1.1 cleanup_focus_root()

```
run_imageset_creation.cleanup_focus_root ()
```

```
@brief Delete only loose files directly in focus/ (not in subdirectories).
```

Definition at line 70 of file run_imageset_creation.py.

```
00070 def cleanup_focus_root():
00071     """
00072     @brief Delete only loose files directly in focus/ (not in subdirectories).
00073     """
00074     for fn in os.listdir(FOCUS_DIR):
00075         p = os.path.join(FOCUS_DIR, fn)
00076         if os.path.isfile(p) and PATTERN.match(fn):
00077             os.remove(p)
00078
```

4.10.1.2 copy_focus_into_camera_folders()

```
run_imageset_creation.copy_focus_into_camera_folders (
             str pose,
             dict cam_to_path)
```

```
@brief Copy focus images into camera-specific subdirectories.

@param pose Pose identifier string (e.g., "pose000")
@param cam_to_path Dictionary mapping camera IDs to source image paths
```

Definition at line 57 of file run_imageset_creation.py.

```
00057 def copy_focus_into_camera_folders(pose: str, cam_to_path: dict):
00058     """
00059     @brief Copy focus images into camera-specific subdirectories.
00060
00061     @param pose Pose identifier string (e.g., "pose000")
00062     @param cam_to_path Dictionary mapping camera IDs to source image paths
00063     """
00064     for cam, src in cam_to_path.items():
00065         ext = os.path.splitext(src)[1].lower()
00066         dst = os.path.join(FOCUS_DIR, cam, f"{pose}_{cam}{ext}")
00067         if os.path.abspath(src) != os.path.abspath(dst):
00068             shutil.copy2(src, dst)
00069
```

4.10.1.3 ensure_camera_subfolders()

```
run_imageset_creation.ensure_camera_subfolders (
            str base_dir)
```

```
@brief Create subdirectory for each camera ID under base_dir.

@param base_dir Base directory path
```

Definition at line 32 of file run_imageset_creation.py.

```
00032 def ensure_camera_subfolders(base_dir: str):
00033     """
00034     @brief Create subdirectory for each camera ID under base_dir.
00035
00036     @param base_dir Base directory path
00037     """
00038     os.makedirs(base_dir, exist_ok=True)
00039     for cid in CAMERA_IDS:
00040         os.makedirs(os.path.join(base_dir, cid), exist_ok=True)
00041
```

4.10.1.4 group_focus_images_by_pose()

```
run_imageset_creation.group_focus_images_by_pose ()
```

```
@brief Group focus images by pose ID from flat directory structure.

@return Dictionary mapping pose IDs to camera-path dictionaries
```

Definition at line 42 of file run_imageset_creation.py.

```
00042 def group_focus_images_by_pose():
00043     """
00044     @brief Group focus images by pose ID from flat directory structure.
00045
00046     @return Dictionary mapping pose IDs to camera-path dictionaries
00047     """
00048     pose_to_cam = defaultdict(dict)
00049     for fn in sorted(os.listdir(FOCUS_DIR)):
00050         m = PATTERN.match(fn)
00051         if not m:
00052             continue
00053         pose, cam, ext = m.group(1), m.group(2), m.group(3)
00054         pose_to_cam[pose][cam] = os.path.join(FOCUS_DIR, fn)
00055     return pose_to_cam
00056
```

4.10.1.5 main()

```
run_imageset_creation.main ()
```

```
@brief Main pipeline: organize focus images and generate depth maps.
```

Definition at line 79 of file run_imageset_creation.py.

```
00079 def main():
00080     """
00081     @brief Main pipeline: organize focus images and generate depth maps.
00082     """
00083     if not os.path.isdir(FOCUS_DIR):
00084         raise SystemExit(f"focus folder not found: {FOCUS_DIR}")
00085
00086     ensure_camera_subfolders(FOCUS_DIR)
00087     ensure_camera_subfolders(DEPTH_DIR)
00088
```

```
00089     pose_to_cam = group_focus_images_by_pose()
00090     if not pose_to_cam:
00091         raise SystemExit("No focus images found. Expected: pose000_Center.png etc.")
00092
00093     poses = sorted(pose_to_cam.keys())
00094     print("Found poses:", len(poses))
00095
00096     for i, pose in enumerate(poses, start=1):
00097         cam_to_focus_path = pose_to_cam[pose]
00098
00099         # Sort focus images into camera folders
00100         copy_focus_into_camera_folders(pose, cam_to_focus_path)
00101
00102         # Generate depth maps for this pose
00103         depthmaps = depthmap.generate_depthmaps_for_pose(cam_to_focus_path)
00104
00105         # Save depth maps: depth/<CamId>/<pose>_<CamId>_depth.png
00106         for cam, depth_u16 in depthmaps.items():
00107             out_path = os.path.join(DEPTH_DIR, cam, f"{pose}_{cam}_depth.png")
00108             ok = depthmap.imwrite_u16(out_path, depth_u16)
00109             if not ok:
00110                 print("FAILED write:", out_path)
00111
00112         print(f"[{i}/{len(poses)}] {pose}: views={len(cam_to_focus_path)} depthmaps={len(depthmaps)}")
00113
00114     if CLEAN_FOCUS_ROOT_AFTER:
00115         cleanup_focus_root()
00116
00117     print("DONE.")
00118     print("Focus by camera:", FOCUS_DIR)
00119     print("Depth by camera:", DEPTH_DIR)
00120
```

4.10.2 Variable Documentation

4.10.2.1 CAMERA_IDS

```
list run_imageset_creation.CAMERA_IDS
```

Initial value:

```
00001 =  [
00002     "Center", "Up1", "Up2", "Up3",
00003     "Down1", "Down2", "Down3",
00004     "Left1", "Left2", "Left3",
00005     "Right1", "Right2", "Right3"
00006 ]
```

Definition at line 19 of file run_imageset_creation.py.

4.10.2.2 CLEAN_FOCUS_ROOT_AFTER

```
bool run_imageset_creation.CLEAN_FOCUS_ROOT_AFTER = True
```

Definition at line 17 of file run_imageset_creation.py.

4.10.2.3 DEPTH_DIR

```
run_imageset_creation.DEPTH_DIR = os.path.join(ROOT, "depth")
```

Definition at line 14 of file run_imageset_creation.py.

4.10.2.4 FOCUS_DIR

```
run_imageset_creation.FOCUS_DIR = os.path.join(ROOT, "focus")
```

Definition at line 13 of file run_imageset_creation.py.

4.10.2.5 PATTERN

```
run_imageset_creation.PATTERN
```

Initial value:

```
00001 =  re.compile(
00002     r"^(pose\d+)_(Center|Up[123]|Down[123]|Left[123]|Right[123])\.(png|jpg|jpeg)$",
00003     re.IGNORECASE
00004 )
```

Definition at line 27 of file run_imageset_creation.py.

4.10.2.6 ROOT

```
str run_imageset_creation.ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_↩
Imageset"
```

Definition at line 12 of file run_imageset_creation.py.

4.11 run_initial_calibration Namespace Reference

Functions

- parse_args ()
- main (args)

Variables

- str DEFAULT_BASE_DIR = "/data/calibrationLFC/sets/imageset5"
- int DEFAULT_NUM_POSES = 28
- list CAM_IDS
- repoRoot = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

4.11.1 Detailed Description

```
Test runner for initial calibration.

Usage:
  cd calibrationLFC
  python3 run_initial_calibration.py --base_dir /path/to/imageset --num_poses 28 --bundle-adjust
  python3 run_initial_calibration.py --base_dir /path/to/imageset --num_poses 28 --no-bundle-adjust

Performs validation checks (file existence, corner detection on pose 0)
then runs the complete initial calibration pipeline. Outputs summary statistics
for intrinsics, extrinsics, and health score.
```

4.11.2 Function Documentation

4.11.2.1 main()

```
run_initial_calibration.main (
              args)
```

@brief Run complete initial calibration pipeline with validation and summary output.

@param args

Definition at line 59 of file run_initial_calibration.py.

```
00059 def main(args):
00060     """
00061     @brief Run complete initial calibration pipeline with validation and summary output.
00062 
00063     @param args
00064     """
00065     print("Creating ImageSet...")
00066     imageSet = ImageSet(args.base_dir, args.num_poses, CAM_IDS)
00067 
00068     print("Instantiating Controller and InitialCalibration...")
00069     controller = Controller(bundleAdjust=args.bundle_adjust)
00070     logger = controller.resultLogger
00071 
00072     print("\nQuick checks (pose 0): file exists / corners detected")
00073     for cam in CAM_IDS:
00074         path = imageSet.getImagePath(0, cam)
00075         exists = os.path.exists(path)
00076         cornersFound = None
00077 
00078         try:
00079             cornersFound, _, _ = controller.initialCalibration.detectCorners(path)
00080         except Exception as e:
00081             cornersFound = f"error: {e.__class__.__name__}"
00082         print(f" {cam}: {path} - exists={exists} - corners={cornersFound}")
00083 
00084     print("\nRunning initial calibration (this may take time depending on data)...")
00085     try:
00086         calibrationState = controller.runInitialCalibration(imageSet)
00087     except Exception as e:
00088         logger.logger.error(f"Calibration failed with exception: {e}", exc_info=True)
00089         print("Calibration failed, see calibration.log for details.")
00090         raise
00091 
00092     print("\nCalibration finished. Attempting to print summary...")
00093     try:
00094         stateDict = calibrationState.__getState__()
00095     except Exception:
00096         stateDict = None
00097 
00098     if stateDict is None:
00099         print("Could not extract a state dictionary.")
00100         return
00101 
00102     intr = stateDict.get("intrinsics", {})
00103     extr = stateDict.get("extrinsics", {})
00104 
00105     # Print calibration results per camera
00106     print("\nCalibration Results by Camera")
00107     print("=" * 50)
00108 
00109     for cam in CAM_IDS:
00110         print(f"\n--- {cam} ---")
00111 
00112         # Intrinsic parameters
00113         if cam in intr:
00114             data = intr[cam]
00115             K = data["cameraMatrix"]
00116             dist = data["distortionCoeffs"].ravel()
00117             err = data["reprojectionError"]
00118 
00119             print("Intrinsics:")
00120             print(" K =")
00121             print(f"  {K[0][0]:.3f} {K[0][1]:.3f} {K[0][2]:.3f}")
00122             print(f"  {K[1][0]:.3f} {K[1][1]:.3f} {K[1][2]:.3f}")
00123             print(f"  {K[2][0]:.3f} {K[2][1]:.3f} {K[2][2]:.3f}")
00124             print(" distCoeffs =", dist)
00125             print(f" reprojectionError = {err:.6f}")
```

```
00126         else:
00127             print("Intrinsics: (no data)")
00128 
00129         # Extrinsic parameters
00130         if cam in extr:
00131             data = extr[cam]
00132             R = data["rotationMatrix"]
00133             T = data["translationVector"].ravel()
00134             rms = data["stereoRms"]
00135 
00136             print("Extrinsics:")
00137             print(" R =")
00138             print(f"  {R[0][0]:.5f} {R[0][1]:.5f} {R[0][2]:.5f}")
00139             print(f"  {R[1][0]:.5f} {R[1][1]:.5f} {R[1][2]:.5f}")
00140             print(f"  {R[2][0]:.5f} {R[2][1]:.5f} {R[2][2]:.5f}")
00141             print(" T =", T)
00142             print(f" stereoRMS = {rms:.6f}")
00143         else:
00144             print("  Extrinsics: (no data)")
00145 
00146     # Print health score
00147     print("\nOverall Health Score")
00148     print("=" * 50)
00149     score = controller.selfHealthCheck(calibrationState)
00150     print(f"Global HealthScore = {score:.2f} / 100")
00151 
```

4.11.2.2 parse_args()

```
run_initial_calibration.parse_args ()

@brief Parse command line arguments for calibration configuration.

@return Parsed arguments with base_dir, num_poses, and bundle_adjust settings
```

Definition at line 34 of file run_initial_calibration.py.

```
00034 def parse_args():
00035     """
00036     @brief Parse command line arguments for calibration configuration.
00037 
00038     @return Parsed arguments with base_dir, num_poses, and bundle_adjust settings
00039     """
00040     parser = argparse.ArgumentParser(description="Run initial camera calibration")
00041     parser.add_argument("--base_dir", default=DEFAULT_BASE_DIR, help="Path to imageset directory")
00042     parser.add_argument("--num_poses", type=int, default=DEFAULT_NUM_POSES, help="Number of poses in
      the imageset")
00043     parser.add_argument(
00044         "--bundle-adjust",
00045         dest="bundle_adjust",
00046         action="store_true",
00047         help="Enable bundle adjustment (default)",
00048     )
00049     parser.add_argument(
00050         "--no-bundle-adjust",
00051         dest="bundle_adjust",
00052         action="store_false",
00053         help="Disable bundle adjustment",
00054     )
00055     parser.set_defaults(bundle_adjust=True)
00056     return parser.parse_args()
00057 
00058 
```

4.11.3 Variable Documentation

4.11.3.1 CAM_IDS

```
list run_initial_calibration.CAM_IDS
```

Initial value:

```
00001 =  [
00002     "Center",
00003     "Up1", "Up2", "Up3",
00004     "Down1", "Down2", "Down3",
00005     "Left1", "Left2", "Left3",
00006     "Right1", "Right2", "Right3",
00007 ]
```

Definition at line 25 of file run_initial_calibration.py.

4.11.3.2 DEFAULT_BASE_DIR

```
str run_initial_calibration.DEFAULT_BASE_DIR = "/data/calibrationLFC/sets/imageset5"
```

Definition at line 23 of file run_initial_calibration.py.

4.11.3.3 DEFAULT_NUM_POSES

```
int run_initial_calibration.DEFAULT_NUM_POSES = 28
```

Definition at line 24 of file run_initial_calibration.py.

4.11.3.4 repoRoot

```
run_initial_calibration.repoRoot = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

Definition at line 154 of file run_initial_calibration.py.

4.12 run_LiFCal_calibration Namespace Reference

Functions

- str read_text (str path)
- None write_text (str path, str s)
- str replace_yaml_key_line (str text, str key, str new_val)
- str patch_settings_yaml (str template_text, str focus_dir, str depth_dir, str mla_xml)
- int run_lifcal_recalib (str settings_path, str fixed_params_path, str store_dir)
- str find_results_folder (str store_dir)
- None copy_essentials (str result_dir, str out_cam_dir)
- main ()

Variables

- str LIFCAL_BIN = "/data/external/LiFCal/build/bin/LiFCal"
- str SETTINGS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Settings.yaml"
- str FIXED_PARAMS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Fixed_Paramerters.txt"
- str MLA_CALIB_XML = "/data/external/LiFCal/LiFCal_Data/Recalibration/MLA_Calibration.xml"
- str ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
- FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
- DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
- str OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/CalibrationByCamera"
- list CAMERA_IDS

4.12.1 Function Documentation

4.12.1.1 copy_essentials()

```
None run_LiFCal_calibration.copy_essentials (
            str result_dir,
            str out_cam_dir)
```

```
@brief Copy calibrationProtocol.txt and extrinsicOrientations.xml to output directory.

@param result_dir Source directory containing LiFCal results
@param out_cam_dir Destination directory for essential files
```

Definition at line 137 of file run_LiFCal_calibration.py.

```
00137 def copy_essentials(result_dir: str, out_cam_dir: str) -> None:
00138     """
00139     @brief Copy calibrationProtocol.txt and extrinsicOrientations.xml to output directory.
00140
00141     @param result_dir Source directory containing LiFCal results
00142     @param out_cam_dir Destination directory for essential files
00143     """
00144     proto = os.path.join(result_dir, "calibrationProtocol.txt")
00145     extr = os.path.join(result_dir, "extrinsicOrientations.xml")
00146
00147     missing = []
00148     if not os.path.isfile(proto):
00149         missing.append("calibrationProtocol.txt")
00150     if not os.path.isfile(extr):
00151         missing.append("extrinsicOrientations.xml")
00152     if missing:
00153         raise RuntimeError(f"Missing files in {result_dir}: {missing}")
00154
00155     shutil.copy2(proto, os.path.join(out_cam_dir, "calibrationProtocol.txt"))
00156     shutil.copy2(extr, os.path.join(out_cam_dir, "extrinsicOrientations.xml"))
00157
00158
```

4.12.1.2 find_results_folder()

```
str run_LiFCal_calibration.find_results_folder (
            str store_dir)
```

```
@brief Find LiFCal results folder.

@param store_dir LiFCal output directory
@return Path to results folder (typically Calibration_Results_... subdirectory)
```

Definition at line 114 of file run_LiFCal_calibration.py.

```
00114 def find_results_folder(store_dir: str) -> str:
00115     """
00116     @brief Find LiFCal results folder.
00117
00118     @param store_dir LiFCal output directory
00119     @return Path to results folder (typically Calibration_Results_... subdirectory)
00120     """
00121     if not os.path.isdir(store_dir):
00122         raise RuntimeError(f"store_dir does not exist: {store_dir}")
00123
00124     subdirs = []
00125     for name in os.listdir(store_dir):
00126         p = os.path.join(store_dir, name)
00127         if os.path.isdir(p):
00128             subdirs.append(p)
00129
00130     if not subdirs:
00131         return store_dir
00132
00133     subdirs.sort(key=lambda p: os.path.getmtime(p))
00134     return subdirs[-1]
00135
00136
```

4.12.1.3 main()

run_LiFCal_calibration.main ()

@brief Main pipeline: run LiFCal for each camera, export JSON, and log health.

Definition at line 159 of file run_LiFCal_calibration.py.

```
00159 def main():
00160     """
00161     @brief Main pipeline: run LiFCal for each camera, export JSON, and log health.
00162     """
00163     if not os.path.isfile(LIFCAL_BIN):
00164         raise SystemExit(f"LiFCal binary not found: {LIFCAL_BIN}")
00165     if not os.path.isfile(SETTINGS):
00166         raise SystemExit(f"Settings template not found: {SETTINGS}")
00167     if not os.path.isfile(FIXED_PARAMS):
00168         raise SystemExit(f"Fixed parameters file not found: {FIXED_PARAMS}")
00169     if not os.path.isfile(MLA_CALIB_XML):
00170         raise SystemExit(f"MLA_Calibration.xml not found: {MLA_CALIB_XML}")
00171 
00172     templ = read_text(SETTINGS)
00173 
00174     timestamp = datetime.now().strftime("%Y_%m_%d_%H%M%S")
00175     session_out = os.path.join(OUT_ROOT, f"run_{timestamp}")
00176     os.makedirs(session_out, exist_ok=True)
00177 
00178     print("Output:", session_out)
00179 
00180     for cam in CAMERA_IDS:
00181         focus_dir = os.path.join(FOCUS_ROOT, cam)
00182         depth_dir = os.path.join(DEPTH_ROOT, cam)
00183 
00184         if not os.path.isdir(focus_dir):
00185             print(f"[SKIP] {cam}: focus dir missing: {focus_dir}")
00186             continue
00187         if not os.path.isdir(depth_dir):
00188             print(f"[SKIP] {cam}: depth dir missing: {depth_dir}")
00189             continue
00190 
00191         cam_out = os.path.join(session_out, cam)
00192         os.makedirs(cam_out, exist_ok=True)
00193 
00194         settings_cam = os.path.join(cam_out, "Settings.yaml")
00195         patched = patch_settings_yaml(templ, focus_dir, depth_dir, MLA_CALIB_XML)
00196         write_text(settings_cam, patched)
00197 
00198         store_dir = os.path.join(cam_out, "lifcal_store")
00199         rc = run_lifcal_recalib(settings_cam, FIXED_PARAMS, store_dir)
00200 
00201         if rc != 0:
00202             print(f"[FAIL] {cam}: LiFCal returncode={rc} (siehe {store_dir}/lifcal_stdout.log)")
00203             continue
00204 
00205         result_dir = find_results_folder(store_dir)
00206 
00207         try:
00208             copy_essentials(result_dir, cam_out)   # ???iert protocol + xml nach cam_out
00209         except Exception as e:
00210             print(f"[FAIL] {cam}: {e}")
00211             print(f"        check log: {store_dir}/lifcal_stdout.log")
00212             continue
00213 
00214         # store dir wegräumen
00215         shutil.rmtree(store_dir, ignore_errors=True)
00216 
00217         # ? JSON EXPORT: result_dir muss cam_out sein, weil DA liegen jetzt die 2 Dateien
00218         try:
00219             json_path = lifcal_to_json.export_lifcal_parameters_json_in_place(
00220                 result_dir=cam_out,
00221                 cam_id=cam,
00222                 image_dir="LiFCal_Imageset",
00223                 run_type="recalib",
00224                 pixel_size_mm=0.0055,
00225                 fallback_cx=640.0,  # compatibility only
00226                 fallback_cy=360.0,  # compatibility only
00227                 out_name="parameters.json"
00228             )
00229         except Exception as e:
00230             print(f"[WARN] {cam}: JSON export failed: {e}")
00231 
00232     # Build combined JSON from all cameras
```

```
00233     try:
00234         combined_path = lifcal_to_json.build_combined_parameters_json(
00235             run_root=session_out,
00236             camera_ids=CAMERA_IDS,
00237             out_name="combined.json"
00238         )
00239         print("[OK] COMBINED:", combined_path)
00240     except Exception as e:
00241         print("[WARN] combined.json not created:", e)
00242 
00243     # Log health score and update baseline
00244     try:
00245         import sys
00246         CALIB_ROOT = "/data/calibrationLFC"
00247         if CALIB_ROOT not in sys.path:
00248             sys.path.insert(0, CALIB_ROOT)
00249 
00250         from ResultLogger import ResultLogger
00251         logger = ResultLogger(baseDir="/data/baseline")
00252         score = logger.logRecalibration(
00253             combined_json_path=combined_path,
00254             meta={
00255                 "runType": "recalib",
00256                 "imageDir": "LiFCal_Imageset",
00257                 "bundleAdjust": True
00258             }
00259         )
00260         print(f"[OK] HealthScore (lifcal) = {score:.2f}")
00261     except Exception as e:
00262         print("[WARN] LiFCal logging/health failed:", e)
00263 
00264 
```

4.12.1.4 patch_settings_yaml()

```
str run_LiFCal_calibration.patch_settings_yaml (
              str template_text,
              str focus_dir,
              str depth_dir,
              str mla_xml)
```

```
@brief Update YAML settings with camera-specific paths.

@param template_text Original YAML template content
@param focus_dir Path to focus images directory
@param depth_dir Path to depth maps directory
@param mla_xml Path to MLA calibration XML file
@return Modified YAML text with updated paths
```

Definition at line 69 of file run_LiFCal_calibration.py.

```
00069 def patch_settings_yaml(template_text: str, focus_dir: str, depth_dir: str, mla_xml: str) -> str:
00070     """
00071     @brief Update YAML settings with camera-specific paths.
00072 
00073     @param template_text Original YAML template content
00074     @param focus_dir Path to focus images directory
00075     @param depth_dir Path to depth maps directory
00076     @param mla_xml Path to MLA calibration XML file
00077     @return Modified YAML text with updated paths
00078     """
00079     out = template_text
00080     out = replace_yaml_key_line(out, "Path.totalFocusImages", focus_dir)
00081     out = replace_yaml_key_line(out, "Path.virtualDepthData", depth_dir)
00082     out = replace_yaml_key_line(out, "Path.microLensCalibration", mla_xml)
00083     return out
00084 
00085 
```

4.12.1.5 read_text()

```
str run_LiFCal_calibration.read_text (
              str path)
```

```
@brief Read text file with UTF-8 encoding.

@param path Path to the text file
@return File content as string
```

Definition at line 33 of file run_LiFCal_calibration.py.

```
00033 def read_text(path: str) -> str:
00034     """
00035     @brief Read text file with UTF-8 encoding.
00036
00037     @param path Path to the text file
00038     @return File content as string
00039     """
00040     with open(path, "r", encoding="utf-8", errors="replace") as f:
00041         return f.read()
00042
00043
```

4.12.1.6 replace_yaml_key_line()

```
str run_LiFCal_calibration.replace_yaml_key_line (
              str text,
              str key,
              str new_val)
```

```
@brief Replace YAML key-value lines.

@param text YAML text content
@param key YAML key to find and replace
@param new_val New value to set (will be quoted)
@return Modified YAML text
```

Definition at line 56 of file run_LiFCal_calibration.py.

```
00056 def replace_yaml_key_line(text: str, key: str, new_val: str) -> str:
00057     """
00058     @brief Replace YAML key-value lines.
00059
00060     @param text YAML text content
00061     @param key YAML key to find and replace
00062     @param new_val New value to set (will be quoted)
00063     @return Modified YAML text
00064     """
00065     pattern = re.compile(rf"^({re.escape(key)}\s*:\s*)(.*)$", re.MULTILINE)
00066     return pattern.sub(rf'\1"{new_val}"', text)
00067
00068
```

4.12.1.7 run_lifcal_recalib()

```
int run_LiFCal_calibration.run_lifcal_recalib (
              str settings_path,
              str fixed_params_path,
              str store_dir)
```

```
@brief Run LiFCal recalibration and return exit code.

@param settings_path Path to Settings.yaml file
@param fixed_params_path Path to fixed parameters file
@param store_dir Directory where LiFCal will store results
@return Exit code from LiFCal process
```

Definition at line 86 of file run_LiFCal_calibration.py.

```
00086 def run_lifcal_recalib(settings_path: str, fixed_params_path: str, store_dir: str) -> int:
00087     """
00088     @brief Run LiFCal recalibration and return exit code.
00089
00090     @param settings_path Path to Settings.yaml file
00091     @param fixed_params_path Path to fixed parameters file
00092     @param store_dir Directory where LiFCal will store results
00093     @return Exit code from LiFCal process
00094     """
00095     os.makedirs(store_dir, exist_ok=True)
00096
00097     cmd = [LIFCAL_BIN, "recalib", settings_path, fixed_params_path]
00098
00099     proc = subprocess.Popen(
00100         cmd,
00101         stdin=subprocess.PIPE,
00102         stdout=subprocess.PIPE,
00103         stderr=subprocess.STDOUT,
00104         text=True,
00105         cwd=os.path.dirname(LIFCAL_BIN),
00106     )
00107
00108     out, _ = proc.communicate(input=f"y\n{store_dir}\n")
00109     write_text(os.path.join(store_dir, "lifcal_stdout.log"), out)
00110
00111     return proc.returncode
00112
00113
```

4.12.1.8 write_text()

```
None run_LiFCal_calibration.write_text (
             str path,
             str s)
```

```
@brief Write text to file, creating directories if needed.

@param path Output file path
@param s Text content to write
```

Definition at line 44 of file run_LiFCal_calibration.py.

```
00044 def write_text(path: str, s: str) -> None:
00045     """
00046     @brief Write text to file, creating directories if needed.
00047
00048     @param path Output file path
00049     @param s Text content to write
00050     """
00051     os.makedirs(os.path.dirname(path), exist_ok=True)
00052     with open(path, "w", encoding="utf-8") as f:
00053         f.write(s)
00054
00055
```

4.12.2 Variable Documentation

4.12.2.1 CAMERA_IDS

```
list run_LiFCal_calibration.CAMERA_IDS
```

Initial value:

```
00001 =  [
00002     "Center", "Up1", "Up2", "Up3",
00003     "Down1", "Down2", "Down3",
00004     "Left1", "Left2", "Left3",
00005     "Right1", "Right2", "Right3"
00006 ]
```

Definition at line 25 of file run_LiFCal_calibration.py.

4.12.2.2 DEPTH_ROOT

```
run_LiFCal_calibration.DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
```

Definition at line 21 of file run_LiFCal_calibration.py.

4.12.2.3 FIXED_PARAMS

```
str run_LiFCal_calibration.FIXED_PARAMS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Fixed↩
_Paramerters.txt"
```

Definition at line 16 of file run_LiFCal_calibration.py.

4.12.2.4 FOCUS_ROOT

```
run_LiFCal_calibration.FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
```

Definition at line 20 of file run_LiFCal_calibration.py.

4.12.2.5 LIFCAL_BIN

```
str run_LiFCal_calibration.LIFCAL_BIN = "/data/external/LiFCal/build/bin/LiFCal"
```

Definition at line 13 of file run_LiFCal_calibration.py.

4.12.2.6 MLA_CALIB_XML

```
str run_LiFCal_calibration.MLA_CALIB_XML = "/data/external/LiFCal/LiFCal_Data/Recalibration/MLA↩
_Calibration.xml"
```

Definition at line 17 of file run_LiFCal_calibration.py.

4.12.2.7 OUT_ROOT

```
str run_LiFCal_calibration.OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/Calibration↩
ByCamera"
```

Definition at line 23 of file run_LiFCal_calibration.py.

4.12.2.8 ROOT_IMAGESET

```
str run_LiFCal_calibration.ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/Li↩
FCal_Imageset"
```

Definition at line 19 of file run_LiFCal_calibration.py.

4.12.2.9 SETTINGS

```
str run_LiFCal_calibration.SETTINGS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Settings.↩
yaml"
```

Definition at line 15 of file run_LiFCal_calibration.py.

4.13 run_periodic_lifcal Namespace Reference

Functions

- setup_logger ()
- bool acquire_lock (logger)
- release_lock (logger)
- int run_cmd (logger, cmd, cwd=None)
- int count_images_in_dir (str d, exts=(".png", ".jpg", ".jpeg"))
- bool needs_imageset_creation (logger)
- str find_latest_run_dir (str out_root)
- str combined_json_path (str run_dir)
- try_log_lifcal_with_resultlogger (logger, str combined_path)
- run_cycle (logger)
- main ()

Variables

- int INTERVAL_SECONDS = 5 ∗ 60
- str RUN_IMAGESET_CREATION = "/data/external/LiFCal/LiFCal_Data/run_imageset_creation.py"
- str RUN_LIFCAL_CALIB = "/data/external/LiFCal/LiFCal_Data/run_LiFCal_calibration.py"
- str LIFCAL_OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/CalibrationByCamera"
- str BASELINE_DIR = "/data/baseline"
- LOG_PATH = os.path.join(BASELINE_DIR, "calibration.log")
- str LOCK_PATH = "/tmp/lifcal_periodic.lock"
- str ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
- FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
- DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
- list CAMERA_IDS
- int MIN_POSES_PER_CAM = 1

4.13.1 Function Documentation

4.13.1.1 acquire_lock()

```
bool run_periodic_lifcal.acquire_lock (
              logger)


@brief Create lock file to prevent overlapping scheduler cycles.

@param logger Logger instance for status messages
@return True if lock was acquired, False otherwise
```

Definition at line 63 of file run_periodic_lifcal.py.

```
00063 def acquire_lock(logger) -> bool:
00064     """
00065     @brief Create lock file to prevent overlapping scheduler cycles.
00066
00067     @param logger Logger instance for status messages
00068     @return True if lock was acquired, False otherwise
00069     """
00070     if os.path.exists(LOCK_PATH):
00071         logger.warning(f"Lock exists, skipping this cycle: {LOCK_PATH}")
00072         return False
00073     try:
00074         with open(LOCK_PATH, "w") as f:
00075             f.write(str(os.getpid()))
00076         return True
00077     except Exception as e:
00078         logger.error(f"Could not create lock: {e}")
00079         return False
00080
00081
```

4.13.1.2 combined_json_path()

```
str run_periodic_lifcal.combined_json_path (
             str run_dir)

@brief Return path to combined.json inside run directory if present.

@param run_dir Path to run directory
@return Path to combined.json file, or empty string if not found
```

Definition at line 192 of file run_periodic_lifcal.py.

```
00192 def combined_json_path(run_dir: str) -> str:
00193     """
00194     @brief Return path to combined.json inside run directory if present.
00195
00196     @param run_dir Path to run directory
00197     @return Path to combined.json file, or empty string if not found
00198     """
00199     if not run_dir:
00200         return ""
00201     p = os.path.join(run_dir, "combined.json")
00202     return p if os.path.isfile(p) else ""
00203
00204
```

4.13.1.3 count_images_in_dir()

```
int run_periodic_lifcal.count_images_in_dir (
             str d,
              exts = (".png", ".jpg", ".jpeg"))

@brief Count images in directory matching expected extensions.

@param d Directory path to scan
@param exts Tuple of file extensions to match (default: (".png", ".jpg", ".jpeg"))
@return Number of matching image files
```

Definition at line 124 of file run_periodic_lifcal.py.

```
00124 def count_images_in_dir(d: str, exts=(".png", ".jpg", ".jpeg")) -> int:
00125     """
00126     @brief Count images in directory matching expected extensions.
00127
00128     @param d Directory path to scan
00129     @param exts Tuple of file extensions to match (default: (".png", ".jpg", ".jpeg"))
00130     @return Number of matching image files
00131     """
00132     if not os.path.isdir(d):
00133         return 0
00134     cnt = 0
00135     for fn in os.listdir(d):
00136         if fn.lower().endswith(exts):
00137             cnt += 1
00138     return cnt
00139
00140
```

4.13.1.4 find_latest_run_dir()

```
str run_periodic_lifcal.find_latest_run_dir (
             str out_root)
```

```
@brief Return latest run_* directory inside output root, or empty string if none.

@param out_root Root directory containing run_* folders
@return Path to the latest run directory, or empty string
```

Definition at line 172 of file run_periodic_lifcal.py.

```
00172 def find_latest_run_dir(out_root: str) -> str:
00173     """
00174     @brief Return latest run_* directory inside output root, or empty string if none.
00175
00176     @param out_root Root directory containing run_* folders
00177     @return Path to the latest run directory, or empty string
00178     """
00179     if not os.path.isdir(out_root):
00180         return ""
00181     runs = []
00182     for name in os.listdir(out_root):
00183         p = os.path.join(out_root, name)
00184         if os.path.isdir(p) and name.startswith("run_"):
00185             runs.append(p)
00186     if not runs:
00187         return ""
00188     runs.sort(key=lambda p: os.path.getmtime(p))
00189     return runs[-1]
00190
00191
```

4.13.1.5 main()

```
run_periodic_lifcal.main ()
```

```
@brief Periodic loop that runs LiFCal recalibration on a schedule.
```

Definition at line 277 of file run_periodic_lifcal.py.

```
00277 def main():
00278     """
00279     @brief Periodic loop that runs LiFCal recalibration on a schedule.
00280     """
00281     logger = setup_logger()
00282     logger.info("Starting periodic LiFCal scheduler.")
00283     logger.info(f"Interval: {INTERVAL_SECONDS}s")
00284     logger.info(f"Lock: {LOCK_PATH}")
00285
00286     while True:
00287         start = time.time()
00288
00289         if acquire_lock(logger):
00290             try:
00291                 logger.info("=== CYCLE START ===")
00292                 run_cycle(logger)
00293                 logger.info("=== CYCLE END ===")
00294             finally:
00295                 release_lock(logger)
00296
00297         elapsed = time.time() - start
00298         sleep_s = max(1.0, INTERVAL_SECONDS - elapsed)
00299         logger.info(f"Sleeping {sleep_s:.1f}s\n")
00300         time.sleep(sleep_s)
00301
00302
```

4.13.1.6 needs_imageset_creation()

```
bool run_periodic_lifcal.needs_imageset_creation (
              logger)
```

```
@brief Check if imageset exists and has enough images per camera; trigger creation if not.

@param logger Logger instance for status messages
@return True if imageset needs to be created, False if it already exists
```

Definition at line 141 of file run_periodic_lifcal.py.

```
00141 def needs_imageset_creation(logger) -> bool:
00142     """
00143     @brief Check if imageset exists and has enough images per camera; trigger creation if not.
00144
00145     @param logger Logger instance for status messages
00146     @return True if imageset needs to be created, False if it already exists
00147     """
00148     if not os.path.isdir(ROOT_IMAGESET):
00149         logger.info("Imageset missing: ROOT_IMAGESET not found.")
00150         return True
00151     if not os.path.isdir(FOCUS_ROOT) or not os.path.isdir(DEPTH_ROOT):
00152         logger.info("Imageset missing: focus/depth folders not found.")
00153         return True
00154
00155     for cam in CAMERA_IDS:
00156         fdir = os.path.join(FOCUS_ROOT, cam)
00157         ddir = os.path.join(DEPTH_ROOT, cam)
00158
00159         fcnt = count_images_in_dir(fdir, exts=(".png", ".jpg", ".jpeg"))
00160         dcnt = count_images_in_dir(ddir, exts=(".png", ".jpg", ".jpeg"))
00161
00162         if fcnt < MIN_POSES_PER_CAM:
00163             logger.info(f"Imageset not ready: focus/{cam} has {fcnt} images (<{MIN_POSES_PER_CAM}).")
00164             return True
00165         if dcnt < MIN_POSES_PER_CAM:
00166             logger.info(f"Imageset not ready: depth/{cam} has {dcnt} images (<{MIN_POSES_PER_CAM}).")
00167             return True
00168     logger.info("Imageset looks ready. Skipping run_imageset_creation.py.")
00169     return False
00170
00171
```

4.13.1.7 release_lock()

```
run_periodic_lifcal.release_lock (
              logger)
```

```
@brief Remove lock file after a cycle completes.

@param logger Logger instance for error messages
```

Definition at line 82 of file run_periodic_lifcal.py.

```
00082 def release_lock(logger):
00083     """
00084     @brief Remove lock file after a cycle completes.
00085
00086     @param logger Logger instance for error messages
00087     """
00088     try:
00089         if os.path.exists(LOCK_PATH):
00090             os.remove(LOCK_PATH)
00091     except Exception as e:
00092         logger.error(f"Could not remove lock: {e}")
00093
00094
```

4.13.1.8 run_cmd()

```
int run_periodic_lifcal.run_cmd (
              logger,
              cmd,
              cwd = None)
```

```
@brief Run a subprocess, log combined stdout/stderr, and return exit code.

@param logger Logger instance for command output
@param cmd Command as list of strings
@param cwd Working directory for the command (default: None)
@return Exit code of the subprocess
```

Definition at line 95 of file run_periodic_lifcal.py.

```
00095 def run_cmd(logger, cmd, cwd=None) -> int:
00096     """
00097     @brief Run a subprocess, log combined stdout/stderr, and return exit code.
00098
00099     @param logger Logger instance for command output
00100     @param cmd Command as list of strings
00101     @param cwd Working directory for the command (default: None)
00102     @return Exit code of the subprocess
00103     """
00104     logger.info(f"RUN: {' '.join(cmd)}")
00105     try:
00106         p = subprocess.run(
00107             cmd,
00108             cwd=cwd,
00109             stdout=subprocess.PIPE,
00110             stderr=subprocess.STDOUT,
00111             text=True
00112         )
00113         out = p.stdout or ""
00114         if len(out) > 5000:
00115             logger.info(out[:5000] + "\n... (truncated) ...")
00116         else:
00117             logger.info(out)
00118         return int(p.returncode)
00119     except Exception as e:
00120         logger.error(f"Command failed: {e}", exc_info=True)
00121         return 1
00122
00123
```

4.13.1.9 run_cycle()

```
run_periodic_lifcal.run_cycle (
              logger)
```

```
@brief Execute one scheduler cycle: prepare imageset, run LiFCal, log health.

@param logger Logger instance for cycle logging
```

Definition at line 234 of file run_periodic_lifcal.py.

```
00234 def run_cycle(logger):
00235     """
00236     @brief Execute one scheduler cycle: prepare imageset, run LiFCal, log health.
00237
00238     @param logger Logger instance for cycle logging
00239     """
00240     # 1) Only prepare imageset if needed
00241     if needs_imageset_creation(logger):
00242         if not os.path.isfile(RUN_IMAGESET_CREATION):
00243             logger.error(f"Missing: {RUN_IMAGESET_CREATION}")
00244             return
00245
00246         rc = run_cmd(logger, [sys.executable, RUN_IMAGESET_CREATION])
00247         if rc != 0:
00248             logger.error(f"run_imageset_creation failed (rc={rc}). Skipping LiFCal.")
```

```
00249             return
00250
00251     # 2) LiFCal calibration
00252     if not os.path.isfile(RUN_LIFCAL_CALIB):
00253         logger.error(f"Missing: {RUN_LIFCAL_CALIB}")
00254         return
00255
00256     rc = run_cmd(logger, [sys.executable, RUN_LIFCAL_CALIB])
00257     if rc != 0:
00258         logger.error(f"run_LiFCal_calibration failed (rc={rc}).")
00259         return
00260
00261     # 3) locate combined.json (latest run)
00262     latest = find_latest_run_dir(LIFCAL_OUT_ROOT)
00263     cpath = combined_json_path(latest)
00264
00265     if not cpath:
00266         logger.warning("No combined.json found after LiFCal run.")
00267         return
00268
00269     logger.info(f"Latest combined.json: {cpath}")
00270
00271     # 4) log health + baseline update using your ResultLogger
00272     ok = try_log_lifcal_with_resultlogger(logger, cpath)
00273     if not ok:
00274         logger.info("LiFCal finished, but no baseline/health logging was executed.")
00275
00276
```

4.13.1.10 setup_logger()

```
run_periodic_lifcal.setup_logger ()
```

@brief Create logger that writes to file and stdout.

@return Logger instance configured for calibration logging

Definition at line 41 of file run_periodic_lifcal.py.

```
00041 def setup_logger():
00042     """
00043     @brief Create logger that writes to file and stdout.
00044
00045     @return Logger instance configured for calibration logging
00046     """
00047     os.makedirs(BASELINE_DIR, exist_ok=True)
00048     logger = logging.getLogger("lifcal_periodic")
00049     logger.setLevel(logging.INFO)
00050
00051     if not logger.handlers:
00052         fh = logging.FileHandler(LOG_PATH, encoding="utf-8")
00053         fh.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00054         logger.addHandler(fh)
00055
00056         sh = logging.StreamHandler(sys.stdout)
00057         sh.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00058         logger.addHandler(sh)
00059
00060     return logger
00061
00062
```

4.13.1.11 try_log_lifcal_with_resultlogger()

```
run_periodic_lifcal.try_log_lifcal_with_resultlogger (
               logger,
              str combined_path)
```

@brief Use ResultLogger to record LiFCal health score and update baseline.

@param logger Logger instance for status messages
@param combined_path Path to combined.json from LiFCal
@return True if logging succeeded, False otherwise

Definition at line 205 of file run_periodic_lifcal.py.

```
00205 def try_log_lifcal_with_resultlogger(logger, combined_path: str):
00206     """
00207     @brief Use ResultLogger to record LiFCal health score and update baseline.
00208
00209     @param logger Logger instance for status messages
00210     @param combined_path Path to combined.json from LiFCal
00211     @return True if logging succeeded, False otherwise
00212     """
00213     try:
00214         repo_dir = "/data/calibrationLFC"
00215         if repo_dir not in sys.path:
00216             sys.path.insert(0, repo_dir)
00217
00218         from ResultLogger import ResultLogger
00219         rl = ResultLogger(baseDir=BASELINE_DIR)
00220
00221         meta = {
00222             "runType": "recalib",
00223             "source": "periodic_scheduler",
00224             "timestamp": datetime.now().isoformat(timespec="seconds"),
00225         }
00226         score = rl.logRecalibration(combined_path, meta=meta)
00227         logger.info(f"LiFCal HealthScore logged via ResultLogger: {score:.2f}")
00228         return True
00229     except Exception as e:
00230         logger.warning(f"Could not use ResultLogger for LiFCal logging: {e}")
00231         return False
00232
00233
```

4.13.2 Variable Documentation

4.13.2.1 BASELINE_DIR

```
str run_periodic_lifcal.BASELINE_DIR = "/data/baseline"
```

Definition at line 20 of file run_periodic_lifcal.py.

4.13.2.2 CAMERA_IDS

```
list run_periodic_lifcal.CAMERA_IDS
```

Initial value:

```
00001 =  [
00002     "Center", "Up1", "Up2", "Up3",
00003     "Down1", "Down2", "Down3",
00004     "Left1", "Left2", "Left3",
00005     "Right1", "Right2", "Right3"
00006 ]
```

Definition at line 30 of file run_periodic_lifcal.py.

4.13.2.3 DEPTH_ROOT

```
run_periodic_lifcal.DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
```

Definition at line 28 of file run_periodic_lifcal.py.

4.13.2.4 FOCUS_ROOT

```
run_periodic_lifcal.FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
```

Definition at line 27 of file run_periodic_lifcal.py.

4.13.2.5 INTERVAL_SECONDS

```
int run_periodic_lifcal.INTERVAL_SECONDS = 5 * 60
```

Definition at line 14 of file run_periodic_lifcal.py.

4.13.2.6 LIFCAL_OUT_ROOT

```
str run_periodic_lifcal.LIFCAL_OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/Calibration↩
ByCamera"
```

Definition at line 19 of file run_periodic_lifcal.py.

4.13.2.7 LOCK_PATH

```
str run_periodic_lifcal.LOCK_PATH = "/tmp/lifcal_periodic.lock"
```

Definition at line 23 of file run_periodic_lifcal.py.

4.13.2.8 LOG_PATH

```
run_periodic_lifcal.LOG_PATH = os.path.join(BASELINE_DIR, "calibration.log")
```

Definition at line 21 of file run_periodic_lifcal.py.

4.13.2.9 MIN_POSES_PER_CAM

```
int run_periodic_lifcal.MIN_POSES_PER_CAM = 1
```

Definition at line 38 of file run_periodic_lifcal.py.

4.13.2.10 ROOT_IMAGESET

```
str run_periodic_lifcal.ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/Li↩
FCal_Imageset"
```

Definition at line 26 of file run_periodic_lifcal.py.

4.13.2.11 RUN_IMAGESET_CREATION

```
str run_periodic_lifcal.RUN_IMAGESET_CREATION = "/data/external/LiFCal/LiFCal_Data/run_↩
imageset_creation.py"
```

Definition at line 16 of file run_periodic_lifcal.py.

4.13.2.12 RUN_LIFCAL_CALIB

```
str run_periodic_lifcal.RUN_LIFCAL_CALIB = "/data/external/LiFCal/LiFCal_Data/run_LiFCal_↩
calibration.py"
```

Definition at line 17 of file run_periodic_lifcal.py.

Chapter 5

Class Documentation

5.1 CalibrationState.CalibrationState Class Reference

Public Member Functions

- __init__ (self)
- setIntrinsics (self, camId, cameraMatrix, distortionCoeffs, repError)
- setExtrinsics (self, camId, rotationMatrix, translationVector, stereoRMS)
- setTimeStamp (self, timestamp)
- __getState__ (self)

Public Attributes

- timeStamp = timestamp
- intrinsics
- extrinsics

5.1.1 Detailed Description

```
@brief Stores the complete calibration state for a multi-camera system.

@details This class manages intrinsic and extrinsic parameters for all cameras
in the system. It provides methods to set and retrieve calibration data including
camera matrices, distortion coefficients, rotation matrices, and translation vectors.
```

Definition at line 1 of file CalibrationState.py.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 __init__()

```
CalibrationState.CalibrationState.__init__ (
              self)
```

```
@brief Constructor for CalibrationState.

@details Initializes empty dictionaries for intrinsic and extrinsic parameters
and sets the timestamp to None.
```

Definition at line 10 of file CalibrationState.py.

```
00010     def __init__(self):
00011         """
00012         @brief Constructor for CalibrationState.
00013 
00014         @details Initializes empty dictionaries for intrinsic and extrinsic parameters
00015         and sets the timestamp to None.
00016         """
00017         self.intrinsics = {}
00018         self.extrinsics = {}
00019         self.timeStamp = None
00020 
```

5.1.3 Member Function Documentation

5.1.3.1 __getState__()

```
CalibrationState.CalibrationState.__getState__ (
              self)
```

```
@brief Return the complete calibration state as dictionary.

@return Dictionary containing intrinsics, extrinsics, and timestamp
```

Definition at line 59 of file CalibrationState.py.

```
00059     def __getState__(self):
00060         """
00061         @brief Return the complete calibration state as dictionary.
00062 
00063         @return Dictionary containing intrinsics, extrinsics, and timestamp
00064         """
00065         return {
00066             "intrinsics": self.intrinsics,
00067             "extrinsics": self.extrinsics,
00068             "timeStamp": self.timeStamp
00069         }
```

5.1.3.2 setExtrinsics()

```
CalibrationState.CalibrationState.setExtrinsics (
              self,
              camId,
              rotationMatrix,
              translationVector,
              stereoRMS)
```

```
@brief Store extrinsic calibration parameters for a camera.

@param camId Camera identifier
@param rotationMatrix 3x3 rotation matrix representing camera orientation
@param translationVector 3x1 translation vector representing camera position
@param stereoRMS Root mean square error of the stereo calibration
```

Definition at line 36 of file CalibrationState.py.

```
00036     def setExtrinsics(self, camId, rotationMatrix, translationVector, stereoRMS):
00037         """
00038         @brief Store extrinsic calibration parameters for a camera.
00039
00040         @param camId Camera identifier
00041         @param rotationMatrix 3x3 rotation matrix representing camera orientation
00042         @param translationVector 3x1 translation vector representing camera position
00043         @param stereoRMS Root mean square error of the stereo calibration
00044         """
00045         self.extrinsics[camId] = {
00046             "rotationMatrix": rotationMatrix,
00047             "translationVector": translationVector,
00048             "stereoRms": stereoRMS
00049         }
00050
```

5.1.3.3 setIntrinsics()

```
CalibrationState.CalibrationState.setIntrinsics (
              self,
              camId,
              cameraMatrix,
              distortionCoeffs,
              repError)
```

```
@brief Store intrinsic calibration parameters for a camera.

@param camId Camera identifier
@param cameraMatrix 3x3 camera matrix containing focal lengths and principal point
@param distortionCoeffs Distortion coefficients vector
@param repError Reprojection error of the calibration
```

Definition at line 21 of file CalibrationState.py.

```
00021     def setIntrinsics(self, camId, cameraMatrix, distortionCoeffs, repError):
00022         """
00023         @brief Store intrinsic calibration parameters for a camera.
00024
00025         @param camId Camera identifier
00026         @param cameraMatrix 3x3 camera matrix containing focal lengths and principal point
00027         @param distortionCoeffs Distortion coefficients vector
00028         @param repError Reprojection error of the calibration
00029         """
00030         self.intrinsics[camId] = {
00031             "cameraMatrix": cameraMatrix,
00032             "distortionCoeffs": distortionCoeffs,
00033             "reprojectionError": repError
00034         }
00035
```

5.1.3.4 setTimeStamp()

```
CalibrationState.CalibrationState.setTimeStamp (
              self,
              timestamp)
```

```
@brief Set the calibration timestamp.

@param timestamp Timestamp indicating when the calibration was performed
```

Definition at line 51 of file CalibrationState.py.

```
00051     def setTimeStamp(self, timestamp):
00052         """
00053         @brief Set the calibration timestamp.
00054 
00055         @param timestamp Timestamp indicating when the calibration was performed
00056         """
00057         self.timeStamp = timestamp
00058 
```

5.1.4 Member Data Documentation

5.1.4.1 extrinsics

`CalibrationState.CalibrationState.extrinsics`

Definition at line 67 of file CalibrationState.py.

5.1.4.2 intrinsics

`CalibrationState.CalibrationState.intrinsics`

Definition at line 66 of file CalibrationState.py.

5.1.4.3 timeStamp

`CalibrationState.CalibrationState.timeStamp = timestamp`

Definition at line 57 of file CalibrationState.py.

The documentation for this class was generated from the following file:

- calibrationLFC/CalibrationState.py

5.2 Controller.Controller Class Reference

Public Member Functions

- __init__ (self, bundleAdjust)
- runInitialCalibration (self, imageSet)
- float selfHealthCheck (self, calibrationState)

Public Attributes

- initialCalibration = InitialCalibration()
- list scoreHistory = [ ]
- currentState = None
- resultLogger = ResultLogger()
- bundleAdjust = bundleAdjust

5.2.1 Detailed Description

```
@brief Main controller for managing calibration workflow.

@details Orchestrates the complete calibration process including initial calibration,
health monitoring, and result logging. Manages the calibration state and maintains
a history of health scores.
```

Definition at line 6 of file Controller.py.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 __init__()

```
Controller.Controller.__init__ (
              self,
              bundleAdjust)


@brief Constructor for Controller.

@param bundleAdjust Boolean flag indicating whether to apply bundle adjustment

@details Initializes the controller with an InitialCalibration instance,
empty score history, result logger, and bundle adjustment setting.
```

Definition at line 15 of file Controller.py.

```
00015     def __init__(self, bundleAdjust):
00016         """
00017         @brief Constructor for Controller.
00018
00019         @param bundleAdjust Boolean flag indicating whether to apply bundle adjustment
00020
00021         @details Initializes the controller with an InitialCalibration instance,
00022         empty score history, result logger, and bundle adjustment setting.
00023         """
00024         self.initialCalibration = InitialCalibration()
00025         self.scoreHistory = []
00026         self.currentState = None
00027         self.resultLogger = ResultLogger()
00028         self.bundleAdjust = bundleAdjust
00029
```

5.2.3 Member Function Documentation

5.2.3.1 runInitialCalibration()

```
Controller.Controller.runInitialCalibration (
              self,
              imageSet)


@brief Execute initial calibration for all cameras in the image set.

@param imageSet ImageSet object containing calibration images and camera information
@return CalibrationState object with computed intrinsic and extrinsic parameters
```

Definition at line 30 of file Controller.py.

```
00030     def runInitialCalibration(self, imageSet):
00031         """
00032         @brief Execute initial calibration for all cameras in the image set.
00033 
00034         @param imageSet ImageSet object containing calibration images and camera information
00035         @return CalibrationState object with computed intrinsic and extrinsic parameters
00036 
00037         """
00038         print("Controller: starting initial calibration...")
00039         calibrationState = self.initialCalibration.run(imageSet, self.bundleAdjust)
00040 
00041         # Prepare metadata for logging
00042         meta = {
00043             "runType": "initial",
00044             "numPoses": imageSet.numPoses,
00045             "imageDir": imageSet.baseDir,
00046             "cameraIds": imageSet.cameraIds,
00047             "bundleAdjust": self.bundleAdjust,
00048         }
00049         self.resultLogger.logInitialCalibration(calibrationState, meta=meta)
00050 
00051         return calibrationState
00052 
```

5.2.3.2 selfHealthCheck()

```
float Controller.Controller.selfHealthCheck (
              self,
              calibrationState)
```

```
@brief Computes a global health score for the calibration quality.

@param calibrationState CalibrationState object to evaluate
@return Health score as float between 0 and 100, or 0.0 if evaluation fails
```

Definition at line 53 of file Controller.py.

```
00053     def selfHealthCheck(self, calibrationState) -> float:
00054         """
00055         @brief Computes a global health score for the calibration quality.
00056 
00057         @param calibrationState CalibrationState object to evaluate
00058         @return Health score as float between 0 and 100, or 0.0 if evaluation fails
00059 
00060         """
00061         # Extract state dictionary
00062         try:
00063             state_dict = calibrationState.__getState__()
00064         except Exception as e:
00065             self.resultLogger.logger.error(
00066                 f"HealthCheck: could not extract state dict: {e}", exc_info=True
00067             )
00068             return 0.0
00069 
00070         # Compute health score from calibration metrics
00071         try:
00072             report = compute_initial_health_from_state(state_dict)
00073             score = float(report["health"])
00074             self.resultLogger.logger.info(
00075                 f"HealthScore = {score:.2f} (method={report.get('method')})"
00076             )
00077             return score
00078         except Exception as e:
00079             self.resultLogger.logger.error(f"HealthCheck failed: {e}", exc_info=True)
00080             return 0.0
```

5.2.4 Member Data Documentation

5.2.4.1 bundleAdjust

```
Controller.Controller.bundleAdjust = bundleAdjust
```

Definition at line 28 of file Controller.py.

5.2.4.2 currentState

```
Controller.Controller.currentState = None
```

Definition at line 26 of file Controller.py.

5.2.4.3 initialCalibration

```
Controller.Controller.initialCalibration = InitialCalibration()
```

Definition at line 24 of file Controller.py.

5.2.4.4 resultLogger

```
Controller.Controller.resultLogger = ResultLogger()
```

Definition at line 27 of file Controller.py.

5.2.4.5 scoreHistory

```
list Controller.Controller.scoreHistory = []
```

Definition at line 25 of file Controller.py.

The documentation for this class was generated from the following file:

- calibrationLFC/Controller.py

5.3 ImageSet.ImageSet Class Reference

Public Member Functions

- __init__ (self, str baseDir, int numPoses, List[str] cameraIds)
- str getImagePath (self, int poseIndex, str cameraId)

Public Attributes

- baseDir = baseDir
- numPoses = numPoses
- cameraIds = cameraIds

5.3.1 Detailed Description

```
@brief Represents a set of calibration images organized by pose and camera.
```

Definition at line 3 of file ImageSet.py.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 __init__()

```
ImageSet.ImageSet.__init__ (
              self,
             str baseDir,
             int numPoses,
             List[str] cameraIds)
```

```
@brief Constructor for ImageSet.

@param baseDir Base directory containing calibration images
@param numPoses Total number of poses in the calibration set
@param cameraIds List of camera identifiers
```

Definition at line 8 of file ImageSet.py.

```
00008     def __init__(self, baseDir: str, numPoses: int, cameraIds: List[str]):
00009         """
00010         @brief Constructor for ImageSet.
00011
00012         @param baseDir Base directory containing calibration images
00013         @param numPoses Total number of poses in the calibration set
00014         @param cameraIds List of camera identifiers
00015         """
00016         self.baseDir = baseDir
00017         self.numPoses = numPoses
00018         self.cameraIds = cameraIds
00019
```

5.3.3 Member Function Documentation

5.3.3.1 getImagePath()

```
str ImageSet.ImageSet.getImagePath (
              self,
             int poseIndex,
             str cameraId)
```

```
@brief Construct the file path for a specific pose and camera.

@param poseIndex Zero-based pose index
@param cameraId Camera identifier string
@return String path in format: baseDir/pose_XXX_cameraId.png
```

Definition at line 20 of file ImageSet.py.

```
00020     def getImagePath(self, poseIndex: int, cameraId: str) -> str:
00021         """
00022         @brief Construct the file path for a specific pose and camera.
00023
00024         @param poseIndex Zero-based pose index
00025         @param cameraId Camera identifier string
00026         @return String path in format: baseDir/pose_XXX_cameraId.png
00027         """
00028         poseStr = f"pose_{poseIndex:03d}"
00029         return f"{self.baseDir}/{poseStr}_{cameraId}.png"
00030
00031         """
00032             Beispiel:
00033             baseDir = "/data/images"
00034             poseIndex = 5  → poseStr = "pose_005"
00035             cameraId = "center"
00036
00037             Ergebnis: "/data/images/pose_005_center.png"
00038         """
```

5.3.4 Member Data Documentation

5.3.4.1 baseDir

```
ImageSet.ImageSet.baseDir = baseDir
```

Definition at line 16 of file ImageSet.py.

5.3.4.2 cameraIds

```
ImageSet.ImageSet.cameraIds = cameraIds
```

Definition at line 18 of file ImageSet.py.

5.3.4.3 numPoses

```
ImageSet.ImageSet.numPoses = numPoses
```

Definition at line 17 of file ImageSet.py.

The documentation for this class was generated from the following file:

- calibrationLFC/ImageSet.py

5.4 InitialCalibration.InitialCalibration Class Reference

Public Member Functions

- __init__ (self, chessboardSize=(8, 8), squareSize=2/9)
- createObjectPoints (self)
- detectCorners (self, imagePath)
- calibrateIntrinsics (self, imageSet)
- calibrateExtrinsics (self, imageSet, state, refCamId)
- bundleAdjustExtrinsics (self, imageSet, state, refCamId)
- run (self, imageSet, bundleAdjust=True)

Public Attributes

- chessboardSize = chessboardSize
- squareSize = squareSize
- tuple criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

5.4.1 Detailed Description

```
@brief Multi-camera calibration using chessboard patterns and bundle adjustment.
```

Definition at line 7 of file InitialCalibration.py.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 __init__()

```
InitialCalibration.InitialCalibration.__init__ (
              self,
              chessboardSize = (8,8),
              squareSize = 2/9)


@brief Constructor for InitialCalibration.

@param chessboardSize Tuple (cols, rows) defining chessboard dimensions (default: (8,8))
@param squareSize Physical size of one chessboard square in meters (default: 2/9)
```

Definition at line 12 of file InitialCalibration.py.

```
00012     def __init__(self, chessboardSize=(8,8), squareSize=2/9):
00013         """
00014         @brief Constructor for InitialCalibration.
00015
00016         @param chessboardSize Tuple (cols, rows) defining chessboard dimensions (default: (8,8))
00017         @param squareSize Physical size of one chessboard square in meters (default: 2/9)
00018         """
00019         self.chessboardSize = chessboardSize
00020         self.squareSize = squareSize
00021         self.criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
00022
```

5.4.3 Member Function Documentation

5.4.3.1 bundleAdjustExtrinsics()

```
InitialCalibration.InitialCalibration.bundleAdjustExtrinsics (
              self,
              imageSet,
              state,
              refCamId)


@brief Global bundle adjustment optimizing extrinsics and board poses.

@param imageSet ImageSet object containing calibration images
@param state CalibrationState with initial intrinsics and extrinsics
@param refCamId Camera ID to use as reference coordinate system
@return Updated CalibrationState with refined extrinsics after bundle adjustment
```

Definition at line 156 of file InitialCalibration.py.

```
00156     def bundleAdjustExtrinsics(self, imageSet, state, refCamId):
00157         """
00158         @brief Global bundle adjustment optimizing extrinsics and board poses.
00159
00160         @param imageSet ImageSet object containing calibration images
00161         @param state CalibrationState with initial intrinsics and extrinsics
00162         @param refCamId Camera ID to use as reference coordinate system
00163         @return Updated CalibrationState with refined extrinsics after bundle adjustment
00164         """
00165         objp = self.createObjectPoints()
00166         cameraIds = list(imageSet.cameraIds)
00167         if refCamId not in cameraIds:
00168             raise ValueError("refCamId not in imageSet.cameraIds")
00169
00170         ref_idx = cameraIds.index(refCamId)
00171
00172         # Initialize board poses via PnP-RANSAC in reference camera
00173         K_ref = state.intrinsics[refCamId]["cameraMatrix"]
00174         dist_ref = state.intrinsics[refCamId]["distortionCoeffs"]
```

```
00175
00176         pose_rvecs = {}
00177         pose_tvecs = {}
00178         valid_poses = []
00179
00180         for pose in range(imageSet.numPoses):
00181             img_path = imageSet.getImagePath(pose, refCamId)
00182             ok, corners, size = self.detectCorners(img_path)
00183             if not ok:
00184                 continue
00185
00186             # Use PnP-RANSAC for robust board pose estimation
00187             ok_pnp, rvec, tvec, inliers = cv2.solvePnPRansac(
00188                 objp,
00189                 corners,
00190                 K_ref,
00191                 dist_ref,
00192                 reprojectionError=2.0,
00193                 confidence=0.99,
00194                 flags=cv2.SOLVEPNP_ITERATIVE
00195             )
00196
00197             if not ok_pnp or inliers is None or len(inliers) < 10:
00198                 # Skip pose if too few reliable points
00199                 continue
00200
00201             pose_rvecs[pose] = rvec.astype(np.float64).reshape(3)
00202             pose_tvecs[pose] = tvec.astype(np.float64).reshape(3)
00203             valid_poses.append(pose)
00204
00205         if len(valid_poses) < 3:
00206             print("[BA] Not enough valid poses for bundle adjustment.")
00207             return state
00208
00209         # Initialize extrinsics from state
00210         # Only optimize non-reference cameras
00211         optim_cams = []
00212         cam_param_index = {}
00213
00214         for ci, cam in enumerate(cameraIds):
00215             if cam == refCamId:
00216                 continue
00217             if cam not in state.extrinsics:
00218                 continue
00219
00220             optim_cams.append(ci)
00221             cam_param_index[ci] = len(cam_param_index)
00222
00223         num_optim_cams = len(optim_cams)
00224         if num_optim_cams == 0:
00225             print("[BA] No extrinsic data to optimize, skipping BA.")
00226             return state
00227
00228         # Collect measurements (all cameras, all valid poses)
00229         # Each measurement: (cam_idx, pose, 2D points)
00230         measurements = []
00231
00232         for ci, cam in enumerate(cameraIds):
00233             for pose in valid_poses:
00234                 img_path = imageSet.getImagePath(pose, cam)
00235                 ok, corners, size = self.detectCorners(img_path)
00236                 if not ok:
00237                     continue
00238                 pts2d = corners.reshape(-1, 2).astype(np.float64)
00239                 measurements.append((ci, pose, pts2d))
00240
00241         if len(measurements) == 0:
00242             print("[BA] No overlapping detections found, skipping BA.")
00243             return state
00244
00245         # Initialize parameter vector
00246         # Layout: [cams (without ref): (r_cam(3), t_cam(3))... , poses: (r_p(3), t_p(3))...]
00247
00248         param_cam = []
00249         for ci in optim_cams:
00250             cam = cameraIds[ci]
00251             extr = state.extrinsics[cam]
00252             R = extr["rotationMatrix"]
00253             T = extr["translationVector"].reshape(3)
00254             rvec_cam, _ = cv2.Rodrigues(R)
00255             param_cam.append(rvec_cam.reshape(3))
00256             param_cam.append(T)
00257         param_cam = np.concatenate(param_cam).astype(np.float64)
00258
00259         param_pose = []
00260         for pose in valid_poses:
00261             param_pose.append(pose_rvecs[pose])
```

```
00262             param_pose.append(pose_tvecs[pose])
00263         param_pose = np.concatenate(param_pose).astype(np.float64)
00264
00265         x0 = np.concatenate([param_cam, param_pose])
00266
00267         # Helper functions for parameter decoding
00268         def decode_params(params):
00269             # Cameras
00270             cam_rvecs = {}
00271             cam_tvecs = {}
00272             idx = 0
00273             for ci in optim_cams:
00274                 r = params[idx:idx+3]; idx += 3
00275                 t = params[idx:idx+3]; idx += 3
00276                 cam_rvecs[ci] = r
00277                 cam_tvecs[ci] = t
00278
00279             # Poses
00280             pose_r = {}
00281             pose_t = {}
00282             for pose in valid_poses:
00283                 r = params[idx:idx+3]; idx += 3
00284                 t = params[idx:idx+3]; idx += 3
00285                 pose_r[pose] = r
00286                 pose_t[pose] = t
00287
00288             return cam_rvecs, cam_tvecs, pose_r, pose_t
00289
00290         # Residuals function
00291         def residuals(params):
00292             cam_rvecs, cam_tvecs, pose_r, pose_t = decode_params(params)
00293
00294             # Precompute rotation matrices
00295             R_cam = {}
00296             for ci, r in cam_rvecs.items():
00297                 R_cam[ci], _ = cv2.Rodrigues(r.astype(np.float64))
00298
00299             R_pose = {}
00300             for pose, r in pose_r.items():
00301                 R_pose[pose], _ = cv2.Rodrigues(r.astype(np.float64))
00302
00303             residual_list = []
00304
00305             for ci, pose, pts2d in measurements:
00306                 cam_id = cameraIds[ci]
00307                 K = state.intrinsics[cam_id]["cameraMatrix"]
00308                 dist = state.intrinsics[cam_id]["distortionCoeffs"]
00309
00310                 # Board pose in reference frame
00311                 r_p = pose_r[pose]
00312                 t_p = pose_t[pose]
00313                 R_p = R_pose[pose]
00314
00315                 if ci == ref_idx:
00316                     # Reference camera: use board pose directly
00317                     r_total = r_p
00318                     t_total = t_p
00319                 else:
00320                     if ci not in cam_rvecs:
00321                         # For non-optimized cameras, use static extrinsics
00322                         extr = state.extrinsics[cam_id]
00323                         R_c = extr["rotationMatrix"]
00324                         t_c = extr["translationVector"].reshape(3)
00325                     else:
00326                         R_c = R_cam[ci]
00327                         t_c = cam_tvecs[ci]
00328
00329                     # Composition: X_cam = R_c * (R_p * X + t_p) + t_c
00330                     R_total = R_c @ R_p
00331                     t_total = R_c @ t_p + t_c
00332                     r_total, _ = cv2.Rodrigues(R_total.astype(np.float64))
00333                     r_total = r_total.reshape(3)
00334
00335                 # Projection
00336                 proj, _ = cv2.projectPoints(
00337                     objp,
00338                     r_total.astype(np.float64),
00339                     t_total.astype(np.float64),
00340                     K,
00341                     dist
00342                 )
00343                 proj = proj.reshape(-1, 2)
00344
00345                 res = (proj - pts2d).reshape(-1)
00346                 residual_list.append(res)
00347
00348             if not residual_list:
```

```
00349                 return np.zeros(0, dtype=np.float64)
00350
00351             return np.concatenate(residual_list)
00352
00353         # Optimization (robust, soft_l1)
00354         result = least_squares(
00355             residuals,
00356             x0,
00357             verbose=2,
00358             method="trf",
00359             loss="soft_l1",
00360             f_scale=1.0
00361         )
00362
00363         print(
00364             f"[BA] Done. Final cost: {result.cost:.4f}, "
00365             f"RMS per coord: {np.sqrt(2*result.cost/len(result.fun)):.4f} px"
00366         )
00367
00368         # Write results back to state.extrinsics
00369         cam_rvecs_opt, cam_tvecs_opt, pose_r_opt, pose_t_opt = decode_params(result.x)
00370
00371         # Reference camera remains identity
00372         state.setExtrinsics(
00373             refCamId,
00374             np.eye(3, dtype=np.float64),
00375             np.zeros((3, 1), dtype=np.float64),
00376             0.0
00377         )
00378
00379         # Store optimized cameras
00380         for ci in optim_cams:
00381             cam_id = cameraIds[ci]
00382             r = cam_rvecs_opt[ci]
00383             t = cam_tvecs_opt[ci]
00384             R, _ = cv2.Rodrigues(r.astype(np.float64))
00385             T = t.reshape(3, 1).astype(np.float64)
00386
00387             # Approximate RMS for this camera from global residuals
00388             # (simple average across all measurements)
00389             cam_residuals = []
00390             for m_ci, pose, pts2d in measurements:
00391                 if m_ci != ci:
00392                     continue
00393                 # Reproject with final parameters
00394                 R_p, _ = cv2.Rodrigues(pose_r_opt[pose].astype(np.float64))
00395                 t_p = pose_t_opt[pose].astype(np.float64)
00396
00397                 R_total = R @ R_p
00398                 t_total = R @ t_p + t.flatten()
00399                 r_total, _ = cv2.Rodrigues(R_total)
00400                 K = state.intrinsics[cam_id]["cameraMatrix"]
00401                 dist = state.intrinsics[cam_id]["distortionCoeffs"]
00402
00403                 proj, _ = cv2.projectPoints(
00404                     objp, r_total, t_total, K, dist
00405                 )
00406                 proj = proj.reshape(-1, 2)
00407                 res = (proj - pts2d).reshape(-1)
00408                 cam_residuals.append(res)
00409
00410             if cam_residuals:
00411                 cam_residuals = np.concatenate(cam_residuals)
00412                 stereo_rms = float(np.sqrt(np.mean(cam_residuals**2)))
00413             else:
00414                 stereo_rms = state.extrinsics[cam_id].get("stereoRms", 0.0)
00415
00416             state.setExtrinsics(cam_id, R, T, stereo_rms)
00417
00418         return state
00419
```

5.4.3.2 calibrateExtrinsics()

```
InitialCalibration.InitialCalibration.calibrateExtrinsics (
            self,
            imageSet,
            state,
            refCamId)
```

@brief Compute pairwise stereo calibration between reference and each other camera.

@param imageSet ImageSet object containing calibration images
@param state CalibrationState with intrinsics already computed
@param refCamId Camera ID to use as reference coordinate system
@return Updated CalibrationState with extrinsics for all cameras

Definition at line 92 of file InitialCalibration.py.

```
00092     def calibrateExtrinsics(self, imageSet, state, refCamId):
00093         """
00094         @brief Compute pairwise stereo calibration between reference and each other camera.
00095 
00096         @param imageSet ImageSet object containing calibration images
00097         @param state CalibrationState with intrinsics already computed
00098         @param refCamId Camera ID to use as reference coordinate system
00099         @return Updated CalibrationState with extrinsics for all cameras
00100         """
00101         objectPoints = self.createObjectPoints()
00102         K_ref = state.intrinsics[refCamId]["cameraMatrix"]
00103         dist_ref = state.intrinsics[refCamId]["distortionCoeffs"]
00104 
00105         for cam in imageSet.cameraIds:
00106             if cam == refCamId:
00107                 state.setExtrinsics(cam, np.eye(3), np.zeros((3,1)), 0.0)
00108                 continue
00109 
00110             if cam not in state.intrinsics:
00111                 continue
00112 
00113             K_cam = state.intrinsics[cam]["cameraMatrix"]
00114             dist_cam = state.intrinsics[cam]["distortionCoeffs"]
00115 
00116             objList = []
00117             imgRef = []
00118             imgCam = []
00119             imageSize = None
00120 
00121             for pose in range(imageSet.numPoses):
00122                 pRef = imageSet.getImagePath(pose, refCamId)
00123                 pCam = imageSet.getImagePath(pose, cam)
00124 
00125                 okR, cornersR, sizeR = self.detectCorners(pRef)
00126                 okC, cornersC, sizeC = self.detectCorners(pCam)
00127 
00128                 if okR and okC:
00129                     objList.append(objectPoints)
00130                     imgRef.append(cornersR)
00131                     imgCam.append(cornersC)
00132                     imageSize = sizeR
00133 
00134             if len(objList) < 3:
00135                 continue
00136 
00137             flags = cv2.CALIB_FIX_INTRINSIC
00138 
00139             stereo_rms, KR0, d0, KC1, d1, R, T, E, F = cv2.stereoCalibrate(
00140                 objList,
00141                 imgRef,
00142                 imgCam,
00143                 K_ref,
00144                 dist_ref,
00145                 K_cam,
00146                 dist_cam,
00147                 imageSize,
00148                 criteria=self.criteria,
00149                 flags=flags
00150             )
00151 
00152             state.setExtrinsics(cam, R, T, stereo_rms)
00153 
00154         return state
00155 
```

5.4.3.3 calibrateIntrinsics()

```
InitialCalibration.InitialCalibration.calibrateIntrinsics (
              self,
              imageSet)
```

```
@brief Calibrate intrinsic parameters independently for each camera.

@param imageSet ImageSet object containing calibration images
@return CalibrationState with populated intrinsics for all cameras
```

Definition at line 55 of file InitialCalibration.py.

```
00055     def calibrateIntrinsics(self, imageSet):
00056         """
00057         @brief Calibrate intrinsic parameters independently for each camera.
00058 
00059         @param imageSet ImageSet object containing calibration images
00060         @return CalibrationState with populated intrinsics for all cameras
00061         """
00062         state = CalibrationState()
00063         objp = self.createObjectPoints()
00064 
00065         objpoints = {cam: [] for cam in imageSet.cameraIds}
00066         imgpoints = {cam: [] for cam in imageSet.cameraIds}
00067         imgsize = {}
00068 
00069         # Collect corner points from all poses for each camera
00070         for pose in range(imageSet.numPoses):
00071             for cam in imageSet.cameraIds:
00072                 path = imageSet.getImagePath(pose, cam)
00073                 ok, corners, size = self.detectCorners(path)
00074                 if ok:
00075                     objpoints[cam].append(objp)
00076                     imgpoints[cam].append(corners)
00077                     imgsize[cam] = size
00078 
00079         # Calibrate each camera separately
00080         for cam in imageSet.cameraIds:
00081             if len(objpoints[cam]) < 5:
00082                 print(f"[WARN] Not enough samples for camera {cam}")
00083                 continue
00084 
00085             err, K, dist, rvecs, tvecs = cv2.calibrateCamera(
00086                 objpoints[cam], imgpoints[cam], imgsize[cam], None, None)
00087 
00088             state.setIntrinsics(cam, K, dist, err)
00089 
00090         return state
00091 
```

5.4.3.4 createObjectPoints()

```
InitialCalibration.InitialCalibration.createObjectPoints (
              self)

@brief Generate 3D coordinates of chessboard corners.

@return Numpy array of shape (N, 3) with 3D corner coordinates
```

Definition at line 23 of file InitialCalibration.py.

```
00023     def createObjectPoints(self):
00024         """
00025         @brief Generate 3D coordinates of chessboard corners.
00026 
00027         @return Numpy array of shape (N, 3) with 3D corner coordinates
00028         """
00029         cols, rows = self.chessboardSize
00030         objp = np.zeros((cols * rows, 3), np.float32)
00031         objp[:, :2] = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2)
00032         objp *= self.squareSize
00033         return objp
00034 
```

5.4.3.5 detectCorners()

```
InitialCalibration.InitialCalibration.detectCorners (
              self,
              imagePath)
```

```
@brief Detect and refine chessboard corner positions in image.

@param imagePath Path to the image file
@return Tuple (success, corners, imageSize) where success is bool,
corners are refined 2D points, and imageSize is (width, height)
```

Definition at line 35 of file InitialCalibration.py.

```
00035     def detectCorners(self, imagePath):
00036         """
00037         @brief Detect and refine chessboard corner positions in image.
00038
00039         @param imagePath Path to the image file
00040         @return Tuple (success, corners, imageSize) where success is bool,
00041         corners are refined 2D points, and imageSize is (width, height)
00042         """
00043         img = cv2.imread(imagePath)
00044         if img is None:
00045             return False, None, None
00046         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
00047
00048         found, corners = cv2.findChessboardCorners(gray, self.chessboardSize)
00049         if not found:
00050             return False, None, None
00051
00052         refined = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), self.criteria)
00053         return True, refined, gray.shape[::-1]
00054
```

5.4.3.6 run()

```
InitialCalibration.InitialCalibration.run (
              self,
              imageSet,
              bundleAdjust = True)
```

```
@brief Main entry point for calibration pipeline.

@param imageSet ImageSet object containing calibration images
@param bundleAdjust Whether to apply bundle adjustment (default: True)
@return CalibrationState with complete intrinsic and extrinsic calibration
```

Definition at line 420 of file InitialCalibration.py.

```
00420     def run(self, imageSet, bundleAdjust=True):
00421         """
00422         @brief Main entry point for calibration pipeline.
00423
00424         @param imageSet ImageSet object containing calibration images
00425         @param bundleAdjust Whether to apply bundle adjustment (default: True)
00426         @return CalibrationState with complete intrinsic and extrinsic calibration
00427         """
00428         print("Running intrinsic calibration...")
00429         state = self.calibrateIntrinsics(imageSet)
00430
00431         print("\nRunning pairwise extrinsic calibration...")
00432         state = self.calibrateExtrinsics(imageSet, state, refCamId=imageSet.cameraIds[0])
00433
00434         if bundleAdjust:
00435             print("\nRunning global bundle adjustment on extrinsics...")
00436             state = self.bundleAdjustExtrinsics(imageSet, state, refCamId=imageSet.cameraIds[0])
00437
00438         return state
```

5.4.4 Member Data Documentation

5.4.4.1 chessboardSize

```
InitialCalibration.InitialCalibration.chessboardSize = chessboardSize
```

Definition at line 19 of file InitialCalibration.py.

5.4.4.2 criteria

```
tuple InitialCalibration.InitialCalibration.criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_↩
CRITERIA_MAX_ITER, 30, 0.001)
```

Definition at line 21 of file InitialCalibration.py.

5.4.4.3 squareSize

```
InitialCalibration.InitialCalibration.squareSize = squareSize
```

Definition at line 20 of file InitialCalibration.py.

The documentation for this class was generated from the following file:

- calibrationLFC/InitialCalibration.py

5.5 ResultLogger.ResultLogger Class Reference

Public Member Functions

- __init__ (self, baseDir="/data/baseline")
- to_serializable (self, obj)
- float read_baseline_health (self, str baseline_path)
- None update_baseline (self, dict candidate_obj, float candidate_score, str method)
- logInitialCalibration (self, calibrationState, meta=None)
- logRecalibration (self, str combined_json_path, meta=None)

Public Attributes

- baseDir = baseDir
- logger = logging.getLogger("calibration")

5.5.1 Detailed Description

```
@brief Manages calibration result logging and baseline versioning.
```

Definition at line 8 of file ResultLogger.py.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 __init__()

```
ResultLogger.ResultLogger.__init__ (
              self,
              baseDir = "/data/baseline")


@brief Constructor for ResultLogger.

@param baseDir Directory for storing calibration results and baseline (default: "/data/baseline")
```

Definition at line 13 of file ResultLogger.py.

```
00013     def __init__(self, baseDir="/data/baseline"):
00014         """
00015         @brief Constructor for ResultLogger.
00016 
00017         @param baseDir Directory for storing calibration results and baseline (default:
      "/data/baseline")
00018         """
00019         self.baseDir = baseDir
00020         os.makedirs(baseDir, exist_ok=True)
00021 
00022         self.logger = logging.getLogger("calibration")
00023         self.logger.setLevel(logging.INFO)
00024 
00025         if not self.logger.handlers:
00026             log_path = os.path.join(baseDir, "calibration.log")
00027             file_handler = logging.FileHandler(log_path, encoding="utf-8")
00028             file_handler.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00029             self.logger.addHandler(file_handler)
00030 
```

5.5.3 Member Function Documentation

5.5.3.1 logInitialCalibration()

```
ResultLogger.ResultLogger.logInitialCalibration (
              self,
              calibrationState,
              meta = None)


@brief Store initial calibration result with health score.

@param calibrationState CalibrationState object with intrinsic and extrinsic parameters
@param meta Optional dictionary with metadata about the calibration run
@return Health score as float
```

Definition at line 125 of file ResultLogger.py.

```
00125     def logInitialCalibration(self, calibrationState, meta=None):
00126         """
00127         @brief Store initial calibration result with health score.
00128 
00129         @param calibrationState CalibrationState object with intrinsic and extrinsic parameters
00130         @param meta Optional dictionary with metadata about the calibration run
00131         @return Health score as float
00132         """
00133         if meta is None:
00134             meta = {}
00135 
00136         # Extract calibration state
00137         try:
00138             state_dict = calibrationState.__getState__()
00139         except Exception as e:
00140             self.logger.error(f"Could not extract state dict from CalibrationState: {e}")
```

```
00141             return 0.0
00142
00143         safe_state = self.to_serializable(state_dict)
00144         safe_meta = self.to_serializable(meta)
00145
00146         # Compute health score from metrics
00147         try:
00148             from HealthMonitor import compute_initial_health_from_state
00149             report = compute_initial_health_from_state(state_dict)
00150             score = float(report.get("health", 0.0))
00151         except Exception as e:
00152             self.logger.error(f"Initial HealthScore computation failed: {e}", exc_info=True)
00153             score = 0.0
00154
00155         # Create timestamped filename
00156         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
00157         imagesetnumber = meta.get("imageDir", "unknown")
00158         if isinstance(imagesetnumber, str) and "imageset" in imagesetnumber:
00159             imagesetnumber = imagesetnumber.split("imageset")[-1]
00160
00161         json_name = f"calibration_initial_imageset{imagesetnumber}_{timestamp}.json"
00162         json_path = os.path.join(self.baseDir, json_name)
00163
00164         # Prepare output structure
00165         out = {
00166             "meta": safe_meta,
00167             "state": safe_state
00168         }
00169
00170         # Attach health score to metadata
00171         out.setdefault("meta", {})
00172         out["meta"]["health"] = score
00173         out["meta"]["method"] = "initial"
00174         out["meta"]["runType"] = out["meta"].get("runType", "initial")
00175
00176         # Save to file
00177         try:
00178             with open(json_path, "w", encoding="utf-8") as f:
00179                 json.dump(out, f, indent=2)
00180         except Exception as e:
00181             self.logger.error(f"Failed to write JSON result file '{json_path}': {e}")
00182             return 0.0
00183
00184         intr = state_dict.get("intrinsics", {})
00185         extr = state_dict.get("extrinsics", {})
00186         self.logger.info(
00187             f"Stored initial calibration to '{json_name}' "
00188             f"(intrinsics for {len(intr)} cams, extrinsics for {len(extr)} cams, bundleAdjustf:
      {meta.get('bundleAdjust', True)})."
00189         )
00190         self.logger.info(f"HealthScore = {score:.2f} (method=initial)")
00191
00192         # Update baseline (creates baseline on first run)
00193         self.update_baseline(out, score, method="initial")
00194
00195         return score
00196
```

5.5.3.2 logRecalibration()

```
ResultLogger.ResultLogger.logRecalibration (
                self,
               str combined_json_path,
                meta = None)
```

```
@brief Store LiFCal recalibration result with health score.

@param combined_json_path Path to the combined.json file from LiFCal
@param meta Optional dictionary with metadata about the recalibration run
@return Health score as float
```

Definition at line 197 of file ResultLogger.py.

```
00197     def logRecalibration(self, combined_json_path: str, meta=None):
00198         """
00199         @brief Store LiFCal recalibration result with health score.
00200
```

```
00201         @param combined_json_path Path to the combined.json file from LiFCal
00202         @param meta Optional dictionary with metadata about the recalibration run
00203         @return Health score as float
00204         """
00205         if meta is None:
00206             meta = {}
00207 
00208         # Import locally to avoid errors when not needed
00209         from HealthMonitor import compute_lifcal_health_from_combined_path
00210 
00211         # Read combined.json
00212         try:
00213             with open(combined_json_path, "r", encoding="utf-8") as f:
00214                 combined = json.load(f)
00215         except Exception as e:
00216             self.logger.error(f"Could not read combined.json '{combined_json_path}': {e}")
00217             return 0.0
00218 
00219         # Compute health score
00220         try:
00221             report = compute_lifcal_health_from_combined_path(combined_json_path)
00222             score = float(report.get("health", 0.0))
00223         except Exception as e:
00224             self.logger.error(f"LiFCal HealthScore computation failed: {e}", exc_info=True)
00225             score = 0.0
00226 
00227         # Create timestamped copy
00228         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
00229         json_name = f"calibration_lifcal_{timestamp}.json"
00230         out_path = os.path.join(self.baseDir, json_name)
00231 
00232         combined.setdefault("meta", {})
00233         combined["meta"].update(self.to_serializable(meta))
00234         combined["meta"]["health"] = score
00235         combined["meta"]["method"] = "lifcal"
00236         combined["meta"]["runType"] = combined["meta"].get("runType", "recalib")
00237 
00238         # Save to file
00239         try:
00240             with open(out_path, "w", encoding="utf-8") as f:
00241                 json.dump(combined, f, indent=2)
00242         except Exception as e:
00243             self.logger.error(f"Failed to write LiFCal result '{out_path}': {e}")
00244             return 0.0
00245 
00246         num_cams = len(combined.get("state", {}).get("parameters", {}))
00247         self.logger.info(f"Stored lifcal recalibration to '{json_name}' (cams={num_cams}).")
00248         self.logger.info(f"HealthScore = {score:.2f} (method=lifcal)")
00249 
00250         # Update baseline only if better
00251         self.update_baseline(combined, score, method="lifcal")
00252 
00253         return score
```

5.5.3.3 read_baseline_health()

```
float ResultLogger.ResultLogger.read_baseline_health (
              self,
             str baseline_path)
```

```
@brief Extract health score from baseline.json.

@param baseline_path Path to the baseline.json file
@return Health score as float, or -1.0 if not found or cannot be determined
```

Definition at line 50 of file ResultLogger.py.

```
00050     def read_baseline_health(self, baseline_path: str) -> float:
00051         """
00052         @brief Extract health score from baseline.json.
00053 
00054         @param baseline_path Path to the baseline.json file
00055         @return Health score as float, or -1.0 if not found or cannot be determined
00056         """
00057         if not os.path.isfile(baseline_path):
00058             return -1.0
00059 
00060         try:
```

```
00061             with open(baseline_path, "r", encoding="utf-8") as f:
00062                 base = json.load(f)
00063         except Exception:
00064             return -1.0
00065 
00066         # Try direct health value from metadata
00067         try:
00068             h = float(base.get("meta", {}).get("health", -1.0))
00069             if np.isfinite(h) and h >= 0:
00070                 return h
00071         except Exception:
00072             pass
00073 
00074         # Fallback: compute based on calibration type
00075         try:
00076             from HealthMonitor import (
00077                 compute_initial_health_from_state,
00078                 compute_lifcal_health_from_combined_dict,
00079             )
00080         except Exception:
00081             return -1.0
00082 
00083         run_type = (base.get("meta", {}) or {}).get("runType", "")
00084         state = base.get("state", {})
00085 
00086         try:
00087             if run_type == "initial":
00088                 rep = compute_initial_health_from_state(state)
00089                 return float(rep.get("health", -1.0))
00090             else:
00091                 # Treat as lifcal/combined
00092                 rep = compute_lifcal_health_from_combined_dict(base)
00093                 return float(rep.get("health", -1.0))
00094         except Exception:
00095             return -1.0
00096 
```

5.5.3.4 to_serializable()

```
ResultLogger.ResultLogger.to_serializable (
                self,
                obj)
```

```
@brief Convert numpy arrays and types to JSON-serializable format.

@param obj Object to convert (numpy array, dict, list, or primitive type)
@return JSON-serializable version of the object
```

Definition at line 31 of file ResultLogger.py.

```
00031     def to_serializable(self, obj):
00032         """
00033         @brief Convert numpy arrays and types to JSON-serializable format.
00034 
00035         @param obj Object to convert (numpy array, dict, list, or primitive type)
00036         @return JSON-serializable version of the object
00037         """
00038         if isinstance(obj, np.ndarray):
00039             return obj.tolist()
00040         if isinstance(obj, (np.floating, np.float32, np.float64)):
00041             return float(obj)
00042         if isinstance(obj, (np.integer, np.int32, np.int64)):
00043             return int(obj)
00044         if isinstance(obj, dict):
00045             return {k: self.to_serializable(v) for k, v in obj.items()}
00046         if isinstance(obj, (list, tuple)):
00047             return [self.to_serializable(v) for v in obj]
00048         return obj
00049 
```

5.5.3.5 update_baseline()

```
None ResultLogger.ResultLogger.update_baseline (
                self,
```

```
            dict candidate_obj,
            float candidate_score,
            str method)


@brief Update baseline.json with new calibration if health improved.

@param candidate_obj Dictionary containing calibration data
@param candidate_score Health score of the candidate calibration
@param method Calibration method identifier ("initial" or "lifcal")
```

Definition at line 97 of file ResultLogger.py.

```
00097     def update_baseline(self, candidate_obj: dict, candidate_score: float, method: str) -> None:
00098         """
00099         @brief Update baseline.json with new calibration if health improved.
00100 
00101         @param candidate_obj Dictionary containing calibration data
00102         @param candidate_score Health score of the candidate calibration
00103         @param method Calibration method identifier ("initial" or "lifcal")
00104         """
00105         baseline_path = os.path.join(self.baseDir, "baseline.json")
00106         old_score = self.read_baseline_health(baseline_path)
00107 
00108         if (not os.path.isfile(baseline_path)) or (candidate_score > old_score):
00109             try:
00110                 with open(baseline_path, "w", encoding="utf-8") as f:
00111                     json.dump(candidate_obj, f, indent=2)
00112                 if old_score < 0:
00113                     self.logger.info(f"Baseline created: baseline.json (new={candidate_score:.2f},
      method={method})")
00114                 else:
00115                     self.logger.info(
00116                         f"Baseline updated: baseline.json (old={old_score:.2f},
      new={candidate_score:.2f}, method={method})"
00117                         )
00118             except Exception as e:
00119                 self.logger.error(f"Failed to update baseline.json: {e}")
00120         else:
00121             self.logger.info(
00122                 f"Baseline kept: baseline.json (old={old_score:.2f} >= new={candidate_score:.2f},
      method={method})"
00123             )
00124 
```

5.5.4 Member Data Documentation

5.5.4.1 baseDir

```
ResultLogger.ResultLogger.baseDir = baseDir
```

Definition at line 19 of file ResultLogger.py.

5.5.4.2 logger

```
ResultLogger.ResultLogger.logger = logging.getLogger("calibration")
```

Definition at line 22 of file ResultLogger.py.

The documentation for this class was generated from the following file:

- calibrationLFC/ResultLogger.py

Chapter 6

File Documentation

6.1 calibrationLFC/CalibrationState.py File Reference

Classes

- class CalibrationState.CalibrationState

Namespaces

- namespace CalibrationState

6.2 CalibrationState.py

Go to the documentation of this file.

```
00001 class CalibrationState:
00002     """
00003     @brief Stores the complete calibration state for a multi-camera system.
00004
00005     @details This class manages intrinsic and extrinsic parameters for all cameras
00006     in the system. It provides methods to set and retrieve calibration data including
00007     camera matrices, distortion coefficients, rotation matrices, and translation vectors.
00008     """
00009
00010     def __init__(self):
00011         """
00012         @brief Constructor for CalibrationState.
00013
00014         @details Initializes empty dictionaries for intrinsic and extrinsic parameters
00015         and sets the timestamp to None.
00016         """
00017         self.intrinsics = {}
00018         self.extrinsics = {}
00019         self.timeStamp = None
00020
00021     def setIntrinsics(self, camId, cameraMatrix, distortionCoeffs, repError):
00022         """
00023         @brief Store intrinsic calibration parameters for a camera.
00024
00025         @param camId Camera identifier
00026         @param cameraMatrix 3x3 camera matrix containing focal lengths and principal point
00027         @param distortionCoeffs Distortion coefficients vector
00028         @param repError Reprojection error of the calibration
00029         """
00030         self.intrinsics[camId] = {
00031             "cameraMatrix": cameraMatrix,
00032             "distortionCoeffs": distortionCoeffs,
00033             "reprojectionError": repError
00034         }
```

```
00035
00036     def setExtrinsics(self, camId, rotationMatrix, translationVector, stereoRMS):
00037         """
00038         @brief Store extrinsic calibration parameters for a camera.
00039
00040         @param camId Camera identifier
00041         @param rotationMatrix 3x3 rotation matrix representing camera orientation
00042         @param translationVector 3x1 translation vector representing camera position
00043         @param stereoRMS Root mean square error of the stereo calibration
00044         """
00045         self.extrinsics[camId] = {
00046             "rotationMatrix": rotationMatrix,
00047             "translationVector": translationVector,
00048             "stereoRms": stereoRMS
00049         }
00050
00051     def setTimeStamp(self, timestamp):
00052         """
00053         @brief Set the calibration timestamp.
00054
00055         @param timestamp Timestamp indicating when the calibration was performed
00056         """
00057         self.timeStamp = timestamp
00058
00059     def __getState__(self):
00060         """
00061         @brief Return the complete calibration state as dictionary.
00062
00063         @return Dictionary containing intrinsics, extrinsics, and timestamp
00064         """
00065         return {
00066             "intrinsics": self.intrinsics,
00067             "extrinsics": self.extrinsics,
00068             "timeStamp": self.timeStamp
00069         }
```

6.3 calibrationLFC/Controller.py File Reference

Classes

- class Controller.Controller

Namespaces

- namespace Controller

6.4 Controller.py

Go to the documentation of this file.

```
00001 from InitialCalibration import InitialCalibration
00002 from ResultLogger import ResultLogger
00003 from HealthMonitor import compute_initial_health_from_state
00004
00005
00006 class Controller:
00007     """
00008     @brief Main controller for managing calibration workflow.
00009
00010     @details Orchestrates the complete calibration process including initial calibration,
00011     health monitoring, and result logging. Manages the calibration state and maintains
00012     a history of health scores.
00013     """
00014
00015     def __init__(self, bundleAdjust):
00016         """
00017         @brief Constructor for Controller.
00018
00019         @param bundleAdjust Boolean flag indicating whether to apply bundle adjustment
00020
00021         @details Initializes the controller with an InitialCalibration instance,
```

```
00022         empty score history, result logger, and bundle adjustment setting.
00023         """
00024         self.initialCalibration = InitialCalibration()
00025         self.scoreHistory = []
00026         self.currentState = None
00027         self.resultLogger = ResultLogger()
00028         self.bundleAdjust = bundleAdjust
00029 
00030     def runInitialCalibration(self, imageSet):
00031         """
00032         @brief Execute initial calibration for all cameras in the image set.
00033 
00034         @param imageSet ImageSet object containing calibration images and camera information
00035         @return CalibrationState object with computed intrinsic and extrinsic parameters
00036 
00037         """
00038         print("Controller: starting initial calibration...")
00039         calibrationState = self.initialCalibration.run(imageSet, self.bundleAdjust)
00040 
00041         # Prepare metadata for logging
00042         meta = {
00043             "runType": "initial",
00044             "numPoses": imageSet.numPoses,
00045             "imageDir": imageSet.baseDir,
00046             "cameraIds": imageSet.cameraIds,
00047             "bundleAdjust": self.bundleAdjust,
00048         }
00049         self.resultLogger.logInitialCalibration(calibrationState, meta=meta)
00050 
00051         return calibrationState
00052 
00053     def selfHealthCheck(self, calibrationState) -> float:
00054         """
00055         @brief Computes a global health score for the calibration quality.
00056 
00057         @param calibrationState CalibrationState object to evaluate
00058         @return Health score as float between 0 and 100, or 0.0 if evaluation fails
00059 
00060         """
00061         # Extract state dictionary
00062         try:
00063             state_dict = calibrationState.__getState__()
00064         except Exception as e:
00065             self.resultLogger.logger.error(
00066                 f"HealthCheck: could not extract state dict: {e}", exc_info=True
00067             )
00068             return 0.0
00069 
00070         # Compute health score from calibration metrics
00071         try:
00072             report = compute_initial_health_from_state(state_dict)
00073             score = float(report["health"])
00074             self.resultLogger.logger.info(
00075                 f"HealthScore = {score:.2f} (method={report.get('method')})"
00076             )
00077             return score
00078         except Exception as e:
00079             self.resultLogger.logger.error(f"HealthCheck failed: {e}", exc_info=True)
00080             return 0.0
```

6.5 calibrationLFC/HealthMonitor.py File Reference

Namespaces

- namespace HealthMonitor

Functions

- float HealthMonitor.clamp (float x, float lo=0.0, float hi=1.0)
- HealthMonitor.safe_float (v, default=None)
- float HealthMonitor.score_from_errors (float E_a, float E_b, float w_a=0.6, float w_b=0.4)
- Dict[str, Any] HealthMonitor.compute_initial_health_from_state (Dict[str, Any] state_dict, float r_↩ best=R_BEST, float r_worst=R_WORST, float s_best=S_BEST, float s_worst=S_WORST)
- Dict[str, Optional[float]] HealthMonitor.parse_lifcal_protocol (str protocol_path)

- Dict[str, Any] HealthMonitor.compute_lifcal_health_from_xy (float x_std, float y_std, float x_mae, float y_mae, float r_best=R_BEST, float r_worst=R_WORST, float s_best=S_BEST, float s_worst=S_WORST)
- Dict[str, Any] HealthMonitor.compute_lifcal_health_from_combined_dict (Dict[str, Any] combined)
- Dict[str, Any] HealthMonitor.compute_lifcal_health_from_combined_path (str combined_json_path)

Variables

- float HealthMonitor.R_BEST = 0.03
- float HealthMonitor.R_WORST = 1.0
- float HealthMonitor.S_BEST = 0.0
- float HealthMonitor.S_WORST = 50.0

6.6 HealthMonitor.py

Go to the documentation of this file.

```
00001 """
00002 HealthScore Formulas (0..100)
00003
00004 (A) INITIAL Calibration (OpenCV/Controller JSON)
00005     HealthScore_initial = 100 * [ 1 - (0.6 * E_reproj^2 + 0.4 * E_rms^2) ]
00006
00007     E_reproj = clamp( (reproj_mean - R_BEST) / (R_WORST - R_BEST), 0, 1 )
00008     E_rms    = clamp( (rms_mean    - S_BEST) / (S_WORST - S_BEST), 0, 1 )
00009
00010     Uses:
00011       - intrinsics[*].reprojectionError  (per camera)
00012       - extrinsics[*].stereoRms          (per camera)
00013
00014 (B) LiFCal Calibration (protocol-based)
00015     HealthScore_lifcal = 100 * [ 1 - (0.6 * E_std^2 + 0.4 * E_mae^2) ]
00016
00017     E_std = clamp( ( sqrt(x_std^2 + y_std^2) - S_BEST ) / (S_WORST - S_BEST), 0, 1 )
00018     E_mae = clamp( ( sqrt(x_mae^2 + y_mae^2) - R_BEST ) / (R_WORST - R_BEST), 0, 1 )
00019
00020     Uses:
00021       - std.Dev. x/y  and mae x/y from LiFCal calibrationProtocol.txt
00022
00023 Shared thresholds (for BOTH methods):
00024   R_BEST = 0.03, R_WORST = 1.0, S_BEST = 0.0, S_WORST = 50.0
00025 """
00026
00027 import re
00028 import json
00029 import math
00030 from typing import Dict, Any, Optional
00031
00032 """
00033 @brief HealthMonitor module for computing calibration health scores.
00034
00035 """
00036
00037 # Shared thresholds
00038 R_BEST = 0.03
00039 R_WORST = 1.0
00040 S_BEST = 0.0
00041 S_WORST = 50.0
00042
00043
00044 def clamp(x: float, lo: float = 0.0, hi: float = 1.0) -> float:
00045     """
00046     @brief Clamp value between lower and upper bounds.
00047
00048     @param x Value to clamp
00049     @param lo Lower bound (default: 0.0)
00050     @param hi Upper bound (default: 1.0)
00051     @return Clamped value
00052     """
00053     if x < lo:
00054         return lo
00055     if x > hi:
00056         return hi
00057     return x
00058
00059
```

```
00060 def safe_float(v, default=None):
00061     """
00062     @brief Convert value to float safely, return default on error.
00063
00064     @param v Value to convert
00065     @param default Default value to return on conversion error
00066     @return Converted float value or default
00067     """
00068     try:
00069         if v is None:
00070             return default
00071         return float(v)
00072     except Exception:
00073         return default
00074
00075
00076 def score_from_errors(E_a: float, E_b: float, w_a: float = 0.6, w_b: float = 0.4) -> float:
00077     """
00078     @brief Compute health score from two normalized error metrics.
00079
00080     @param E_a First normalized error metric (0..1)
00081     @param E_b Second normalized error metric (0..1)
00082     @param w_a Weight for first error metric (default: 0.6)
00083     @param w_b Weight for second error metric (default: 0.4)
00084     @return Health score between 0 and 100
00085     """
00086     val = 100.0 * (1.0 - (w_a * (E_a ** 2) + w_b * (E_b ** 2)))
00087     return clamp(val / 100.0, 0.0, 1.0) * 100.0
00088
00089
00090 def compute_initial_health_from_state(
00091     state_dict: Dict[str, Any],
00092     r_best: float = R_BEST,
00093     r_worst: float = R_WORST,
00094     s_best: float = S_BEST,
00095     s_worst: float = S_WORST,
00096 ) -> Dict[str, Any]:
00097     """
00098     @brief Compute health score from initial calibration state.
00099
00100     @param state_dict Dictionary containing calibration state with intrinsics and extrinsics
00101     @param r_best Best reprojection error threshold (default: R_BEST)
00102     @param r_worst Worst reprojection error threshold (default: R_WORST)
00103     @param s_best Best stereo RMS threshold (default: S_BEST)
00104     @param s_worst Worst stereo RMS threshold (default: S_WORST)
00105     @return Dictionary with health score, method name, and detailed metrics
00106     """
00107     st = state_dict.get("state", state_dict)
00108     intr = st.get("intrinsics", {}) or {}
00109     extr = st.get("extrinsics", {}) or {}
00110
00111     reproj_vals = []
00112     rms_vals = []
00113
00114     for cam, intr_data in intr.items():
00115         reproj = safe_float(intr_data.get("reprojectionError"), None)
00116         if reproj is not None:
00117             reproj_vals.append(reproj)
00118
00119         ex = extr.get(cam, {})
00120         rms = safe_float(ex.get("stereoRms"), None)
00121         if rms is not None:
00122             rms_vals.append(rms)
00123
00124     if not reproj_vals and not rms_vals:
00125         return {
00126             "health": 0.0,
00127             "method": "initial",
00128             "details": {"reason": "no reprojectionError and no stereoRms found"},
00129         }
00130
00131     reproj_mean = (sum(reproj_vals) / len(reproj_vals)) if reproj_vals else r_worst
00132     rms_mean = (sum(rms_vals) / len(rms_vals)) if rms_vals else s_worst
00133
00134     E_reproj = clamp((reproj_mean - r_best) / (r_worst - r_best), 0.0, 1.0)
00135     E_rms = clamp((rms_mean - s_best) / (s_worst - s_best), 0.0, 1.0)
00136
00137     score = score_from_errors(E_reproj, E_rms)
00138
00139     return {
00140         "health": float(score),
00141         "method": "initial",
00142         "details": {
00143             "reproj_mean": float(reproj_mean),
00144             "rms_mean": float(rms_mean),
00145             "E_reproj": float(E_reproj),
00146             "E_rms": float(E_rms),
```

```
00147             "thresholds": {
00148                 "R_BEST": float(r_best),
00149                 "R_WORST": float(r_worst),
00150                 "S_BEST": float(s_best),
00151                 "S_WORST": float(s_worst),
00152             },
00153         },
00154     }
00155
00156
00157 def parse_lifcal_protocol(protocol_path: str) -> Dict[str, Optional[float]]:
00158     """
00159     @brief Parse LiFCal calibration protocol text file for error metrics.
00160
00161     @param protocol_path Path to the calibration protocol text file
00162     @return Dictionary with x_std, y_std, x_mae, y_mae values or None if not found
00163     """
00164     txt = open(protocol_path, "r", encoding="utf-8", errors="ignore").read()
00165
00166     def find_float(pattern: str) -> Optional[float]:
00167         m = re.search(pattern, txt, re.MULTILINE)
00168         if not m:
00169             return None
00170         return safe_float(m.group(1), None)
00171
00172     x_std = find_float(r"std\.\s*Dev\.\s*x:\s*([0-9.+-eE]+)")
00173     y_std = find_float(r"std\.\s*Dev\.\s*y:\s*([0-9.+-eE]+)")
00174     x_mae = find_float(r"mae\s*x:\s*([0-9.+-eE]+)")
00175     y_mae = find_float(r"mae\s*y:\s*([0-9.+-eE]+)")
00176
00177     return {"x_std": x_std, "y_std": y_std, "x_mae": x_mae, "y_mae": y_mae}
00178
00179
00180 def compute_lifcal_health_from_xy(
00181     x_std: float,
00182     y_std: float,
00183     x_mae: float,
00184     y_mae: float,
00185     r_best: float = R_BEST,
00186     r_worst: float = R_WORST,
00187     s_best: float = S_BEST,
00188     s_worst: float = S_WORST,
00189 ) -> Dict[str, Any]:
00190     """
00191     @brief Compute LiFCal health score from x/y standard deviation and MAE values.
00192
00193     @param x_std Standard deviation in x direction
00194     @param y_std Standard deviation in y direction
00195     @param x_mae Mean absolute error in x direction
00196     @param y_mae Mean absolute error in y direction
00197     @param r_best Best reprojection error threshold (default: R_BEST)
00198     @param r_worst Worst reprojection error threshold (default: R_WORST)
00199     @param s_best Best stereo RMS threshold (default: S_BEST)
00200     @param s_worst Worst stereo RMS threshold (default: S_WORST)
00201     @return Dictionary with health score, method name, and detailed metrics
00202     """
00203     std_norm = (x_std * x_std + y_std * y_std) ** 0.5
00204     mae_norm = (x_mae * x_mae + y_mae * y_mae) ** 0.5
00205
00206     E_std = clamp((std_norm - s_best) / (s_worst - s_best), 0.0, 1.0)
00207     E_mae = clamp((mae_norm - r_best) / (r_worst - r_best), 0.0, 1.0)
00208
00209     score = score_from_errors(E_std, E_mae)
00210
00211     return {
00212         "health": float(score),
00213         "method": "lifcal",
00214         "details": {
00215             "std_norm": float(std_norm),
00216             "mae_norm": float(mae_norm),
00217             "E_std": float(E_std),
00218             "E_mae": float(E_mae),
00219             "thresholds": {
00220                 "R_BEST": float(r_best),
00221                 "R_WORST": float(r_worst),
00222                 "S_BEST": float(s_best),
00223                 "S_WORST": float(s_worst),
00224             },
00225         },
00226     }
00227
00228 def compute_lifcal_health_from_combined_dict(combined: Dict[str, Any]) -> Dict[str, Any]:
00229     """
00230     @brief Compute LiFCal health from combined.json structure.
00231
00232     @param combined Dictionary from combined.json with parameters for all cameras
00233     @return Dictionary with global health score, method name, and per-camera metrics
```

```
00234     """
00235     params_all = combined.get("state", {}).get("parameters", {})
00236     if not isinstance(params_all, dict) or len(params_all) == 0:
00237         return {"health": 0.0, "method": "lifcal", "perCam": {}}
00238 
00239     per_cam = {}
00240     scores = []
00241 
00242     for cam, p in params_all.items():
00243         rep = (p or {}).get("reprojection", {}) or {}
00244 
00245         x_std = rep.get("x_std", None)
00246         y_std = rep.get("y_std", None)
00247         x_mae = rep.get("x_mae", None)
00248         y_mae = rep.get("y_mae", None)
00249 
00250         # Skip camera if any field is missing
00251         if any(v is None for v in [x_std, y_std, x_mae, y_mae]):
00252             per_cam[cam] = {"health": 0.0, "reason": "missing reprojection fields"}
00253             continue
00254 
00255         std_mag = math.sqrt(float(x_std) ** 2 + float(y_std) ** 2)
00256         mae_mag = math.sqrt(float(x_mae) ** 2 + float(y_mae) ** 2)
00257 
00258         E_std = clamp((std_mag - S_BEST) / (S_WORST - S_BEST), 0.0, 1.0)
00259         E_mae = clamp((mae_mag - R_BEST) / (R_WORST - R_BEST), 0.0, 1.0)
00260 
00261         health = 100.0 * (1.0 - (0.6 * (E_std ** 2) + 0.4 * (E_mae ** 2)))
00262         health = clamp(health, 0.0, 100.0)
00263 
00264         per_cam[cam] = {
00265             "health": float(health),
00266             "std_mag": float(std_mag),
00267             "mae_mag": float(mae_mag),
00268             "E_std": float(E_std),
00269             "E_mae": float(E_mae),
00270         }
00271         scores.append(float(health))
00272 
00273     if len(scores) == 0:
00274         return {"health": 0.0, "method": "lifcal", "perCam": per_cam}
00275 
00276     # Global health: average across all cameras
00277     global_health = sum(scores) / len(scores)
00278 
00279     return {"health": float(global_health), "method": "lifcal", "perCam": per_cam}
00280 
00281 
00282 def compute_lifcal_health_from_combined_path(combined_json_path: str) -> Dict[str, Any]:
00283     """
00284     @brief Load combined.json from file and compute LiFCal health.
00285 
00286     @param combined_json_path Path to the combined.json file
00287     @return Dictionary with global health score, method name, and per-camera metrics
00288     """
00289     with open(combined_json_path, "r", encoding="utf-8") as f:
00290         combined = json.load(f)
00291     return compute_lifcal_health_from_combined_dict(combined)
```

6.7 calibrationLFC/ImageSet.py File Reference

Classes

- class ImageSet.ImageSet

Namespaces

- namespace ImageSet

6.8 ImageSet.py

Go to the documentation of this file.

```
00001 from typing import List
00002
00003 class ImageSet:
00004     """
00005     @brief Represents a set of calibration images organized by pose and camera.
00006     """
00007
00008     def __init__(self, baseDir: str, numPoses: int, cameraIds: List[str]):
00009         """
00010         @brief Constructor for ImageSet.
00011
00012         @param baseDir Base directory containing calibration images
00013         @param numPoses Total number of poses in the calibration set
00014         @param cameraIds List of camera identifiers
00015         """
00016         self.baseDir = baseDir
00017         self.numPoses = numPoses
00018         self.cameraIds = cameraIds
00019
00020     def getImagePath(self, poseIndex: int, cameraId: str) -> str:
00021         """
00022         @brief Construct the file path for a specific pose and camera.
00023
00024         @param poseIndex Zero-based pose index
00025         @param cameraId Camera identifier string
00026         @return String path in format: baseDir/pose_XXX_cameraId.png
00027         """
00028         poseStr = f"pose_{poseIndex:03d}"
00029         return f"{self.baseDir}/{poseStr}_{cameraId}.png"
00030
00031         """
00032             Beispiel:
00033             baseDir = "/data/images"
00034             poseIndex = 5  → poseStr = "pose_005"
00035             cameraId = "center"
00036
00037             Ergebnis: "/data/images/pose_005_center.png"
00038         """
```

6.9 calibrationLFC/InitialCalibration.py File Reference

Classes

- class InitialCalibration.InitialCalibration

Namespaces

- namespace InitialCalibration

6.10 InitialCalibration.py

Go to the documentation of this file.

```
00001 import cv2
00002 import numpy as np
00003 from CalibrationState import CalibrationState
00004 from scipy.optimize import least_squares
00005
00006
00007 class InitialCalibration:
00008     """
00009     @brief Multi-camera calibration using chessboard patterns and bundle adjustment.
00010     """
00011
00012     def __init__(self, chessboardSize=(8,8), squareSize=2/9):
00013         """
```

```
00014         @brief Constructor for InitialCalibration.
00015
00016         @param chessboardSize Tuple (cols, rows) defining chessboard dimensions (default: (8,8))
00017         @param squareSize Physical size of one chessboard square in meters (default: 2/9)
00018         """
00019         self.chessboardSize = chessboardSize
00020         self.squareSize = squareSize
00021         self.criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
00022
00023     def createObjectPoints(self):
00024         """
00025         @brief Generate 3D coordinates of chessboard corners.
00026
00027         @return Numpy array of shape (N, 3) with 3D corner coordinates
00028         """
00029         cols, rows = self.chessboardSize
00030         objp = np.zeros((cols * rows, 3), np.float32)
00031         objp[:, :2] = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2)
00032         objp *= self.squareSize
00033         return objp
00034
00035     def detectCorners(self, imagePath):
00036         """
00037         @brief Detect and refine chessboard corner positions in image.
00038
00039         @param imagePath Path to the image file
00040         @return Tuple (success, corners, imageSize) where success is bool,
00041         corners are refined 2D points, and imageSize is (width, height)
00042         """
00043         img = cv2.imread(imagePath)
00044         if img is None:
00045             return False, None, None
00046         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
00047
00048         found, corners = cv2.findChessboardCorners(gray, self.chessboardSize)
00049         if not found:
00050             return False, None, None
00051
00052         refined = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), self.criteria)
00053         return True, refined, gray.shape[::-1]
00054
00055     def calibrateIntrinsics(self, imageSet):
00056         """
00057         @brief Calibrate intrinsic parameters independently for each camera.
00058
00059         @param imageSet ImageSet object containing calibration images
00060         @return CalibrationState with populated intrinsics for all cameras
00061         """
00062         state = CalibrationState()
00063         objp = self.createObjectPoints()
00064
00065         objpoints = {cam: [] for cam in imageSet.cameraIds}
00066         imgpoints = {cam: [] for cam in imageSet.cameraIds}
00067         imgsize = {}
00068
00069         # Collect corner points from all poses for each camera
00070         for pose in range(imageSet.numPoses):
00071             for cam in imageSet.cameraIds:
00072                 path = imageSet.getImagePath(pose, cam)
00073                 ok, corners, size = self.detectCorners(path)
00074                 if ok:
00075                     objpoints[cam].append(objp)
00076                     imgpoints[cam].append(corners)
00077                     imgsize[cam] = size
00078
00079         # Calibrate each camera separately
00080         for cam in imageSet.cameraIds:
00081             if len(objpoints[cam]) < 5:
00082                 print(f"[WARN] Not enough samples for camera {cam}")
00083                 continue
00084
00085             err, K, dist, rvecs, tvecs = cv2.calibrateCamera(
00086                 objpoints[cam], imgpoints[cam], imgsize[cam], None, None)
00087
00088             state.setIntrinsics(cam, K, dist, err)
00089
00090         return state
00091
00092     def calibrateExtrinsics(self, imageSet, state, refCamId):
00093         """
00094         @brief Compute pairwise stereo calibration between reference and each other camera.
00095
00096         @param imageSet ImageSet object containing calibration images
00097         @param state CalibrationState with intrinsics already computed
00098         @param refCamId Camera ID to use as reference coordinate system
00099         @return Updated CalibrationState with extrinsics for all cameras
00100         """
```

```
00101         objectPoints = self.createObjectPoints()
00102         K_ref = state.intrinsics[refCamId]["cameraMatrix"]
00103         dist_ref = state.intrinsics[refCamId]["distortionCoeffs"]
00104 
00105         for cam in imageSet.cameraIds:
00106             if cam == refCamId:
00107                 state.setExtrinsics(cam, np.eye(3), np.zeros((3,1)), 0.0)
00108                 continue
00109 
00110             if cam not in state.intrinsics:
00111                 continue
00112 
00113             K_cam = state.intrinsics[cam]["cameraMatrix"]
00114             dist_cam = state.intrinsics[cam]["distortionCoeffs"]
00115 
00116             objList = []
00117             imgRef = []
00118             imgCam = []
00119             imageSize = None
00120 
00121             for pose in range(imageSet.numPoses):
00122                 pRef = imageSet.getImagePath(pose, refCamId)
00123                 pCam = imageSet.getImagePath(pose, cam)
00124 
00125                 okR, cornersR, sizeR = self.detectCorners(pRef)
00126                 okC, cornersC, sizeC = self.detectCorners(pCam)
00127 
00128                 if okR and okC:
00129                     objList.append(objectPoints)
00130                     imgRef.append(cornersR)
00131                     imgCam.append(cornersC)
00132                     imageSize = sizeR
00133 
00134             if len(objList) < 3:
00135                 continue
00136 
00137             flags = cv2.CALIB_FIX_INTRINSIC
00138 
00139             stereo_rms, KR0, d0, KC1, d1, R, T, E, F = cv2.stereoCalibrate(
00140                 objList,
00141                 imgRef,
00142                 imgCam,
00143                 K_ref,
00144                 dist_ref,
00145                 K_cam,
00146                 dist_cam,
00147                 imageSize,
00148                 criteria=self.criteria,
00149                 flags=flags
00150             )
00151 
00152             state.setExtrinsics(cam, R, T, stereo_rms)
00153 
00154         return state
00155 
00156     def bundleAdjustExtrinsics(self, imageSet, state, refCamId):
00157         """
00158         @brief Global bundle adjustment optimizing extrinsics and board poses.
00159 
00160         @param imageSet ImageSet object containing calibration images
00161         @param state CalibrationState with initial intrinsics and extrinsics
00162         @param refCamId Camera ID to use as reference coordinate system
00163         @return Updated CalibrationState with refined extrinsics after bundle adjustment
00164         """
00165         objp = self.createObjectPoints()
00166         cameraIds = list(imageSet.cameraIds)
00167         if refCamId not in cameraIds:
00168             raise ValueError("refCamId not in imageSet.cameraIds")
00169 
00170         ref_idx = cameraIds.index(refCamId)
00171 
00172         # Initialize board poses via PnP-RANSAC in reference camera
00173         K_ref = state.intrinsics[refCamId]["cameraMatrix"]
00174         dist_ref = state.intrinsics[refCamId]["distortionCoeffs"]
00175 
00176         pose_rvecs = {}
00177         pose_tvecs = {}
00178         valid_poses = []
00179 
00180         for pose in range(imageSet.numPoses):
00181             img_path = imageSet.getImagePath(pose, refCamId)
00182             ok, corners, size = self.detectCorners(img_path)
00183             if not ok:
00184                 continue
00185 
00186             # Use PnP-RANSAC for robust board pose estimation
00187             ok_pnp, rvec, tvec, inliers = cv2.solvePnPRansac(
```

```
00188                 objp,
00189                 corners,
00190                 K_ref,
00191                 dist_ref,
00192                 reprojectionError=2.0,
00193                 confidence=0.99,
00194                 flags=cv2.SOLVEPNP_ITERATIVE
00195             )
00196
00197             if not ok_pnp or inliers is None or len(inliers) < 10:
00198                 # Skip pose if too few reliable points
00199                 continue
00200
00201             pose_rvecs[pose] = rvec.astype(np.float64).reshape(3)
00202             pose_tvecs[pose] = tvec.astype(np.float64).reshape(3)
00203             valid_poses.append(pose)
00204
00205         if len(valid_poses) < 3:
00206             print("[BA] Not enough valid poses for bundle adjustment.")
00207             return state
00208
00209         # Initialize extrinsics from state
00210         # Only optimize non-reference cameras
00211         optim_cams = []
00212         cam_param_index = {}
00213
00214         for ci, cam in enumerate(cameraIds):
00215             if cam == refCamId:
00216                 continue
00217             if cam not in state.extrinsics:
00218                 continue
00219
00220             optim_cams.append(ci)
00221             cam_param_index[ci] = len(cam_param_index)
00222
00223         num_optim_cams = len(optim_cams)
00224         if num_optim_cams == 0:
00225             print("[BA] No extrinsic data to optimize, skipping BA.")
00226             return state
00227
00228         # Collect measurements (all cameras, all valid poses)
00229         # Each measurement: (cam_idx, pose, 2D points)
00230         measurements = []
00231
00232         for ci, cam in enumerate(cameraIds):
00233             for pose in valid_poses:
00234                 img_path = imageSet.getImagePath(pose, cam)
00235                 ok, corners, size = self.detectCorners(img_path)
00236                 if not ok:
00237                     continue
00238                 pts2d = corners.reshape(-1, 2).astype(np.float64)
00239                 measurements.append((ci, pose, pts2d))
00240
00241         if len(measurements) == 0:
00242             print("[BA] No overlapping detections found, skipping BA.")
00243             return state
00244
00245         # Initialize parameter vector
00246         # Layout: [cams (without ref): (r_cam(3), t_cam(3))... , poses: (r_p(3), t_p(3))...]
00247
00248         param_cam = []
00249         for ci in optim_cams:
00250             cam = cameraIds[ci]
00251             extr = state.extrinsics[cam]
00252             R = extr["rotationMatrix"]
00253             T = extr["translationVector"].reshape(3)
00254             rvec_cam, _ = cv2.Rodrigues(R)
00255             param_cam.append(rvec_cam.reshape(3))
00256             param_cam.append(T)
00257         param_cam = np.concatenate(param_cam).astype(np.float64)
00258
00259         param_pose = []
00260         for pose in valid_poses:
00261             param_pose.append(pose_rvecs[pose])
00262             param_pose.append(pose_tvecs[pose])
00263         param_pose = np.concatenate(param_pose).astype(np.float64)
00264
00265         x0 = np.concatenate([param_cam, param_pose])
00266
00267         # Helper functions for parameter decoding
00268         def decode_params(params):
00269             # Cameras
00270             cam_rvecs = {}
00271             cam_tvecs = {}
00272             idx = 0
00273             for ci in optim_cams:
00274                 r = params[idx:idx+3]; idx += 3
```

```
00275                 t = params[idx:idx+3]; idx += 3
00276                 cam_rvecs[ci] = r
00277                 cam_tvecs[ci] = t
00278
00279             # Poses
00280             pose_r = {}
00281             pose_t = {}
00282             for pose in valid_poses:
00283                 r = params[idx:idx+3]; idx += 3
00284                 t = params[idx:idx+3]; idx += 3
00285                 pose_r[pose] = r
00286                 pose_t[pose] = t
00287
00288             return cam_rvecs, cam_tvecs, pose_r, pose_t
00289
00290         # Residuals function
00291         def residuals(params):
00292             cam_rvecs, cam_tvecs, pose_r, pose_t = decode_params(params)
00293
00294             # Precompute rotation matrices
00295             R_cam = {}
00296             for ci, r in cam_rvecs.items():
00297                 R_cam[ci], _ = cv2.Rodrigues(r.astype(np.float64))
00298
00299             R_pose = {}
00300             for pose, r in pose_r.items():
00301                 R_pose[pose], _ = cv2.Rodrigues(r.astype(np.float64))
00302
00303             residual_list = []
00304
00305             for ci, pose, pts2d in measurements:
00306                 cam_id = cameraIds[ci]
00307                 K = state.intrinsics[cam_id]["cameraMatrix"]
00308                 dist = state.intrinsics[cam_id]["distortionCoeffs"]
00309
00310                 # Board pose in reference frame
00311                 r_p = pose_r[pose]
00312                 t_p = pose_t[pose]
00313                 R_p = R_pose[pose]
00314
00315                 if ci == ref_idx:
00316                     # Reference camera: use board pose directly
00317                     r_total = r_p
00318                     t_total = t_p
00319                 else:
00320                     if ci not in cam_rvecs:
00321                         # For non-optimized cameras, use static extrinsics
00322                         extr = state.extrinsics[cam_id]
00323                         R_c = extr["rotationMatrix"]
00324                         t_c = extr["translationVector"].reshape(3)
00325                     else:
00326                         R_c = R_cam[ci]
00327                         t_c = cam_tvecs[ci]
00328
00329                     # Composition: X_cam = R_c * (R_p * X + t_p) + t_c
00330                     R_total = R_c @ R_p
00331                     t_total = R_c @ t_p + t_c
00332                     r_total, _ = cv2.Rodrigues(R_total.astype(np.float64))
00333                     r_total = r_total.reshape(3)
00334
00335                 # Projection
00336                 proj, _ = cv2.projectPoints(
00337                     objp,
00338                     r_total.astype(np.float64),
00339                     t_total.astype(np.float64),
00340                     K,
00341                     dist
00342                 )
00343                 proj = proj.reshape(-1, 2)
00344
00345                 res = (proj - pts2d).reshape(-1)
00346                 residual_list.append(res)
00347
00348             if not residual_list:
00349                 return np.zeros(0, dtype=np.float64)
00350
00351             return np.concatenate(residual_list)
00352
00353         # Optimization (robust, soft_l1)
00354         result = least_squares(
00355             residuals,
00356             x0,
00357             verbose=2,
00358             method="trf",
00359             loss="soft_l1",
00360             f_scale=1.0
00361         )
```

```
00362
00363         print(
00364             f"[BA] Done. Final cost: {result.cost:.4f}, "
00365             f"RMS per coord: {np.sqrt(2*result.cost/len(result.fun)):.4f} px"
00366         )
00367
00368         # Write results back to state.extrinsics
00369         cam_rvecs_opt, cam_tvecs_opt, pose_r_opt, pose_t_opt = decode_params(result.x)
00370
00371         # Reference camera remains identity
00372         state.setExtrinsics(
00373             refCamId,
00374             np.eye(3, dtype=np.float64),
00375             np.zeros((3, 1), dtype=np.float64),
00376             0.0
00377         )
00378
00379         # Store optimized cameras
00380         for ci in optim_cams:
00381             cam_id = cameraIds[ci]
00382             r = cam_rvecs_opt[ci]
00383             t = cam_tvecs_opt[ci]
00384             R, _ = cv2.Rodrigues(r.astype(np.float64))
00385             T = t.reshape(3, 1).astype(np.float64)
00386
00387             # Approximate RMS for this camera from global residuals
00388             # (simple average across all measurements)
00389             cam_residuals = []
00390             for m_ci, pose, pts2d in measurements:
00391                 if m_ci != ci:
00392                     continue
00393                 # Reproject with final parameters
00394                 R_p, _ = cv2.Rodrigues(pose_r_opt[pose].astype(np.float64))
00395                 t_p = pose_t_opt[pose].astype(np.float64)
00396
00397                 R_total = R @ R_p
00398                 t_total = R @ t_p + t.flatten()
00399                 r_total, _ = cv2.Rodrigues(R_total)
00400                 K = state.intrinsics[cam_id]["cameraMatrix"]
00401                 dist = state.intrinsics[cam_id]["distortionCoeffs"]
00402
00403                 proj, _ = cv2.projectPoints(
00404                     objp, r_total, t_total, K, dist
00405                 )
00406                 proj = proj.reshape(-1, 2)
00407                 res = (proj - pts2d).reshape(-1)
00408                 cam_residuals.append(res)
00409
00410             if cam_residuals:
00411                 cam_residuals = np.concatenate(cam_residuals)
00412                 stereo_rms = float(np.sqrt(np.mean(cam_residuals**2)))
00413             else:
00414                 stereo_rms = state.extrinsics[cam_id].get("stereoRms", 0.0)
00415
00416             state.setExtrinsics(cam_id, R, T, stereo_rms)
00417
00418         return state
00419
00420     def run(self, imageSet, bundleAdjust=True):
00421         """
00422         @brief Main entry point for calibration pipeline.
00423
00424         @param imageSet ImageSet object containing calibration images
00425         @param bundleAdjust Whether to apply bundle adjustment (default: True)
00426         @return CalibrationState with complete intrinsic and extrinsic calibration
00427         """
00428         print("Running intrinsic calibration...")
00429         state = self.calibrateIntrinsics(imageSet)
00430
00431         print("\nRunning pairwise extrinsic calibration...")
00432         state = self.calibrateExtrinsics(imageSet, state, refCamId=imageSet.cameraIds[0])
00433
00434         if bundleAdjust:
00435             print("\nRunning global bundle adjustment on extrinsics...")
00436             state = self.bundleAdjustExtrinsics(imageSet, state, refCamId=imageSet.cameraIds[0])
00437
00438         return state
```

6.11 calibrationLFC/ResultLogger.py File Reference

Classes

- class ResultLogger.ResultLogger

Namespaces

- namespace ResultLogger

6.12 ResultLogger.py

Go to the documentation of this file.

```
00001 import os
00002 import json
00003 import logging
00004 import numpy as np
00005 from datetime import datetime
00006
00007
00008 class ResultLogger:
00009     """
00010     @brief Manages calibration result logging and baseline versioning.
00011     """
00012
00013     def __init__(self, baseDir="/data/baseline"):
00014         """
00015         @brief Constructor for ResultLogger.
00016
00017         @param baseDir Directory for storing calibration results and baseline (default:
      "/data/baseline")
00018         """
00019         self.baseDir = baseDir
00020         os.makedirs(baseDir, exist_ok=True)
00021
00022         self.logger = logging.getLogger("calibration")
00023         self.logger.setLevel(logging.INFO)
00024
00025         if not self.logger.handlers:
00026             log_path = os.path.join(baseDir, "calibration.log")
00027             file_handler = logging.FileHandler(log_path, encoding="utf-8")
00028             file_handler.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00029             self.logger.addHandler(file_handler)
00030
00031     def to_serializable(self, obj):
00032         """
00033         @brief Convert numpy arrays and types to JSON-serializable format.
00034
00035         @param obj Object to convert (numpy array, dict, list, or primitive type)
00036         @return JSON-serializable version of the object
00037         """
00038         if isinstance(obj, np.ndarray):
00039             return obj.tolist()
00040         if isinstance(obj, (np.floating, np.float32, np.float64)):
00041             return float(obj)
00042         if isinstance(obj, (np.integer, np.int32, np.int64)):
00043             return int(obj)
00044         if isinstance(obj, dict):
00045             return {k: self.to_serializable(v) for k, v in obj.items()}
00046         if isinstance(obj, (list, tuple)):
00047             return [self.to_serializable(v) for v in obj]
00048         return obj
00049
00050     def read_baseline_health(self, baseline_path: str) -> float:
00051         """
00052         @brief Extract health score from baseline.json.
00053
00054         @param baseline_path Path to the baseline.json file
00055         @return Health score as float, or -1.0 if not found or cannot be determined
00056         """
00057         if not os.path.isfile(baseline_path):
00058             return -1.0
00059
00060         try:
00061             with open(baseline_path, "r", encoding="utf-8") as f:
00062                 base = json.load(f)
00063         except Exception:
00064             return -1.0
00065
00066         # Try direct health value from metadata
00067         try:
00068             h = float(base.get("meta", {}).get("health", -1.0))
00069             if np.isfinite(h) and h >= 0:
00070                 return h
00071         except Exception:
00072             pass
```

```
00073
00074         # Fallback: compute based on calibration type
00075         try:
00076             from HealthMonitor import (
00077                 compute_initial_health_from_state,
00078                 compute_lifcal_health_from_combined_dict,
00079             )
00080         except Exception:
00081             return -1.0
00082
00083         run_type = (base.get("meta", {}) or {}).get("runType", "")
00084         state = base.get("state", {})
00085
00086         try:
00087             if run_type == "initial":
00088                 rep = compute_initial_health_from_state(state)
00089                 return float(rep.get("health", -1.0))
00090             else:
00091                 # Treat as lifcal/combined
00092                 rep = compute_lifcal_health_from_combined_dict(base)
00093                 return float(rep.get("health", -1.0))
00094         except Exception:
00095             return -1.0
00096
00097     def update_baseline(self, candidate_obj: dict, candidate_score: float, method: str) -> None:
00098         """
00099         @brief Update baseline.json with new calibration if health improved.
00100
00101         @param candidate_obj Dictionary containing calibration data
00102         @param candidate_score Health score of the candidate calibration
00103         @param method Calibration method identifier ("initial" or "lifcal")
00104         """
00105         baseline_path = os.path.join(self.baseDir, "baseline.json")
00106         old_score = self.read_baseline_health(baseline_path)
00107
00108         if (not os.path.isfile(baseline_path)) or (candidate_score > old_score):
00109             try:
00110                 with open(baseline_path, "w", encoding="utf-8") as f:
00111                     json.dump(candidate_obj, f, indent=2)
00112                 if old_score < 0:
00113                     self.logger.info(f"Baseline created: baseline.json (new={candidate_score:.2f},
      method={method})")
00114                 else:
00115                     self.logger.info(
00116                         f"Baseline updated: baseline.json (old={old_score:.2f},
      new={candidate_score:.2f}, method={method})"
00117                     )
00118             except Exception as e:
00119                 self.logger.error(f"Failed to update baseline.json: {e}")
00120         else:
00121             self.logger.info(
00122                 f"Baseline kept: baseline.json (old={old_score:.2f} >= new={candidate_score:.2f},
      method={method})"
00123             )
00124
00125     def logInitialCalibration(self, calibrationState, meta=None):
00126         """
00127         @brief Store initial calibration result with health score.
00128
00129         @param calibrationState CalibrationState object with intrinsic and extrinsic parameters
00130         @param meta Optional dictionary with metadata about the calibration run
00131         @return Health score as float
00132         """
00133         if meta is None:
00134             meta = {}
00135
00136         # Extract calibration state
00137         try:
00138             state_dict = calibrationState.__getState__()
00139         except Exception as e:
00140             self.logger.error(f"Could not extract state dict from CalibrationState: {e}")
00141             return 0.0
00142
00143         safe_state = self.to_serializable(state_dict)
00144         safe_meta = self.to_serializable(meta)
00145
00146         # Compute health score from metrics
00147         try:
00148             from HealthMonitor import compute_initial_health_from_state
00149             report = compute_initial_health_from_state(state_dict)
00150             score = float(report.get("health", 0.0))
00151         except Exception as e:
00152             self.logger.error(f"Initial HealthScore computation failed: {e}", exc_info=True)
00153             score = 0.0
00154
00155         # Create timestamped filename
00156         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```
00157         imagesetnumber = meta.get("imageDir", "unknown")
00158         if isinstance(imagesetnumber, str) and "imageset" in imagesetnumber:
00159             imagesetnumber = imagesetnumber.split("imageset")[-1]
00160 
00161         json_name = f"calibration_initial_imageset{imagesetnumber}_{timestamp}.json"
00162         json_path = os.path.join(self.baseDir, json_name)
00163 
00164         # Prepare output structure
00165         out = {
00166             "meta": safe_meta,
00167             "state": safe_state
00168         }
00169 
00170         # Attach health score to metadata
00171         out.setdefault("meta", {})
00172         out["meta"]["health"] = score
00173         out["meta"]["method"] = "initial"
00174         out["meta"]["runType"] = out["meta"].get("runType", "initial")
00175 
00176         # Save to file
00177         try:
00178             with open(json_path, "w", encoding="utf-8") as f:
00179                 json.dump(out, f, indent=2)
00180         except Exception as e:
00181             self.logger.error(f"Failed to write JSON result file '{json_path}': {e}")
00182             return 0.0
00183 
00184         intr = state_dict.get("intrinsics", {})
00185         extr = state_dict.get("extrinsics", {})
00186         self.logger.info(
00187             f"Stored initial calibration to '{json_name}' "
00188             f"(intrinsics for {len(intr)} cams, extrinsics for {len(extr)} cams, bundleAdjustf:
      {meta.get('bundleAdjust', True)})."
00189         )
00190         self.logger.info(f"HealthScore = {score:.2f} (method=initial)")
00191 
00192         # Update baseline (creates baseline on first run)
00193         self.update_baseline(out, score, method="initial")
00194 
00195         return score
00196 
00197     def logRecalibration(self, combined_json_path: str, meta=None):
00198         """
00199         @brief Store LiFCal recalibration result with health score.
00200 
00201         @param combined_json_path Path to the combined.json file from LiFCal
00202         @param meta Optional dictionary with metadata about the recalibration run
00203         @return Health score as float
00204         """
00205         if meta is None:
00206             meta = {}
00207 
00208         # Import locally to avoid errors when not needed
00209         from HealthMonitor import compute_lifcal_health_from_combined_path
00210 
00211         # Read combined.json
00212         try:
00213             with open(combined_json_path, "r", encoding="utf-8") as f:
00214                 combined = json.load(f)
00215         except Exception as e:
00216             self.logger.error(f"Could not read combined.json '{combined_json_path}': {e}")
00217             return 0.0
00218 
00219         # Compute health score
00220         try:
00221             report = compute_lifcal_health_from_combined_path(combined_json_path)
00222             score = float(report.get("health", 0.0))
00223         except Exception as e:
00224             self.logger.error(f"LiFCal HealthScore computation failed: {e}", exc_info=True)
00225             score = 0.0
00226 
00227         # Create timestamped copy
00228         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
00229         json_name = f"calibration_lifcal_{timestamp}.json"
00230         out_path = os.path.join(self.baseDir, json_name)
00231 
00232         combined.setdefault("meta", {})
00233         combined["meta"].update(self.to_serializable(meta))
00234         combined["meta"]["health"] = score
00235         combined["meta"]["method"] = "lifcal"
00236         combined["meta"]["runType"] = combined["meta"].get("runType", "recalib")
00237 
00238         # Save to file
00239         try:
00240             with open(out_path, "w", encoding="utf-8") as f:
00241                 json.dump(combined, f, indent=2)
00242         except Exception as e:
```

```
00243             self.logger.error(f"Failed to write LiFCal result '{out_path}': {e}")
00244             return 0.0
00245
00246         num_cams = len(combined.get("state", {}).get("parameters", {}))
00247         self.logger.info(f"Stored lifcal recalibration to '{json_name}' (cams={num_cams}).")
00248         self.logger.info(f"HealthScore = {score:.2f} (method=lifcal)")
00249
00250         # Update baseline only if better
00251         self.update_baseline(combined, score, method="lifcal")
00252
00253         return score
```

6.13 calibrationLFC/results/evaluate_calibration.py File Reference

Namespaces

- namespace evaluate_calibration

Functions

- evaluate_calibration.rot_angle_deg (R)
- evaluate_calibration.as_vec3 (v)
- evaluate_calibration.compute_metrics_for_file (Path calib_path, Path gt_path)
- evaluate_calibration.print_per_file_table (results_by_name)
- evaluate_calibration.print_scenario_table (results_by_name)
- evaluate_calibration.main ()

Variables

- evaluate_calibration.THIS_DIR = Path(__file__).resolve().parent
- evaluate_calibration.CALIB_LFC_DIR = THIS_DIR.parent
- evaluate_calibration.PROJECT_ROOT = CALIB_LFC_DIR.parent
- str evaluate_calibration.RESULTS_DIR = CALIB_LFC_DIR / "results"
- str evaluate_calibration.GT_DIR = PROJECT_ROOT / "TestEnvironment" / "params"
- list evaluate_calibration.CAM_IDS
- dict evaluate_calibration.CALIB_FILES
- dict evaluate_calibration.GT_FILES
- dict evaluate_calibration.SCENARIOS

6.14 evaluate_calibration.py

Go to the documentation of this file.

```
00001 import json
00002 import numpy as np
00003 from pathlib import Path
00004
00005 # Paths
00006 THIS_DIR = Path(__file__).resolve().parent
00007 CALIB_LFC_DIR = THIS_DIR.parent          # calibrationLFC/
00008 PROJECT_ROOT = CALIB_LFC_DIR.parent      # CameraCalibration/
00009 RESULTS_DIR = CALIB_LFC_DIR / "results"
00010
00011 # Groundtruth base directory
00012 GT_DIR = PROJECT_ROOT / "TestEnvironment" / "params"
00013
00014 # Camera order
00015 CAM_IDS = [
00016     "Center",
00017     "Up1", "Up2", "Up3",
```

```
00018     "Down1", "Down2", "Down3",
00019     "Left1", "Left2", "Left3",
00020     "Right1", "Right2", "Right3",
00021 ]
00022
00023 # Calibration JSONs
00024 CALIB_FILES = {
00025     "imageset1": RESULTS_DIR / "calibration_initial_imageset1_20251215_173701.json",
00026     "imageset2": RESULTS_DIR / "calibration_initial_imageset2_20251215_181903.json",
00027     "imageset3": RESULTS_DIR / "calibration_initial_imageset3_20251215_185041.json",
00028     "imageset4": RESULTS_DIR / "calibration_initial_imageset4_20251215_165644.json",
00029     "imageset5": RESULTS_DIR / "calibration_initial_imageset5_20251217_011656.json",
00030     "imageset6": RESULTS_DIR / "calibration_initial_imageset6_20251215_201527.json",
00031     "imageset7": RESULTS_DIR / "calibration_initial_imageset7_20251215_203849.json",
00032 }
00033
00034 # Groundtruth per calibration
00035 GT_FILES = {
00036     # Rig0 for imageset1-4
00037     "imageset1": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00038     "imageset2": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00039     "imageset3": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00040     "imageset4": GT_DIR / "groundtruth_extrinsics_Rig0.json",
00041
00042     # Different rigs/sets:
00043     "imageset5": GT_DIR / "groundtruth_extrinsics_Rig1_set5.json",
00044     "imageset6": GT_DIR / "groundtruth_extrinsics_Rig2_set6.json",
00045     "imageset7": GT_DIR / "groundtruth_extrinsics_Rig3_set7.json",
00046 }
00047
00048 # Scenario grouping for thesis table
00049 SCENARIOS = {
00050     "A_Robustheit": ["imageset1", "imageset2", "imageset3", "imageset4"],
00051     "B_Generalisation":    ["imageset5", "imageset6", "imageset7"],
00052 }
00053
00054 # Helper functions
00055 def rot_angle_deg(R):
00056     """Extract rotation angle (degrees) from 3x3 rotation matrix."""
00057     tr = np.trace(R)
00058     c = (tr - 1.0) / 2.0
00059     c = np.clip(c, -1.0, 1.0)
00060     return float(np.degrees(np.arccos(c)))
00061
00062 def as_vec3(v):
00063     """Convert any translation vector to robust 3-element vector."""
00064     arr = np.array(v, dtype=float).reshape(-1)
00065     if arr.size != 3:
00066         raise ValueError(f"Expected 3 entries for translation, got {arr.size}")
00067     return arr
00068
00069
00070 def compute_metrics_for_file(calib_path: Path, gt_path: Path):
00071     """
00072     Loads a calibration JSON and groundtruth JSON and computes:
00073       - numPoses
00074       - mean T (m)
00075       - mean R (°)
00076       - mean reprojection error (px, from intrinsics)
00077
00078     Definitions as in your extrinsic plot:
00079       R_cam = angle(R_est * R_gt^T)
00080       T_cam = || T_est - T_gt ||_2
00081     """
00082     with open(gt_path, "r", encoding="utf-8") as f:
00083         gt = json.load(f)
00084
00085     with open(calib_path, "r", encoding="utf-8") as f:
00086         calib = json.load(f)
00087
00088     meta = calib.get("meta", {})
00089     extr_est = calib["state"]["extrinsics"]
00090     intr_est = calib["state"]["intrinsics"]
00091
00092     bundle_adjust = bool(meta.get("bundleAdjust", False))
00093     num_poses = int(meta.get("numPoses", -1))
00094
00095     rot_err = []
00096     trans_err = []
00097     reproj_err = []
00098
00099     for cam in CAM_IDS:
00100         if cam not in extr_est or cam not in gt:
00101             print(f"[WARN] Camera '{cam}' not in both files for {calib_path.name}, skipping this
      cam.")
00102             continue
00103
```

```
00104         # Extrinsics
00105         R_est = np.array(extr_est[cam]["rotationMatrix"], dtype=float)
00106         R_gt  = np.array(gt[cam]["rotationMatrix"], dtype=float)
00107
00108         T_est = as_vec3(extr_est[cam]["translationVector"])
00109         T_gt  = as_vec3(gt[cam]["translationVector"])
00110
00111         # Rotation error as in plot
00112         R_err = R_est @ R_gt.T
00113         ang = rot_angle_deg(R_err)
00114
00115         # Translation error (absolute, m)
00116         d_abs = float(np.linalg.norm(T_est - T_gt))
00117
00118         rot_err.append(ang)
00119         trans_err.append(d_abs)
00120
00121         # Intrinsic reprojection error
00122         if cam in intr_est:
00123             reproj_err.append(float(intr_est[cam]["reprojectionError"]))
00124
00125     # Mean values
00126     mean_dT = float(np.mean(trans_err))     if trans_err else float("nan")
00127     mean_dR = float(np.mean(rot_err))       if rot_err else float("nan")
00128     mean_reproj = float(np.mean(reproj_err)) if reproj_err else float("nan")
00129
00130     return {
00131         "bundleAdjust": bundle_adjust,
00132         "numPoses": num_poses,
00133         "mean_dT": mean_dT,
00134         "mean_dR": mean_dR,
00135         "mean_reproj": mean_reproj,
00136     }
00137
00138 # Table output (console)
00139 def print_per_file_table(results_by_name):
00140     """
00141     Prints one line per calibration JSON.
00142     """
00143     header = (
00144         f"{'Name':<12} "
00145         f"{'#Poses':>6} "
00146         f"{'BA':<4} "
00147         f"{'mean dT [m]':>12} "
00148         f"{'mean dR [deg]':>13} "
00149         f"{'mean reproj [px]':>16}"
00150     )
00151     print(header)
00152     print("-" * len(header))
00153
00154     for name, res in sorted(results_by_name.items()):
00155         ba_flag = "yes" if res["bundleAdjust"] else "no"
00156         print(
00157             f"{name:<12} "
00158             f"{res['numPoses']:6d} "
00159             f"{ba_flag:<4} "
00160             f"{res['mean_dT']:12.5f} "
00161             f"{res['mean_dR']:12.5f} "
00162             f"{res['mean_reproj']:16.5f}"
00163         )
00164
00165 def print_scenario_table(results_by_name):
00166     """
00167     Aggregates over scenarios (Robustness / Generalization)
00168     and prints a compact table.
00169     """
00170     print("\n\nScenario Summary:\n")
00171
00172     header = (
00173         f"{'Szenario':<22} "
00174         f"{'#Runs':>6} "
00175         f"{'Avg#Poses':>9} "
00176         f"{'mean dT [m]':>12} "
00177         f"{'mean dR [deg]':>13} "
00178         f"{'mean reproj [px]':>16}"
00179     )
00180     print(header)
00181     print("-" * len(header))
00182
00183     for scen_name, calib_names in SCENARIOS.items():
00184         dT_vals = []
00185         dR_vals = []
00186         reproj_vals = []
00187         pose_counts = []
00188
00189         for cname in calib_names:
00190             res = results_by_name.get(cname)
```

```
00191             if res is None:
00192                 continue
00193
00194             dT_vals.append(res["mean_dT"])
00195             dR_vals.append(res["mean_dR"])
00196             reproj_vals.append(res["mean_reproj"])
00197             pose_counts.append(res["numPoses"])
00198
00199         if not dT_vals:
00200             continue
00201
00202         mean_dT = float(np.mean(dT_vals))
00203         mean_dR = float(np.mean(dR_vals))
00204         mean_reproj = float(np.mean(reproj_vals))
00205         mean_poses = float(np.mean(pose_counts))
00206         num_runs = len(dT_vals)
00207
00208         print(
00209             f"{scen_name:<22} "
00210             f"{num_runs:6d} "
00211             f"{mean_poses:9.1f} "
00212             f"{mean_dT:12.5f} "
00213             f"{mean_dR:13.5f} "
00214             f"{mean_reproj:16.5f}"
00215         )
00216
00217 def main():
00218     results_by_name = {}
00219
00220     for name, calib_path in CALIB_FILES.items():
00221         if not calib_path.exists():
00222             print(f"[WARN] File not found: {calib_path}")
00223             continue
00224
00225         gt_path = GT_FILES.get(name)
00226         if gt_path is None:
00227             print(f"[WARN] No GT file mapped for {name}, skipping.")
00228             continue
00229         if not gt_path.exists():
00230             print(f"[WARN] GT file not found for {name}: {gt_path}")
00231             continue
00232
00233         metrics = compute_metrics_for_file(calib_path, gt_path)
00234         results_by_name[name] = metrics
00235
00236     if not results_by_name:
00237         print("[ERROR] No valid results computed. Check file paths.")
00238         return
00239
00240     print("Detailed results per calibration:\n")
00241     print_per_file_table(results_by_name)
00242
00243     print_scenario_table(results_by_name)
00244
00245
00246 if __name__ == "__main__":
00247     main()
```

6.15 calibrationLFC/run_initial_calibration.py File Reference

Namespaces

- namespace run_initial_calibration

Functions

- run_initial_calibration.parse_args ()
- run_initial_calibration.main (args)

Variables

- str run_initial_calibration.DEFAULT_BASE_DIR = "/data/calibrationLFC/sets/imageset5"
- int run_initial_calibration.DEFAULT_NUM_POSES = 28
- list run_initial_calibration.CAM_IDS
- run_initial_calibration.repoRoot = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

6.16 run_initial_calibration.py

Go to the documentation of this file.

```
00001 """
00002 Test runner for initial calibration.
00003
00004 Usage:
00005   cd calibrationLFC
00006   python3 run_initial_calibration.py --base_dir /path/to/imageset --num_poses 28 --bundle-adjust
00007   python3 run_initial_calibration.py --base_dir /path/to/imageset --num_poses 28 --no-bundle-adjust
00008
00009 Performs validation checks (file existence, corner detection on pose 0)
00010 then runs the complete initial calibration pipeline. Outputs summary statistics
00011 for intrinsics, extrinsics, and health score.
00012 """
00013
00014 import argparse
00015 import os
00016 import sys
00017
00018 from ImageSet import ImageSet
00019 from Controller import Controller
00020
00021
00022 # Default configuration parameters
00023 DEFAULT_BASE_DIR = "/data/calibrationLFC/sets/imageset5"
00024 DEFAULT_NUM_POSES = 28
00025 CAM_IDS = [
00026     "Center",
00027     "Up1", "Up2", "Up3",
00028     "Down1", "Down2", "Down3",
00029     "Left1", "Left2", "Left3",
00030     "Right1", "Right2", "Right3",
00031 ]
00032
00033
00034 def parse_args():
00035     """
00036     @brief Parse command line arguments for calibration configuration.
00037
00038     @return Parsed arguments with base_dir, num_poses, and bundle_adjust settings
00039     """
00040     parser = argparse.ArgumentParser(description="Run initial camera calibration")
00041     parser.add_argument("--base_dir", default=DEFAULT_BASE_DIR, help="Path to imageset directory")
00042     parser.add_argument("--num_poses", type=int, default=DEFAULT_NUM_POSES, help="Number of poses in
      the imageset")
00043     parser.add_argument(
00044         "--bundle-adjust",
00045         dest="bundle_adjust",
00046         action="store_true",
00047         help="Enable bundle adjustment (default)",
00048     )
00049     parser.add_argument(
00050         "--no-bundle-adjust",
00051         dest="bundle_adjust",
00052         action="store_false",
00053         help="Disable bundle adjustment",
00054     )
00055     parser.set_defaults(bundle_adjust=True)
00056     return parser.parse_args()
00057
00058
00059 def main(args):
00060     """
00061     @brief Run complete initial calibration pipeline with validation and summary output.
00062
00063     @param args
00064     """
00065     print("Creating ImageSet...")
00066     imageSet = ImageSet(args.base_dir, args.num_poses, CAM_IDS)
00067
00068     print("Instantiating Controller and InitialCalibration...")
00069     controller = Controller(bundleAdjust=args.bundle_adjust)
00070     logger = controller.resultLogger
00071
00072     print("\nQuick checks (pose 0): file exists / corners detected")
00073     for cam in CAM_IDS:
00074         path = imageSet.getImagePath(0, cam)
00075         exists = os.path.exists(path)
00076         cornersFound = None
00077
00078         try:
00079             cornersFound, _, _ = controller.initialCalibration.detectCorners(path)
00080         except Exception as e:
00081             cornersFound = f"error: {e.__class__.__name__}"
```

```
00082         print(f" {cam}: {path} - exists={exists} - corners={cornersFound}")
00083
00084     print("\nRunning initial calibration (this may take time depending on data)...")
00085     try:
00086         calibrationState = controller.runInitialCalibration(imageSet)
00087     except Exception as e:
00088         logger.logger.error(f"Calibration failed with exception: {e}", exc_info=True)
00089         print("Calibration failed, see calibration.log for details.")
00090         raise
00091
00092     print("\nCalibration finished. Attempting to print summary...")
00093     try:
00094         stateDict = calibrationState.__getState__()
00095     except Exception:
00096         stateDict = None
00097
00098     if stateDict is None:
00099         print("Could not extract a state dictionary.")
00100         return
00101
00102     intr = stateDict.get("intrinsics", {})
00103     extr = stateDict.get("extrinsics", {})
00104
00105     # Print calibration results per camera
00106     print("\nCalibration Results by Camera")
00107     print("=" * 50)
00108
00109     for cam in CAM_IDS:
00110         print(f"\n--- {cam} ---")
00111
00112         # Intrinsic parameters
00113         if cam in intr:
00114             data = intr[cam]
00115             K = data["cameraMatrix"]
00116             dist = data["distortionCoeffs"].ravel()
00117             err = data["reprojectionError"]
00118
00119             print("Intrinsics:")
00120             print(" K =")
00121             print(f"  {K[0][0]:.3f} {K[0][1]:.3f} {K[0][2]:.3f}")
00122             print(f"  {K[1][0]:.3f} {K[1][1]:.3f} {K[1][2]:.3f}")
00123             print(f"  {K[2][0]:.3f} {K[2][1]:.3f} {K[2][2]:.3f}")
00124             print(" distCoeffs =", dist)
00125             print(f" reprojectionError = {err:.6f}")
00126         else:
00127             print("Intrinsics: (no data)")
00128
00129         # Extrinsic parameters
00130         if cam in extr:
00131             data = extr[cam]
00132             R = data["rotationMatrix"]
00133             T = data["translationVector"].ravel()
00134             rms = data["stereoRms"]
00135
00136             print("Extrinsics:")
00137             print(" R =")
00138             print(f"  {R[0][0]:.5f} {R[0][1]:.5f} {R[0][2]:.5f}")
00139             print(f"  {R[1][0]:.5f} {R[1][1]:.5f} {R[1][2]:.5f}")
00140             print(f"  {R[2][0]:.5f} {R[2][1]:.5f} {R[2][2]:.5f}")
00141             print(" T =", T)
00142             print(f" stereoRMS = {rms:.6f}")
00143         else:
00144             print("  Extrinsics: (no data)")
00145
00146     # Print health score
00147     print("\nOverall Health Score")
00148     print("=" * 50)
00149     score = controller.selfHealthCheck(calibrationState)
00150     print(f"Global HealthScore = {score:.2f} / 100")
00151
00152 if __name__ == "__main__":
00153     # Set up Python path to find calibrationLFC modules
00154     repoRoot = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
00155     if repoRoot not in sys.path:
00156         sys.path.insert(0, repoRoot)
00157     main(parse_args())
```

6.17 calibrationLFC/run_periodic_lifcal.py File Reference

Namespaces

- namespace run_periodic_lifcal

Functions

- run_periodic_lifcal.setup_logger ()
- bool run_periodic_lifcal.acquire_lock (logger)
- run_periodic_lifcal.release_lock (logger)
- int run_periodic_lifcal.run_cmd (logger, cmd, cwd=None)
- int run_periodic_lifcal.count_images_in_dir (str d, exts=(".png", ".jpg", ".jpeg"))
- bool run_periodic_lifcal.needs_imageset_creation (logger)
- str run_periodic_lifcal.find_latest_run_dir (str out_root)
- str run_periodic_lifcal.combined_json_path (str run_dir)
- run_periodic_lifcal.try_log_lifcal_with_resultlogger (logger, str combined_path)
- run_periodic_lifcal.run_cycle (logger)
- run_periodic_lifcal.main ()

Variables

- int run_periodic_lifcal.INTERVAL_SECONDS = 5 ∗ 60
- str run_periodic_lifcal.RUN_IMAGESET_CREATION = "/data/external/LiFCal/LiFCal_Data/run_imageset_↩ creation.py"
- str run_periodic_lifcal.RUN_LIFCAL_CALIB = "/data/external/LiFCal/LiFCal_Data/run_LiFCal_calibration.py"
- str run_periodic_lifcal.LIFCAL_OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/Calibration↩ ByCamera"
- str run_periodic_lifcal.BASELINE_DIR = "/data/baseline"
- run_periodic_lifcal.LOG_PATH = os.path.join(BASELINE_DIR, "calibration.log")
- str run_periodic_lifcal.LOCK_PATH = "/tmp/lifcal_periodic.lock"
- str run_periodic_lifcal.ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_↩ Imageset"
- run_periodic_lifcal.FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
- run_periodic_lifcal.DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
- list run_periodic_lifcal.CAMERA_IDS
- int run_periodic_lifcal.MIN_POSES_PER_CAM = 1

6.18 run_periodic_lifcal.py

Go to the documentation of this file.

```
00001 import os
00002 import sys
00003 import time
00004 import json
00005 import logging
00006 import subprocess
00007 from datetime import datetime
00008
00009 """
00010 @brief Periodic LiFCal scheduler that prepares imagesets, runs calibration, and logs health.
00011
00012 """
00013
00014 INTERVAL_SECONDS = 5 * 60  # 5 Minutes interval
00015
00016 RUN_IMAGESET_CREATION = "/data/external/LiFCal/LiFCal_Data/run_imageset_creation.py"
00017 RUN_LIFCAL_CALIB      = "/data/external/LiFCal/LiFCal_Data/run_LiFCal_calibration.py"
00018
00019 LIFCAL_OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/CalibrationByCamera"
00020 BASELINE_DIR    = "/data/baseline"
00021 LOG_PATH        = os.path.join(BASELINE_DIR, "calibration.log")
00022
00023 LOCK_PATH = "/tmp/lifcal_periodic.lock"
00024
00025 # Prepared images
00026 ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
00027 FOCUS_ROOT    = os.path.join(ROOT_IMAGESET, "focus")   # focus/<CamId>/*.png
```

```
00028 DEPTH_ROOT    = os.path.join(ROOT_IMAGESET, "depth")   # depth/<CamId>/*_depth.png
00029
00030 CAMERA_IDS = [
00031     "Center", "Up1", "Up2", "Up3",
00032     "Down1", "Down2", "Down3",
00033     "Left1", "Left2", "Left3",
00034     "Right1", "Right2", "Right3"
00035 ]
00036
00037 # Minimum number of poses required per camera
00038 MIN_POSES_PER_CAM = 1
00039
00040
00041 def setup_logger():
00042     """
00043     @brief Create logger that writes to file and stdout.
00044
00045     @return Logger instance configured for calibration logging
00046     """
00047     os.makedirs(BASELINE_DIR, exist_ok=True)
00048     logger = logging.getLogger("lifcal_periodic")
00049     logger.setLevel(logging.INFO)
00050
00051     if not logger.handlers:
00052         fh = logging.FileHandler(LOG_PATH, encoding="utf-8")
00053         fh.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00054         logger.addHandler(fh)
00055
00056         sh = logging.StreamHandler(sys.stdout)
00057         sh.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(message)s"))
00058         logger.addHandler(sh)
00059
00060     return logger
00061
00062
00063 def acquire_lock(logger) -> bool:
00064     """
00065     @brief Create lock file to prevent overlapping scheduler cycles.
00066
00067     @param logger Logger instance for status messages
00068     @return True if lock was acquired, False otherwise
00069     """
00070     if os.path.exists(LOCK_PATH):
00071         logger.warning(f"Lock exists, skipping this cycle: {LOCK_PATH}")
00072         return False
00073     try:
00074         with open(LOCK_PATH, "w") as f:
00075             f.write(str(os.getpid()))
00076         return True
00077     except Exception as e:
00078         logger.error(f"Could not create lock: {e}")
00079         return False
00080
00081
00082 def release_lock(logger):
00083     """
00084     @brief Remove lock file after a cycle completes.
00085
00086     @param logger Logger instance for error messages
00087     """
00088     try:
00089         if os.path.exists(LOCK_PATH):
00090             os.remove(LOCK_PATH)
00091     except Exception as e:
00092         logger.error(f"Could not remove lock: {e}")
00093
00094
00095 def run_cmd(logger, cmd, cwd=None) -> int:
00096     """
00097     @brief Run a subprocess, log combined stdout/stderr, and return exit code.
00098
00099     @param logger Logger instance for command output
00100     @param cmd Command as list of strings
00101     @param cwd Working directory for the command (default: None)
00102     @return Exit code of the subprocess
00103     """
00104     logger.info(f"RUN: {' '.join(cmd)}")
00105     try:
00106         p = subprocess.run(
00107             cmd,
00108             cwd=cwd,
00109             stdout=subprocess.PIPE,
00110             stderr=subprocess.STDOUT,
00111             text=True
00112         )
00113         out = p.stdout or ""
00114         if len(out) > 5000:
```

```
00115             logger.info(out[:5000] + "\n... (truncated) ...")
00116         else:
00117             logger.info(out)
00118         return int(p.returncode)
00119     except Exception as e:
00120         logger.error(f"Command failed: {e}", exc_info=True)
00121         return 1
00122
00123
00124 def count_images_in_dir(d: str, exts=(".png", ".jpg", ".jpeg")) -> int:
00125     """
00126     @brief Count images in directory matching expected extensions.
00127
00128     @param d Directory path to scan
00129     @param exts Tuple of file extensions to match (default: (".png", ".jpg", ".jpeg"))
00130     @return Number of matching image files
00131     """
00132     if not os.path.isdir(d):
00133         return 0
00134     cnt = 0
00135     for fn in os.listdir(d):
00136         if fn.lower().endswith(exts):
00137             cnt += 1
00138     return cnt
00139
00140
00141 def needs_imageset_creation(logger) -> bool:
00142     """
00143     @brief Check if imageset exists and has enough images per camera; trigger creation if not.
00144
00145     @param logger Logger instance for status messages
00146     @return True if imageset needs to be created, False if it already exists
00147     """
00148     if not os.path.isdir(ROOT_IMAGESET):
00149         logger.info("Imageset missing: ROOT_IMAGESET not found.")
00150         return True
00151     if not os.path.isdir(FOCUS_ROOT) or not os.path.isdir(DEPTH_ROOT):
00152         logger.info("Imageset missing: focus/depth folders not found.")
00153         return True
00154
00155     for cam in CAMERA_IDS:
00156         fdir = os.path.join(FOCUS_ROOT, cam)
00157         ddir = os.path.join(DEPTH_ROOT, cam)
00158
00159         fcnt = count_images_in_dir(fdir, exts=(".png", ".jpg", ".jpeg"))
00160         dcnt = count_images_in_dir(ddir, exts=(".png", ".jpg", ".jpeg"))
00161
00162         if fcnt < MIN_POSES_PER_CAM:
00163             logger.info(f"Imageset not ready: focus/{cam} has {fcnt} images (<{MIN_POSES_PER_CAM}).")
00164             return True
00165         if dcnt < MIN_POSES_PER_CAM:
00166             logger.info(f"Imageset not ready: depth/{cam} has {dcnt} images (<{MIN_POSES_PER_CAM}).")
00167             return True
00168     logger.info("Imageset looks ready. Skipping run_imageset_creation.py.")
00169     return False
00170
00171
00172 def find_latest_run_dir(out_root: str) -> str:
00173     """
00174     @brief Return latest run_* directory inside output root, or empty string if none.
00175
00176     @param out_root Root directory containing run_* folders
00177     @return Path to the latest run directory, or empty string
00178     """
00179     if not os.path.isdir(out_root):
00180         return ""
00181     runs = []
00182     for name in os.listdir(out_root):
00183         p = os.path.join(out_root, name)
00184         if os.path.isdir(p) and name.startswith("run_"):
00185             runs.append(p)
00186     if not runs:
00187         return ""
00188     runs.sort(key=lambda p: os.path.getmtime(p))
00189     return runs[-1]
00190
00191
00192 def combined_json_path(run_dir: str) -> str:
00193     """
00194     @brief Return path to combined.json inside run directory if present.
00195
00196     @param run_dir Path to run directory
00197     @return Path to combined.json file, or empty string if not found
00198     """
00199     if not run_dir:
00200         return ""
00201     p = os.path.join(run_dir, "combined.json")
```

```
00202     return p if os.path.isfile(p) else ""
00203
00204
00205 def try_log_lifcal_with_resultlogger(logger, combined_path: str):
00206     """
00207     @brief Use ResultLogger to record LiFCal health score and update baseline.
00208
00209     @param logger Logger instance for status messages
00210     @param combined_path Path to combined.json from LiFCal
00211     @return True if logging succeeded, False otherwise
00212     """
00213     try:
00214         repo_dir = "/data/calibrationLFC"
00215         if repo_dir not in sys.path:
00216             sys.path.insert(0, repo_dir)
00217
00218         from ResultLogger import ResultLogger
00219         rl = ResultLogger(baseDir=BASELINE_DIR)
00220
00221         meta = {
00222             "runType": "recalib",
00223             "source": "periodic_scheduler",
00224             "timestamp": datetime.now().isoformat(timespec="seconds"),
00225         }
00226         score = rl.logRecalibration(combined_path, meta=meta)
00227         logger.info(f"LiFCal HealthScore logged via ResultLogger: {score:.2f}")
00228         return True
00229     except Exception as e:
00230         logger.warning(f"Could not use ResultLogger for LiFCal logging: {e}")
00231         return False
00232
00233
00234 def run_cycle(logger):
00235     """
00236     @brief Execute one scheduler cycle: prepare imageset, run LiFCal, log health.
00237
00238     @param logger Logger instance for cycle logging
00239     """
00240     # 1) Only prepare imageset if needed
00241     if needs_imageset_creation(logger):
00242         if not os.path.isfile(RUN_IMAGESET_CREATION):
00243             logger.error(f"Missing: {RUN_IMAGESET_CREATION}")
00244             return
00245
00246         rc = run_cmd(logger, [sys.executable, RUN_IMAGESET_CREATION])
00247         if rc != 0:
00248             logger.error(f"run_imageset_creation failed (rc={rc}). Skipping LiFCal.")
00249             return
00250
00251     # 2) LiFCal calibration
00252     if not os.path.isfile(RUN_LIFCAL_CALIB):
00253         logger.error(f"Missing: {RUN_LIFCAL_CALIB}")
00254         return
00255
00256     rc = run_cmd(logger, [sys.executable, RUN_LIFCAL_CALIB])
00257     if rc != 0:
00258         logger.error(f"run_LiFCal_calibration failed (rc={rc}).")
00259         return
00260
00261     # 3) locate combined.json (latest run)
00262     latest = find_latest_run_dir(LIFCAL_OUT_ROOT)
00263     cpath = combined_json_path(latest)
00264
00265     if not cpath:
00266         logger.warning("No combined.json found after LiFCal run.")
00267         return
00268
00269     logger.info(f"Latest combined.json: {cpath}")
00270
00271     # 4) log health + baseline update using your ResultLogger
00272     ok = try_log_lifcal_with_resultlogger(logger, cpath)
00273     if not ok:
00274         logger.info("LiFCal finished, but no baseline/health logging was executed.")
00275
00276
00277 def main():
00278     """
00279     @brief Periodic loop that runs LiFCal recalibration on a schedule.
00280     """
00281     logger = setup_logger()
00282     logger.info("Starting periodic LiFCal scheduler.")
00283     logger.info(f"Interval: {INTERVAL_SECONDS}s")
00284     logger.info(f"Lock: {LOCK_PATH}")
00285
00286     while True:
00287         start = time.time()
00288
```

```
00289         if acquire_lock(logger):
00290             try:
00291                 logger.info("=== CYCLE START ===")
00292                 run_cycle(logger)
00293                 logger.info("=== CYCLE END ===")
00294             finally:
00295                 release_lock(logger)
00296 
00297         elapsed = time.time() - start
00298         sleep_s = max(1.0, INTERVAL_SECONDS - elapsed)
00299         logger.info(f"Sleeping {sleep_s:.1f}s\n")
00300         time.sleep(sleep_s)
00301 
00302 
00303 if __name__ == "__main__":
00304     main()
```

6.19 external/LiFCal/LiFCal_Data/depthmap.py File Reference

Namespaces

- namespace depthmap

Functions

- depthmap.build_matchers ()
- bool depthmap.imwrite_u16 (str path, np.ndarray img_u16)
- np.ndarray depthmap.disparity (np.ndarray left_gray, np.ndarray right_gray)
- np.ndarray depthmap.to_u16_from_disp (np.ndarray disp)
- np.ndarray depthmap.fuse_disparities_median (list disp_list)
- Dict[str, np.ndarray] depthmap.generate_depthmaps_for_pose (Dict[str, str] cam_to_focus_path)

Variables

- depthmap._LEFT_MATCHER
- depthmap._RIGHT_MATCHER
- depthmap._WLS

6.20 depthmap.py

Go to the documentation of this file.

```
00001 import os
00002 import cv2
00003 import numpy as np
00004 from typing import Optional, Dict
00005 
00006 """
00007 @brief Depth map generation for LiFCal calibration using stereo matching and fusion.
00008 
00009 """
00010 
00011 def build_matchers():
00012     """
00013     @brief Build stereo matchers with SGBM and optional Weighted Least Squares (WLS) filtering.
00014 
00015     @return Tuple (left_matcher, right_matcher, wls_filter)
00016     """
00017     left_matcher = cv2.StereoSGBM_create(
00018         minDisparity=0,
00019         numDisparities=128,  # must be divisible by 16
00020         blockSize=7,
00021         P1=8 * 7 * 7,
```

```
00022         P2=32 * 7 * 7,
00023         uniquenessRatio=5,
00024         speckleWindowSize=100,
00025         speckleRange=2,
00026         disp12MaxDiff=1,
00027         preFilterCap=31,
00028         mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
00029     )
00030
00031     right_matcher = None
00032     wls = None
00033
00034     try:
00035         if hasattr(cv2, "ximgproc"):
00036             right_matcher = cv2.ximgproc.createRightMatcher(left_matcher)
00037             wls = cv2.ximgproc.createDisparityWLSFilter(left_matcher)
00038             wls.setLambda(6000)
00039             wls.setSigmaColor(1.0)
00040     except Exception:
00041         right_matcher = None
00042         wls = None
00043
00044     return left_matcher, right_matcher, wls
00045
00046
00047 _LEFT_MATCHER, _RIGHT_MATCHER, _WLS = build_matchers()
00048
00049
00050 def imwrite_u16(path: str, img_u16: np.ndarray) -> bool:
00051     """
00052     @brief Write uint16 png, create dirs automatically.
00053
00054     @param path Output file path
00055     @param img_u16 Image array in uint16 format
00056     @return True if write succeeded, False otherwise
00057     """
00058     os.makedirs(os.path.dirname(path), exist_ok=True)
00059     if img_u16.dtype != np.uint16:
00060         img_u16 = img_u16.astype(np.uint16)
00061     return bool(cv2.imwrite(path, img_u16))
00062
00063
00064 def disparity(left_gray: np.ndarray, right_gray: np.ndarray) -> np.ndarray:
00065     """
00066     @brief Compute stereo disparity map with optional WLS filtering.
00067
00068     @param left_gray Left grayscale image
00069     @param right_gray Right grayscale image
00070     @return Float32 disparity map with NaNs for invalid pixels (<=0)
00071     """
00072     dl = _LEFT_MATCHER.compute(left_gray, right_gray)  # int16 scaled by 16
00073
00074     if _RIGHT_MATCHER is not None and _WLS is not None:
00075         dr = _RIGHT_MATCHER.compute(right_gray, left_gray)
00076         d = _WLS.filter(dl, left_gray, None, dr)
00077     else:
00078         d = dl
00079
00080     d = d.astype(np.float32) / 16.0
00081     d[d <= 0] = np.nan
00082     return d
00083
00084
00085 def to_u16_from_disp(disp: np.ndarray) -> np.ndarray:
00086     """
00087     @brief Convert disparity to uint16 depth map to be compatible with LiFCal.
00088
00089     @param disp Float32 disparity map
00090     @return Uint16 depth map
00091     """
00092     disp = np.nan_to_num(disp, nan=0.0, posinf=0.0, neginf=0.0)
00093
00094     mx = float(np.percentile(disp, 99))
00095     if not np.isfinite(mx) or mx <= 1e-6:
00096         return np.zeros(disp.shape, dtype=np.uint16)
00097
00098     disp = np.clip(disp, 0.0, mx)
00099     depth16 = (disp / mx * 65535.0).astype(np.uint16)
00100     return depth16
00101
00102
00103 def fuse_disparities_median(disp_list: list) -> np.ndarray:
00104     """
00105     @brief Fuse multiple disparity maps robustly using median.
00106
00107     @param disp_list List of float32 disparity maps to fuse
00108     @return Float32 fused disparity map with 0 where no valid data exists
```

```
00109     """
00110     stack = np.stack(disp_list, axis=0)  # (K,H,W)
00111
00112     # Locations where at least one value is valid
00113     valid_any = np.any(~np.isnan(stack), axis=0)
00114
00115     # Initialize result with zeros
00116     disp_final = np.zeros(stack.shape[1:], dtype=np.float32)
00117
00118     # Compute median only where we have valid data
00119     if np.any(valid_any):
00120         disp_final[valid_any] = np.nanmedian(stack[:, valid_any], axis=0)
00121
00122     # Remove non-positive disparities
00123     disp_final[disp_final <= 0] = 0.0
00124     return disp_final
00125
00126
00127 def generate_depthmaps_for_pose(cam_to_focus_path: Dict[str, str]) -> Dict[str, np.ndarray]:
00128     """
00129     @brief Generate depth maps for all cameras in a multi-camera pose.
00130
00131     @param cam_to_focus_path Dictionary mapping camera IDs to image file paths
00132     @return Dictionary mapping camera IDs to uint16 depth maps
00133     """
00134
00135     # 1) Load images
00136     imgs = {}
00137     for cam, path in cam_to_focus_path.items():
00138         im = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
00139         if im is None:
00140             continue
00141         imgs[cam] = im
00142
00143     if len(imgs) < 2:
00144         return {}
00145
00146     # helper
00147     def opposite_name(name: str) -> Optional[str]:
00148         if name.startswith("Left"):
00149             return name.replace("Left", "Right", 1)
00150         if name.startswith("Right"):
00151             return name.replace("Right", "Left", 1)
00152         if name.startswith("Up"):
00153             return name.replace("Up", "Down", 1)
00154         if name.startswith("Down"):
00155             return name.replace("Down", "Up", 1)
00156         return None
00157
00158     depthmaps = {}
00159
00160     for cam, img in imgs.items():
00161         disp_list = []
00162
00163         if cam == "Center":
00164             # Center uses all other views
00165             for other_cam, other_img in imgs.items():
00166                 if other_cam == "Center":
00167                     continue
00168                 d = disparity(img, other_img)
00169                 # Keep only if there is any valid disparity
00170                 if np.any(~np.isnan(d)):
00171                     disp_list.append(d)
00172
00173         else:
00174             # Opposite direction if available
00175             opp = opposite_name(cam)
00176             if opp is not None and opp in imgs:
00177                 if cam.startswith("Right") or cam.startswith("Down"):
00178                     d = disparity(imgs[opp], img)
00179                 else:
00180                     d = disparity(img, imgs[opp])
00181
00182                 if np.any(~np.isnan(d)):
00183                     disp_list.append(d)
00184
00185             # Add Center view
00186             if "Center" in imgs:
00187                 d = disparity(img, imgs["Center"])
00188                 if np.any(~np.isnan(d)):
00189                     disp_list.append(d)
00190
00191         if not disp_list:
00192             # No valid disparity found; skip
00193             continue
00194
00195         # Robust fusion
```

```
00196         disp_final = fuse_disparities_median(disp_list)
00197
00198         # Convert to uint16 depth map
00199         depth16 = to_u16_from_disp(disp_final)
00200         depthmaps[cam] = depth16
00201
00202     return depthmaps
```

6.21 external/LiFCal/LiFCal_Data/lifcal_to_json.py File Reference

Namespaces

- namespace lifcal_to_json

Functions

- dict lifcal_to_json.parse_protocol (str protocol_path)
- Dict[str, Any] lifcal_to_json.parse_extrinsics_xml (str xml_path)
- str lifcal_to_json.export_lifcal_parameters_json_in_place (str result_dir, str cam_id, str image_dir, str run_↩ type="recalib", float pixel_size_mm_fallback=0.0055, str protocol_name="calibrationProtocol.txt", str extr↩ _name="extrinsicOrientations.xml", str out_name="parameters.json", Optional[float] pixel_size_mm=None, Optional[float] fallback_cx=None, Optional[float] fallback_cy=None)
- str lifcal_to_json.build_combined_parameters_json (str run_root, list camera_ids, str out_name="combined.↩ json")

6.22 lifcal_to_json.py

Go to the documentation of this file.

```
00001 import os
00002 import re
00003 import json
00004 import xml.etree.ElementTree as ET
00005 from typing import Dict, Any, Optional
00006
00007 import numpy as np
00008 import cv2
00009
00010 """
00011 @brief code to parse LiFCal calibration protocol and extrinsic XML files, and export parameters.json
      for each camera, as well as a combined.json for all cameras. This is used to convert LiFCal's output
      into a structured JSON format that can be easily consumed for further analysis or processing.
00012
00013 """
00014
00015 def parse_protocol(protocol_path: str) -> dict:
00016     """
00017     @brief Parse LiFCal calibration protocol text file.
00018
00019     @return Dictionary containing parsed calibration parameters and reprojection errors
00020     """
00021     txt = open(protocol_path, "r", encoding="utf-8", errors="ignore").read()
00022
00023     def find_float(pattern: str, default=None) -> Optional[float]:
00024         m = re.search(pattern, txt, re.MULTILINE)
00025         if not m:
00026             return default
00027         try:
00028             return float(m.group(1))
00029         except Exception:
00030             return default
00031
00032     pixel_size = find_float(r"Pixel Size:\s*([0-9.+-eE]+)\s*mm", None)
00033
00034     fL  = find_float(r"^\s*fL\s*:\s*([0-9.+-eE]+)\s*$", None)
00035     bL0 = find_float(r"^\s*bL0\s*:\s*([0-9.+-eE]+)\s*$", None)
00036     B   = find_float(r"^\s*B\s*:\s*([0-9.+-eE]+)\s*$", None)
00037     cx  = find_float(r"^\s*cx\s*:\s*([0-9.+-eE]+)\s*$", None)
```

```
00038     cy  = find_float(r"^\s*cy\s*:\s*([0-9.+-eE]+)\s*$", None)
00039
00040     a0 = find_float(r"^\s*a0\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00041     a1 = find_float(r"^\s*a1\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00042     b0 = find_float(r"^\s*b0\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00043     b1 = find_float(r"^\s*b1\s*:\s*([0-9.+-eE]+)\s*$", 0.0)
00044
00045     x_std = find_float(r"std\.\s*Dev\.\s*x:\s*([0-9.+-eE]+)", None)
00046     y_std = find_float(r"std\.\s*Dev\.\s*y:\s*([0-9.+-eE]+)", None)
00047     x_mae = find_float(r"mae\s*x:\s*([0-9.+-eE]+)", None)
00048     y_mae = find_float(r"mae\s*y:\s*([0-9.+-eE]+)", None)
00049
00050     return {
00051         "pixelSizeMm": pixel_size,
00052         "fL_mm": fL,
00053         "bL0_mm": bL0,
00054         "B_mm": B,
00055         "cx_px": cx,
00056         "cy_px": cy,
00057         "a0": a0,
00058         "a1": a1,
00059         "b0": b0,
00060         "b1": b1,
00061         "reprojection": {
00062             "x_std": x_std,
00063             "y_std": y_std,
00064             "x_mae": x_mae,
00065             "y_mae": y_mae
00066         }
00067     }
00068
00069
00070 def parse_extrinsics_xml(xml_path: str) -> Dict[str, Any]:
00071     """
00072     @brief Parse LiFCal extrinsic orientations XML file.
00073
00074     @param xml_path Path to extrinsicOrientations.xml file
00075     @return Dictionary mapping pose keys to rotation and translation data
00076     """
00077     root = ET.parse(xml_path).getroot()
00078     out: Dict[str, Any] = {}
00079
00080     for frame in root.findall("Frame"):
00081         fid = int(frame.attrib.get("id"))
00082         pose_key = "pose%03d" % fid
00083
00084         rot = frame.find("Rotation")
00085         tra = frame.find("Translation")
00086
00087         rvec = [0.0, 0.0, 0.0]
00088         tvec = [0.0, 0.0, 0.0]
00089
00090         if rot is not None:
00091             for c in rot.findall("Coeff"):
00092                 i = int(c.attrib["i"])
00093                 rvec[i] = float(c.text.strip())
00094
00095         if tra is not None:
00096             for c in tra.findall("Coeff"):
00097                 i = int(c.attrib["i"])
00098                 tvec[i] = float(c.text.strip())
00099
00100         rvec_np = np.array(rvec, dtype=np.float64).reshape(3, 1)
00101         R, _ = cv2.Rodrigues(rvec_np)
00102
00103         out[pose_key] = {
00104             "rotationMatrix": R.astype(float).tolist(),
00105             "rotationVector": [[float(rvec[0])], [float(rvec[1])], [float(rvec[2])]],
00106             "translationVector": [[float(tvec[0])], [float(tvec[1])], [float(tvec[2])]],
00107         }
00108
00109     return out
00110
00111
00112 def export_lifcal_parameters_json_in_place(
00113     result_dir: str,
00114     cam_id: str,
00115     image_dir: str,
00116     run_type: str = "recalib",
00117     pixel_size_mm_fallback: float = 0.0055,
00118     protocol_name: str = "calibrationProtocol.txt",
00119     extr_name: str = "extrinsicOrientations.xml",
00120     out_name: str = "parameters.json",
00121     # Aliases (kept for runner compatibility)
00122     pixel_size_mm: Optional[float] = None,
00123     fallback_cx: Optional[float] = None,  # accepted for compatibility, not used
00124     fallback_cy: Optional[float] = None,  # accepted for compatibility, not used
```

```
00125 ) -> str:
00126     """
00127     @brief Write LiFCal parameters.json for a single camera.
00128 
00129     @param result_dir Directory containing LiFCal output files
00130     @param cam_id Camera identifier
00131     @param image_dir Directory containing input images
00132     @param run_type Type of calibration run (default: "recalib")
00133     @param pixel_size_mm_fallback Fallback pixel size in mm (default: 0.0055)
00134     @param protocol_name Protocol filename (default: "calibrationProtocol.txt")
00135     @param extr_name Extrinsics filename (default: "extrinsicOrientations.xml")
00136     @param out_name Output JSON filename (default: "parameters.json")
00137     @param pixel_size_mm Optional pixel size override
00138     @param fallback_cx Optional fallback cx (compatibility only, not used)
00139     @param fallback_cy Optional fallback cy (compatibility only, not used)
00140     @return Path to written parameters.json file
00141     """
00142     if pixel_size_mm is not None:
00143         pixel_size_mm_fallback = float(pixel_size_mm)
00144 
00145     protocol_path = os.path.join(result_dir, protocol_name)
00146     extr_path = os.path.join(result_dir, extr_name)
00147 
00148     if not os.path.isfile(protocol_path):
00149         raise FileNotFoundError(protocol_path)
00150     if not os.path.isfile(extr_path):
00151         raise FileNotFoundError(extr_path)
00152 
00153     protocol = parse_protocol(protocol_path)
00154     extr_by_pose = parse_extrinsics_xml(extr_path)
00155 
00156     # Ensure pixel size is available
00157     px = protocol.get("pixelSizeMm", None)
00158     if px is None or (not np.isfinite(px)) or px <= 0:
00159         protocol["pixelSizeMm"] = float(pixel_size_mm_fallback)
00160 
00161     # Compact reprojection error (average MAE)
00162     rep = protocol.get("reprojection", {})
00163     x_mae = rep.get("x_mae", None)
00164     y_mae = rep.get("y_mae", None)
00165     avg_mae = None
00166     if x_mae is not None and y_mae is not None and np.isfinite(x_mae) and np.isfinite(y_mae):
00167         avg_mae = float((x_mae + y_mae) / 2.0)
00168 
00169     data = {
00170         "meta": {
00171             "runType": run_type,
00172             "numPoses": int(len(extr_by_pose)),
00173             "imageDir": image_dir,
00174             "cameraIds": [cam_id],
00175             "bundleAdjust": True,
00176         },
00177         "state": {
00178             "parameters": {
00179                 cam_id: {
00180                     "pixelSizeMm": protocol.get("pixelSizeMm"),
00181                     "fL_mm": protocol.get("fL_mm"),
00182                     "bL0_mm": protocol.get("bL0_mm"),
00183                     "B_mm": protocol.get("B_mm"),
00184                     "cx_px": protocol.get("cx_px"),
00185                     "cy_px": protocol.get("cy_px"),
00186                     "a0": protocol.get("a0"),
00187                     "a1": protocol.get("a1"),
00188                     "b0": protocol.get("b0"),
00189                     "b1": protocol.get("b1"),
00190                     "reprojection": rep,
00191                     "reprojectionAvgMae": avg_mae,
00192                 }
00193             },
00194             "extrinsicsByPose": {cam_id: extr_by_pose},
00195             "timeStamp": None
00196         }
00197     }
00198 
00199     out_path = os.path.join(result_dir, out_name)
00200     with open(out_path, "w", encoding="utf-8") as f:
00201         json.dump(data, f, indent=2)
00202 
00203     return out_path
00204 
00205 
00206 def build_combined_parameters_json(run_root: str, camera_ids: list, out_name: str = "combined.json")
      -> str:
00207     """
00208     @brief Combine per-camera parameters.json files into a single combined.json.
00209 
00210     @param run_root Root directory containing per-camera subdirectories
```

```
00211     @param camera_ids List of camera identifiers
00212     @param out_name Output filename (default: "combined.json")
00213     @return Path to written combined.json file
00214     """
00215     combined = {
00216         "meta": {
00217             "runType": "combined",
00218             "cameraIds": camera_ids,
00219         },
00220         "state": {
00221             "parameters": {},
00222             "extrinsicsByPose": {},
00223             "timeStamp": None
00224         }
00225     }
00226
00227     found_any = False
00228
00229     for cam in camera_ids:
00230         cam_dir = os.path.join(run_root, cam)
00231         jpath = os.path.join(cam_dir, "parameters.json")
00232
00233         if not os.path.isfile(jpath):
00234             continue
00235
00236         with open(jpath, "r", encoding="utf-8") as f:
00237             data = json.load(f)
00238
00239         st = data.get("state", {})
00240         params = st.get("parameters", {})
00241         extr_pose = st.get("extrinsicsByPose", {})
00242
00243         if cam in params:
00244             combined["state"]["parameters"][cam] = params[cam]
00245             found_any = True
00246
00247         if cam in extr_pose:
00248             combined["state"]["extrinsicsByPose"][cam] = extr_pose[cam]
00249             found_any = True
00250
00251     if not found_any:
00252         raise RuntimeError("No parameters.json found to combine.")
00253
00254     out_path = os.path.join(run_root, out_name)
00255     with open(out_path, "w", encoding="utf-8") as f:
00256         json.dump(combined, f, indent=2)
00257
00258     return out_path
```

6.23 external/LiFCal/LiFCal_Data/run_imageset_creation.py File Reference

Namespaces

- namespace run_imageset_creation

Functions

- run_imageset_creation.ensure_camera_subfolders (str base_dir)
- run_imageset_creation.group_focus_images_by_pose ()
- run_imageset_creation.copy_focus_into_camera_folders (str pose, dict cam_to_path)
- run_imageset_creation.cleanup_focus_root ()
- run_imageset_creation.main ()

Variables

- str run_imageset_creation.ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
- run_imageset_creation.FOCUS_DIR = os.path.join(ROOT, "focus")
- run_imageset_creation.DEPTH_DIR = os.path.join(ROOT, "depth")
- bool run_imageset_creation.CLEAN_FOCUS_ROOT_AFTER = True
- list run_imageset_creation.CAMERA_IDS
- run_imageset_creation.PATTERN

6.24 run_imageset_creation.py

Go to the documentation of this file.

```
00001 import os
00002 import re
00003 import shutil
00004 from collections import defaultdict
00005 import depthmap
00006
00007 """
00008 @brief Organizes focus images by camera and generates depth maps for LiFCal recalibration.
00009
00010 """
00011
00012 ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
00013 FOCUS_DIR = os.path.join(ROOT, "focus")
00014 DEPTH_DIR = os.path.join(ROOT, "depth")
00015
00016 # If True: original files in focus root directory are deleted after sorting
00017 CLEAN_FOCUS_ROOT_AFTER = True
00018
00019 CAMERA_IDS = [
00020     "Center", "Up1", "Up2", "Up3",
00021     "Down1", "Down2", "Down3",
00022     "Left1", "Left2", "Left3",
00023     "Right1", "Right2", "Right3"
00024 ]
00025
00026 # Expected filenames: pose000_Center.png ... pose000_Down3.png
00027 PATTERN = re.compile(
00028     r"^(pose\d+)_(Center|Up[123]|Down[123]|Left[123]|Right[123])\.(png|jpg|jpeg)$",
00029     re.IGNORECASE
00030 )
00031
00032 def ensure_camera_subfolders(base_dir: str):
00033     """
00034     @brief Create subdirectory for each camera ID under base_dir.
00035
00036     @param base_dir Base directory path
00037     """
00038     os.makedirs(base_dir, exist_ok=True)
00039     for cid in CAMERA_IDS:
00040         os.makedirs(os.path.join(base_dir, cid), exist_ok=True)
00041
00042 def group_focus_images_by_pose():
00043     """
00044     @brief Group focus images by pose ID from flat directory structure.
00045
00046     @return Dictionary mapping pose IDs to camera-path dictionaries
00047     """
00048     pose_to_cam = defaultdict(dict)
00049     for fn in sorted(os.listdir(FOCUS_DIR)):
00050         m = PATTERN.match(fn)
00051         if not m:
00052             continue
00053         pose, cam, ext = m.group(1), m.group(2), m.group(3)
00054         pose_to_cam[pose][cam] = os.path.join(FOCUS_DIR, fn)
00055     return pose_to_cam
00056
00057 def copy_focus_into_camera_folders(pose: str, cam_to_path: dict):
00058     """
00059     @brief Copy focus images into camera-specific subdirectories.
00060
00061     @param pose Pose identifier string (e.g., "pose000")
00062     @param cam_to_path Dictionary mapping camera IDs to source image paths
00063     """
00064     for cam, src in cam_to_path.items():
00065         ext = os.path.splitext(src)[1].lower()
00066         dst = os.path.join(FOCUS_DIR, cam, f"{pose}_{cam}{ext}")
00067         if os.path.abspath(src) != os.path.abspath(dst):
00068             shutil.copy2(src, dst)
00069
00070 def cleanup_focus_root():
00071     """
00072     @brief Delete only loose files directly in focus/ (not in subdirectories).
00073     """
00074     for fn in os.listdir(FOCUS_DIR):
00075         p = os.path.join(FOCUS_DIR, fn)
00076         if os.path.isfile(p) and PATTERN.match(fn):
00077             os.remove(p)
00078
00079 def main():
00080     """
00081     @brief Main pipeline: organize focus images and generate depth maps.
00082     """
```

```
00083     if not os.path.isdir(FOCUS_DIR):
00084         raise SystemExit(f"focus folder not found: {FOCUS_DIR}")
00085 
00086     ensure_camera_subfolders(FOCUS_DIR)
00087     ensure_camera_subfolders(DEPTH_DIR)
00088 
00089     pose_to_cam = group_focus_images_by_pose()
00090     if not pose_to_cam:
00091         raise SystemExit("No focus images found. Expected: pose000_Center.png etc.")
00092 
00093     poses = sorted(pose_to_cam.keys())
00094     print("Found poses:", len(poses))
00095 
00096     for i, pose in enumerate(poses, start=1):
00097         cam_to_focus_path = pose_to_cam[pose]
00098 
00099         # Sort focus images into camera folders
00100         copy_focus_into_camera_folders(pose, cam_to_focus_path)
00101 
00102         # Generate depth maps for this pose
00103         depthmaps = depthmap.generate_depthmaps_for_pose(cam_to_focus_path)
00104 
00105         # Save depth maps: depth/<CamId>/<pose>_<CamId>_depth.png
00106         for cam, depth_u16 in depthmaps.items():
00107             out_path = os.path.join(DEPTH_DIR, cam, f"{pose}_{cam}_depth.png")
00108             ok = depthmap.imwrite_u16(out_path, depth_u16)
00109             if not ok:
00110                 print("FAILED write:", out_path)
00111 
00112         print(f"[{i}/{len(poses)}] {pose}: views={len(cam_to_focus_path)} depthmaps={len(depthmaps)}")
00113 
00114     if CLEAN_FOCUS_ROOT_AFTER:
00115         cleanup_focus_root()
00116 
00117     print("DONE.")
00118     print("Focus by camera:", FOCUS_DIR)
00119     print("Depth by camera:", DEPTH_DIR)
00120 
00121 if __name__ == "__main__":
00122     main()
```

6.25 external/LiFCal/LiFCal_Data/run_LiFCal_calibration.py File Reference

Namespaces

- namespace run_LiFCal_calibration

Functions

- str run_LiFCal_calibration.read_text (str path)
- None run_LiFCal_calibration.write_text (str path, str s)
- str run_LiFCal_calibration.replace_yaml_key_line (str text, str key, str new_val)
- str run_LiFCal_calibration.patch_settings_yaml (str template_text, str focus_dir, str depth_dir, str mla_xml)
- int run_LiFCal_calibration.run_lifcal_recalib (str settings_path, str fixed_params_path, str store_dir)
- str run_LiFCal_calibration.find_results_folder (str store_dir)
- None run_LiFCal_calibration.copy_essentials (str result_dir, str out_cam_dir)
- run_LiFCal_calibration.main ()

Variables

- str run_LiFCal_calibration.LIFCAL_BIN = "/data/external/LiFCal/build/bin/LiFCal"
- str run_LiFCal_calibration.SETTINGS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Settings.yaml"
- str run_LiFCal_calibration.FIXED_PARAMS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Fixed_↩ Paramerters.txt"

- str run_LiFCal_calibration.MLA_CALIB_XML = "/data/external/LiFCal/LiFCal_Data/Recalibration/MLA_↩ Calibration.xml"
- str run_LiFCal_calibration.ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_↩ Imageset"
- run_LiFCal_calibration.FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")
- run_LiFCal_calibration.DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")
- str run_LiFCal_calibration.OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/CalibrationBy↩ Camera"
- list run_LiFCal_calibration.CAMERA_IDS

6.26 run_LiFCal_calibration.py

Go to the documentation of this file.

```
00001 import os
00002 import re
00003 import shutil
00004 import subprocess
00005 from datetime import datetime
00006 import lifcal_to_json
00007
00008 """
00009 @brief Runs LiFCal recalibration for all cameras and exports results to JSON.
00010 """
00011
00012 # Configuration paths
00013 LIFCAL_BIN = "/data/external/LiFCal/build/bin/LiFCal"
00014
00015 SETTINGS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Settings.yaml"
00016 FIXED_PARAMS = "/data/external/LiFCal/LiFCal_Data/Recalibration/Fixed_Paramerters.txt"
00017 MLA_CALIB_XML = "/data/external/LiFCal/LiFCal_Data/Recalibration/MLA_Calibration.xml"
00018
00019 ROOT_IMAGESET = "/data/external/LiFCal/LiFCal_Data/Recalibration/LiFCal_Imageset"
00020 FOCUS_ROOT = os.path.join(ROOT_IMAGESET, "focus")    # focus/<CamId>/
00021 DEPTH_ROOT = os.path.join(ROOT_IMAGESET, "depth")    # depth/<CamId>/
00022
00023 OUT_ROOT = "/data/external/LiFCal/LiFCal_Data/Recalibration/CalibrationByCamera"
00024
00025 CAMERA_IDS = [
00026     "Center", "Up1", "Up2", "Up3",
00027     "Down1", "Down2", "Down3",
00028     "Left1", "Left2", "Left3",
00029     "Right1", "Right2", "Right3"
00030 ]
00031
00032
00033 def read_text(path: str) -> str:
00034     """
00035     @brief Read text file with UTF-8 encoding.
00036
00037     @param path Path to the text file
00038     @return File content as string
00039     """
00040     with open(path, "r", encoding="utf-8", errors="replace") as f:
00041         return f.read()
00042
00043
00044 def write_text(path: str, s: str) -> None:
00045     """
00046     @brief Write text to file, creating directories if needed.
00047
00048     @param path Output file path
00049     @param s Text content to write
00050     """
00051     os.makedirs(os.path.dirname(path), exist_ok=True)
00052     with open(path, "w", encoding="utf-8") as f:
00053         f.write(s)
00054
00055
00056 def replace_yaml_key_line(text: str, key: str, new_val: str) -> str:
00057     """
00058     @brief Replace YAML key-value lines.
00059
00060     @param text YAML text content
00061     @param key YAML key to find and replace
00062     @param new_val New value to set (will be quoted)
00063     @return Modified YAML text
00064     """
```

```
00065     pattern = re.compile(rf"^({re.escape(key)}\s*:\s*)(.*)$", re.MULTILINE)
00066     return pattern.sub(rf'\1"{new_val}"', text)
00067
00068
00069 def patch_settings_yaml(template_text: str, focus_dir: str, depth_dir: str, mla_xml: str) -> str:
00070     """
00071     @brief Update YAML settings with camera-specific paths.
00072
00073     @param template_text Original YAML template content
00074     @param focus_dir Path to focus images directory
00075     @param depth_dir Path to depth maps directory
00076     @param mla_xml Path to MLA calibration XML file
00077     @return Modified YAML text with updated paths
00078     """
00079     out = template_text
00080     out = replace_yaml_key_line(out, "Path.totalFocusImages", focus_dir)
00081     out = replace_yaml_key_line(out, "Path.virtualDepthData", depth_dir)
00082     out = replace_yaml_key_line(out, "Path.microLensCalibration", mla_xml)
00083     return out
00084
00085
00086 def run_lifcal_recalib(settings_path: str, fixed_params_path: str, store_dir: str) -> int:
00087     """
00088     @brief Run LiFCal recalibration and return exit code.
00089
00090     @param settings_path Path to Settings.yaml file
00091     @param fixed_params_path Path to fixed parameters file
00092     @param store_dir Directory where LiFCal will store results
00093     @return Exit code from LiFCal process
00094     """
00095     os.makedirs(store_dir, exist_ok=True)
00096
00097     cmd = [LIFCAL_BIN, "recalib", settings_path, fixed_params_path]
00098
00099     proc = subprocess.Popen(
00100         cmd,
00101         stdin=subprocess.PIPE,
00102         stdout=subprocess.PIPE,
00103         stderr=subprocess.STDOUT,
00104         text=True,
00105         cwd=os.path.dirname(LIFCAL_BIN),
00106     )
00107
00108     out, _ = proc.communicate(input=f"y\n{store_dir}\n")
00109     write_text(os.path.join(store_dir, "lifcal_stdout.log"), out)
00110
00111     return proc.returncode
00112
00113
00114 def find_results_folder(store_dir: str) -> str:
00115     """
00116     @brief Find LiFCal results folder.
00117
00118     @param store_dir LiFCal output directory
00119     @return Path to results folder (typically Calibration_Results_... subdirectory)
00120     """
00121     if not os.path.isdir(store_dir):
00122         raise RuntimeError(f"store_dir does not exist: {store_dir}")
00123
00124     subdirs = []
00125     for name in os.listdir(store_dir):
00126         p = os.path.join(store_dir, name)
00127         if os.path.isdir(p):
00128             subdirs.append(p)
00129
00130     if not subdirs:
00131         return store_dir
00132
00133     subdirs.sort(key=lambda p: os.path.getmtime(p))
00134     return subdirs[-1]
00135
00136
00137 def copy_essentials(result_dir: str, out_cam_dir: str) -> None:
00138     """
00139     @brief Copy calibrationProtocol.txt and extrinsicOrientations.xml to output directory.
00140
00141     @param result_dir Source directory containing LiFCal results
00142     @param out_cam_dir Destination directory for essential files
00143     """
00144     proto = os.path.join(result_dir, "calibrationProtocol.txt")
00145     extr = os.path.join(result_dir, "extrinsicOrientations.xml")
00146
00147     missing = []
00148     if not os.path.isfile(proto):
00149         missing.append("calibrationProtocol.txt")
00150     if not os.path.isfile(extr):
00151         missing.append("extrinsicOrientations.xml")
```

```
00152     if missing:
00153         raise RuntimeError(f"Missing files in {result_dir}: {missing}")
00154 
00155     shutil.copy2(proto, os.path.join(out_cam_dir, "calibrationProtocol.txt"))
00156     shutil.copy2(extr, os.path.join(out_cam_dir, "extrinsicOrientations.xml"))
00157 
00158 
00159 def main():
00160     """
00161     @brief Main pipeline: run LiFCal for each camera, export JSON, and log health.
00162     """
00163     if not os.path.isfile(LIFCAL_BIN):
00164         raise SystemExit(f"LiFCal binary not found: {LIFCAL_BIN}")
00165     if not os.path.isfile(SETTINGS):
00166         raise SystemExit(f"Settings template not found: {SETTINGS}")
00167     if not os.path.isfile(FIXED_PARAMS):
00168         raise SystemExit(f"Fixed parameters file not found: {FIXED_PARAMS}")
00169     if not os.path.isfile(MLA_CALIB_XML):
00170         raise SystemExit(f"MLA_Calibration.xml not found: {MLA_CALIB_XML}")
00171 
00172     templ = read_text(SETTINGS)
00173 
00174     timestamp = datetime.now().strftime("%Y_%m_%d_%H%M%S")
00175     session_out = os.path.join(OUT_ROOT, f"run_{timestamp}")
00176     os.makedirs(session_out, exist_ok=True)
00177 
00178     print("Output:", session_out)
00179 
00180     for cam in CAMERA_IDS:
00181         focus_dir = os.path.join(FOCUS_ROOT, cam)
00182         depth_dir = os.path.join(DEPTH_ROOT, cam)
00183 
00184         if not os.path.isdir(focus_dir):
00185             print(f"[SKIP] {cam}: focus dir missing: {focus_dir}")
00186             continue
00187         if not os.path.isdir(depth_dir):
00188             print(f"[SKIP] {cam}: depth dir missing: {depth_dir}")
00189             continue
00190 
00191         cam_out = os.path.join(session_out, cam)
00192         os.makedirs(cam_out, exist_ok=True)
00193 
00194         settings_cam = os.path.join(cam_out, "Settings.yaml")
00195         patched = patch_settings_yaml(templ, focus_dir, depth_dir, MLA_CALIB_XML)
00196         write_text(settings_cam, patched)
00197 
00198         store_dir = os.path.join(cam_out, "lifcal_store")
00199         rc = run_lifcal_recalib(settings_cam, FIXED_PARAMS, store_dir)
00200 
00201         if rc != 0:
00202             print(f"[FAIL] {cam}: LiFCal returncode={rc} (siehe {store_dir}/lifcal_stdout.log)")
00203             continue
00204 
00205         result_dir = find_results_folder(store_dir)
00206 
00207         try:
00208             copy_essentials(result_dir, cam_out)   # ???iert protocol + xml nach cam_out
00209         except Exception as e:
00210             print(f"[FAIL] {cam}: {e}")
00211             print(f"       check log: {store_dir}/lifcal_stdout.log")
00212             continue
00213 
00214         # store dir wegräumen
00215         shutil.rmtree(store_dir, ignore_errors=True)
00216 
00217         # ? JSON EXPORT: result_dir muss cam_out sein, weil DA liegen jetzt die 2 Dateien
00218         try:
00219             json_path = lifcal_to_json.export_lifcal_parameters_json_in_place(
00220                 result_dir=cam_out,
00221                 cam_id=cam,
00222                 image_dir="LiFCal_Imageset",
00223                 run_type="recalib",
00224                 pixel_size_mm=0.0055,
00225                 fallback_cx=640.0,  # compatibility only
00226                 fallback_cy=360.0,  # compatibility only
00227                 out_name="parameters.json"
00228             )
00229         except Exception as e:
00230             print(f"[WARN] {cam}: JSON export failed: {e}")
00231 
00232     # Build combined JSON from all cameras
00233     try:
00234         combined_path = lifcal_to_json.build_combined_parameters_json(
00235             run_root=session_out,
00236             camera_ids=CAMERA_IDS,
00237             out_name="combined.json"
00238         )
```

```
00239         print("[OK] COMBINED:", combined_path)
00240     except Exception as e:
00241         print("[WARN] combined.json not created:", e)
00242 
00243     # Log health score and update baseline
00244     try:
00245         import sys
00246         CALIB_ROOT = "/data/calibrationLFC"
00247         if CALIB_ROOT not in sys.path:
00248             sys.path.insert(0, CALIB_ROOT)
00249 
00250         from ResultLogger import ResultLogger
00251         logger = ResultLogger(baseDir="/data/baseline")
00252         score = logger.logRecalibration(
00253             combined_json_path=combined_path,
00254             meta={
00255                 "runType": "recalib",
00256                 "imageDir": "LiFCal_Imageset",
00257                 "bundleAdjust": True
00258             }
00259         )
00260         print(f"[OK] HealthScore (lifcal) = {score:.2f}")
00261     except Exception as e:
00262         print("[WARN] LiFCal logging/health failed:", e)
00263 
00264 
00265 if __name__ == "__main__":
00266     main()
```